

MSc Mathematics
Track: Biomedical mathematics

Internship Report

Variations of Lasso algorithm for high dimensional data, with a microRNA application

by

Maria Marinaki

April 28, 2026

First supervisor: Wessel van Wieringen

Second reader: Paulo Jorge de Andrade Serra

External Supervisor: Keano Samaritakis, Stan Goertz

Host Organization: ToxGenSolutions BV

ToxGenSolutions

Department of Mathematics
Faculty of Science

Summary

In this thesis, we will work on the mathematical tools required for microRNA analysis and Mild Cognitive Impairment (MCI) prediction, in a high-dimensional setting where interaction effects may be relevant. Even for a panel of 20 microRNAs, introducing pair-wise interactions results in 190 additional candidates, so model building quickly becomes a variable selection problem. Because microRNA studies, like many biomedical studies, demand interpretable models, we compare the following three approaches. A baseline linear regression model built via a forward-selection type of algorithm (iFORM). A Lasso variation that varies from the typical way of building the solution path (BT). And, a Lasso variation that changes the model to incorporate interactions as modifiers (PL). Our results-workflow is as follows:

- First, we compare the performance and variable selection ability of the methods on simulated data, with particular attention to the selection of interaction terms, which is the most challenging aspect.
- Second, we assess how the best-performing methods from the simulations transfer to the balanced case-study dataset, and we benchmark them against a standard logistic regression model. We evaluate performance using two criteria: the stability of the selected variables and predictive accuracy.
- Third, we draw conclusions on the suitability of the considered methods for the balanced case-study dataset.
- Finally, we put these conclusions into perspective by evaluating the most suitable method on the original, imbalanced dataset, and we provide recommendations to the company for future continuation of the study.

Title: Variations of Lasso algorithm for high dimensional data, with a microRNA application

Author: Maria Marinaki, m.marinaki@vu.nl, 2746688

First supervisor: Wessel van Wieringen

Second reader: Paulo Jorge de Andrade Serra

External Supervisor: Keano Samaritakis, Stan Goertz

Host Organization: ToxGenSolutions BV

Date: April 28, 2026

Department of Mathematics

Vrije Universiteit Amsterdam

de Boelelaan 1111, 1081 HV Amsterdam

<http://www.math.vu.nl/>

Contents

1. Introduction	5
2. Modeling with Interactions in a High-Dimensional Data Scenario	6
2.1. High-Dimensional Data	6
2.2. Linear regression	6
2.3. Estimating coefficients on the linear regression model	8
2.4. Interaction Linear Regression	8
2.4.1. Introduction	8
2.4.2. Example	10
2.4.3. Hierarchy condition	11
2.5. Minimization problem & Penalization method	11
2.5.1. Introduction	11
2.5.2. Lasso	11
2.5.3. Group Lasso	13
2.5.4. Purpose of penalization & conclusion	13
2.6. Lasso Regression Solution	14
2.6.1. Introduction	14
2.6.2. Orthonormal design matrix	14
2.6.3. General design matrix	17
2.7. Extension to logistic regression	21
3. Methods	25
3.1. Forward Selection Method (FS)	25
3.1.1. Forward Selection (without interaction terms)	27
3.1.2. interaction Forward Selection (iFOR)	27
3.1.3. Extension to logistic regression	33
3.2. Backtracking with Lasso (BT)	34
3.2.1. Overview	34
3.2.2. Example	35
3.2.3. Review	40
3.2.4. Extension to logistic regression	40
3.3. Pliable Lasso (PL)	41
3.3.1. Method Introduction	41
3.3.2. Simple example to understand modifiers	44
3.3.3. Estimating coefficients	46
3.3.4. Pliable Lasso Algorithm	46
3.3.5. Extension to logistic regression	52
4. Results & Discussion	53
4.1. Introduction	53
4.2. Data	53

4.3. Method Selection	54
4.3.1. Introduction	54
4.3.2. Metrics	55
4.3.3. Model Application	55
4.3.4. Analyzing outcomes	57
4.4. Best Method	58
4.4.1. Stability Analysis	59
4.4.2. Case-study Modeling Choice	59
4.5. Deliverables to the Company	65
4.5.1. Interpretation of the balanced dataset analysis	65
4.5.2. L1_stab's fit on the unbalanced data	66
4.5.3. Takeaway	67
5. Conclusion	69
6. Final Note	71
6.1. Code	71
6.2. Acknowledgments	71
Bibliography	71
A. Detailed Calculations	75
A.1. Back-transformation from Scaling coefficients	75
A.2. Orthonormal design matrix allows coefficients decoupling	77
A.3. Soft-thresholding Lasso solution with decoupled coefficients	78
A.4. Derivation of likelihood function	79
A.5. Completion of squares	79
A.6. FS & iFOR complexity	80
A.7. Shrinkage area of θ_j for Pliable Lasso (Nested Soft-Thresholding)	81
A.8. Deriving the solution system 3.20 from Equation 3.19.	82
B. Simulation Details	83
B.1. Auto-Regressive Correlations AR(1)	83
B.2. Simulated Data	83
C. Additional Tables & Figures	85

1. Introduction

In 1993, researchers first noticed a tiny RNA molecule playing a regulatory role, rather than producing a protein, in worms[19]. A few years later, in 2000, researchers identified a second such molecule in the same species and quickly realized that similar small RNAs exist in various animals[27]. By 2001, these regulatory RNAs, now called microRNAs, were detected and cloned from human cells, opening the door to detailed laboratory experiments[17]. In 2005, expression studies showed that microRNAs are often produced together and can influence each other's levels[3]. Advances in detecting technologies then expanded the catalog to over 2,600 human microRNAs by March 2018[1]. Even a modest analysis of 20 microRNAs (like with the case-study dataset) involves 190 possible pairs and with those plus the 20 individual effects, a total of 210 predictors. So, it becomes essential to use methods that can handle large numbers of variables and pick out only the most relevant.

In Chapter 2, we lay out the foundations of this study, the foundations of regression modeling and introduce the Lasso method. The Lasso method adds a penalty term to the coefficient estimation function to encourage simpler models with a minimized number of variables. This penalty both keeps interpretability and guards against overfitting when many predictors are under consideration. In addition, a variation of linear regression that we will use later to model binary outcomes, called logistic regression, will be analyzed.

In Chapter 3, we show our baseline method and how to extend Lasso to capture interaction terms effectively. This chapter starts from the simplest method of variable selection called Forward Selection. We then extend the method to its more sophisticated counterpart that treats interactions and is suitable for High Dimensional Datasets (HDD), called iFORM. This will be our baseline model. Then, we move on to the Lasso variants. We describe a backtracking strategy that iteratively adds promising interaction terms, and we discuss a Lasso penalty variant that allows coefficients to vary with other factors. These extensions make it feasible to handle datasets with a vast number of potential interactions.

In Chapter 4, we evaluate the methods and translate the findings to the case-study. We first run a simulation study in which the true main and interaction effects are known, allowing direct comparison of recovery performance (precision/recall/F1) and predictive error under different data regimes. We then analyze the real microRNA dataset, address preprocessing and class imbalance, and use stability selection (CPSS) to decide which effects are consistently selected. Finally, we compare several model variants using cross-validation metrics appropriate for classification (deviance, PR AUC, ROC AUC), interpret the practical meaning of the results, and summarize deliverables and recommendations for the company.

Finally, in Chapter 5, we revisit results and outline directions for future research.

2. Modeling with Interactions in a High-Dimensional Data Scenario

For the analysis in this study, we adopt the following notation: boldface letters will denote vectors (e.g. $\mathbf{X}_1$). Scalars (e.g. X_{ij}) and matrices (e.g. X) will not be bolded. We will use $\mathbf{X}_i$, $\mathbf{X}_j$ to denote the i -th row and column of matrix X , respectively. Sometimes, for clarity we will also use $\mathbf{X}_{i,*}$ and $\mathbf{X}_{*,j}$ for rows and columns. Each element will be clearly specified upon its first introduction to avoid any ambiguity.

2.1. High-Dimensional Data

We call a dataset high-dimensional when the number of explanatory variables is larger than the number of samples. Consequently, a very large number of explanatory variables, alone, is not enough to characterize the dataset as high-dimensional.

Here, we discuss the challenge arising due to high dimensionality. When the number of explanatory variables is higher than the number of observations (number of samples), assuming that they could potentially all be relevant, there is no way that we have enough information to draw an informed conclusion.

We will give an example that does not aim for strict mathematical accuracy but for effective intuition building. Imagine we are investigating a crime with 50 suspects, but only 10 witnesses. If all 50 suspects acted independently and at different locations, then the witnesses could not possibly have seen everything that happened. As a result, they would not be able to confirm or rule out every suspect's involvement. One possibility is that the witnesses happened to observe the key suspects, so their testimony is enough to explain the crime. However, we could still not be sure that some of the others were not involved as well. A more sensible approach would be to first narrow down the list of suspects as much as possible so that the available evidence can be used more effectively to identify the most likely explanation.

Mathematically, trying to explain as much as possible of the story with the witnesses you have is called a minimization problem, trying to rule out suspects (in this study) is Lasso regression, and we will explain both concepts until the end of this chapter.

2.2. Linear regression

Linear regression, a statistical model that dates back to 1805, is used to establish a relationship between a dependent variable $\mathbf{Y} = (Y_1, \dots, Y_n)^\top$ and one or more independent variables ($\mathbf{X}_j$). The independent variables can also be called covariates. From now on

we will use the term covariate to refer to each model variable. The general formula for linear regression with p covariates is:

$$\mathbf{Y} = \beta_0 + \beta_1 \mathbf{X}_1 + \cdots + \beta_p \mathbf{X}_p + \boldsymbol{\varepsilon}, \quad (2.1)$$

where $\boldsymbol{\varepsilon} = (\varepsilon_1, \dots, \varepsilon_n)^\top$ is the error term that occurs from the effect of unspecified covariates and/or a totally random element in the relationship specified [26]. We specify that $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The $\mathbf{X}_i$ covariates are typically treated as fixed effects, meaning $\mathbf{X}_i$ are not random variables.

Another way of representing the linear regression model is:

$$\mathbf{Y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad (2.2)$$

where $\boldsymbol{\beta} = (\beta_0, \dots, \beta_p)^\top$ is the vector of coefficients for the covariates and

$$X = \begin{pmatrix} 1 & X_{11} & \cdots & X_{1p} \\ 1 & X_{21} & \cdots & X_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \underbrace{X_{n1}}_{\mathbf{X}_1} & \cdots & \underbrace{X_{np}}_{\mathbf{X}_p} \end{pmatrix}$$

is the design matrix of p covariates and n samples¹.

We will list the required assumptions to make the above model a valid way to describe a given data structure. The assumptions are:

1. Independence of errors: The value of ε_i gives us 0 information about ε_{i+1} for any $i \in \{0, \dots, p\}$.
2. Linearity: There exists a linear relationship between the covariates ($\mathbf{X}_j$) and the dependent variable ($\mathbf{Y}$). In other words, a change in each of the covariates results in a proportional change to the dependent variable.
3. Homoskedasticity: The variance of the residuals should be constant across all levels of covariates $Var(\boldsymbol{\varepsilon}|X) = \sigma^2 \mathbf{1}^2$. For example, if the spread of actual incomes around the predicted income is roughly the same for people with 8 years of education as for people with 16 years, then you have homoskedasticity.
4. No Perfect Multicollinearity: There should not exist pairs of covariates that have a perfect linear relationship with each other.
5. Exogeneity: The covariates should not be correlated with the error term.

In this context, in the following sections, we explain how the model is built.

¹the first column of ones is the unit vector that is multiplied with β_0 , the intercept term.

²with $\mathbf{1}$ we denote the unit vector

2.3. Estimating coefficients on the linear regression model

The coefficients β_j in linear regression are estimated using the ordinary least squares (OLS) method. We will describe OLS with the help of the illustration in Figure 2.1. OLS adjusts the vector β (green part) to minimize the sum of the squared errors (red part) between the observed values (black dots) and the predicted values by the model (blue line). This can be represented mathematically as:

$$\hat{\beta}_{OLS} = \arg \min_{\beta \in \mathbb{R}^{p+1}} (\mathbf{Y} - X\beta)^\top (\mathbf{Y} - X\beta) \quad (2.3)$$

Once the coefficients β_i are estimated, the relationship between the covariates is modeled.

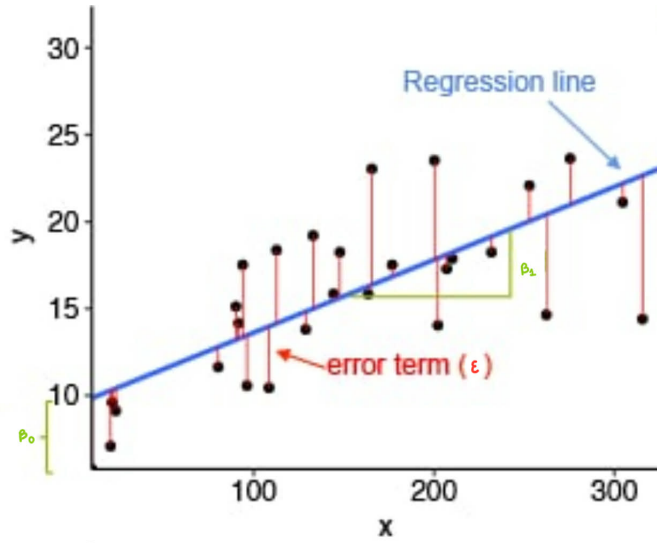


Figure 2.1.: Ordinary Least Squares (OLS) line with one covariate.

Although simple linear regression is a well-studied model, it may not be suitable for all research questions. For example, when formulating a question to explain an observed phenomenon, such as a biological process, we often formulate conditional questions, as discussed in the literature on interaction models [7]. For a conditional question like “How is covariate A affected, provided we have covariate B?”, a model that does not account for interactions between covariates may be inadequate.

2.4. Interaction Linear Regression

2.4.1. Introduction

A model including interactions has the general formula:

$$\mathbf{Y} = \beta_0 + \beta_1 \mathbf{X}_1 + \cdots + \beta_p \mathbf{X}_p + \beta_{11} \mathbf{X}_1^2 + \beta_{12} \mathbf{X}_1 \mathbf{X}_2 + \cdots + \beta_{pp} \mathbf{X}_p^2 + \epsilon \quad (2.4)$$

or, in a matrix form:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}\boldsymbol{\theta} + \boldsymbol{\varepsilon}. \quad (2.5)$$

In the formula above $\boldsymbol{\theta} = (\beta_{11}, \beta_{12}, \dots, \beta_{22}, \beta_{23}, \dots, \beta_{pp})^\top$ is the vector of coefficients, $\boldsymbol{\varepsilon}$ is the error term, $\varepsilon_i \sim \mathcal{N}(0, \sigma_i^2)$ and

$$\mathbf{Z} = \begin{pmatrix} X_{11}^2 & X_{11}X_{12} & \dots & X_{11}X_{1n} & X_{12}^2 & X_{12}X_{13} & \dots & X_{1p}^2 \\ X_{21}^2 & X_{21}X_{22} & \dots & X_{21}X_{2n} & X_{22}^2 & X_{22}X_{23} & \dots & X_{2p}^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \underbrace{X_{n1}^2}_{\mathbf{X}_1^2} & \underbrace{X_{n1}X_{n2}}_{\mathbf{X}_1\mathbf{X}_2} & \dots & X_{n1}X_{nn} & X_{n2}^2 & X_{n2}X_{n3} & \dots & X_{np}^2 \end{pmatrix}$$

the design matrix of interaction terms where $\mathbf{X}_i\mathbf{X}_j = \mathbf{X}_j\mathbf{X}_i$.

So, equivalently:

$$\mathbf{Y} = \beta_0 \mathbf{1} + \sum_{j=1}^p \beta_j \mathbf{X}_j + \sum_{j=1}^p \beta_{jj} (\mathbf{X}_j^2) + \sum_{1 \leq i < j \leq p} \beta_{ij} (\mathbf{X}_i\mathbf{X}_j) + \boldsymbol{\varepsilon}. \quad (2.6)$$

From now on, when we want to differentiate between original covariates and interaction covariates, we will refer to them as main effects and interaction effects, respectively. Also, we might use $\mathbf{X}_{aug}$ to refer to the matrix $\mathbf{X}$, augmented with the columns of design matrix $\mathbf{Z}$.

For this model, the required assumptions are:

1. Independence of errors: The value of ε_i gives us 0 information about ε_{i+1} for any $i \in \{0, \dots, p\}$.
2. Linearity: The relationship between $\mathbf{Y}$ and all covariates including interaction terms should be linear.
3. Homoskedasticity: The variance of the residuals should be constant across all levels of covariates $Var(\boldsymbol{\varepsilon}|\mathbf{X}, \mathbf{Z}) = \sigma^2 \mathbf{1}$, including interaction terms³. This requirement is important to avoid having the variance of one covariate dominating the overall variance of $\mathbf{Y}$. Imagine $\mathbf{X}_j, \mathbf{X}_{j'}$ are at their maximum values. The effect of interaction on $\mathbf{Y}$ would be much larger than when they're small. If the variance remains constant, it will be large and dominate the overall variance of $\mathbf{Y}$.
4. No Perfect Multicollinearity: Including interactions will increase structural multicollinearity, but we still want to forbid perfect multicollinearity.
5. Exogeneity: same as with the classic linear regression model.

Problems regarding assumptions 3 and 4 can be tackled by centering the data around 0. Optionally, we can scale their variance to 1 to reduce the extremeness of variances of interactions, but it is not needed. Although If we want to use the estimated β'_i s and make numerical predictions with them, we need to backtransform β'_i s (see appendix A for a detailed backtrasformation scheme).

This model allows us to examine not only a wider range of hypotheses but also more complex data structures, which is often the case with high-dimensional data (HDD).

³with $\mathbf{1}$ we denote the unit vector

2.4.2. Example

Next, we examine interaction linear regression models with the help of an example. Consider the model that describes how tasty the food is (described by $\mathbf{Y}$) with respect to the type of food ($\mathbf{X}_1$), the condiment ($\mathbf{X}_2$), and their interaction ($\mathbf{X}_1\mathbf{X}_2$):

$$\mathbf{Y} = 2\mathbf{X}_1 + 3\mathbf{X}_2 + 1.5\mathbf{X}_1\mathbf{X}_2 + \varepsilon$$

- $\beta_1 = 2$: For every one-unit increase in $\mathbf{X}_1$, $\mathbf{Y}$ increases by $2 + 1.5\mathbf{X}_2$.
- $\beta_2 = 3$: For every one-unit increase in $\mathbf{X}_2$, $\mathbf{Y}$ increases by $3 + 1.5\mathbf{X}_1$.
- $\beta_{12} = 1.5$: For every one-unit increase in $\mathbf{X}_2$, the effect of $\mathbf{X}_1$ on $\mathbf{Y}$ increases by 1.5, and vice versa.
- $\varepsilon_i \sim N(0, 1)$: The error term is normally distributed with a mean of 0 and a variance of 1.

It is intuitively clear that without choosing the type of food, we cannot select the condiment and expect a tasty outcome. However, we can also fit two models (with and without interactions) and compare their performance. Figure 2.2 shows us that when we fit a model with only main effects, and the linearity assumption is violated, interaction linear regression can be suitable to model the data provided that a relationship between the data exists. The reader can notice in Figure 2.2 that the color of the points (true $\mathbf{Y}$ value) better matches the color of bins (predicted value) in the model that includes interactions.

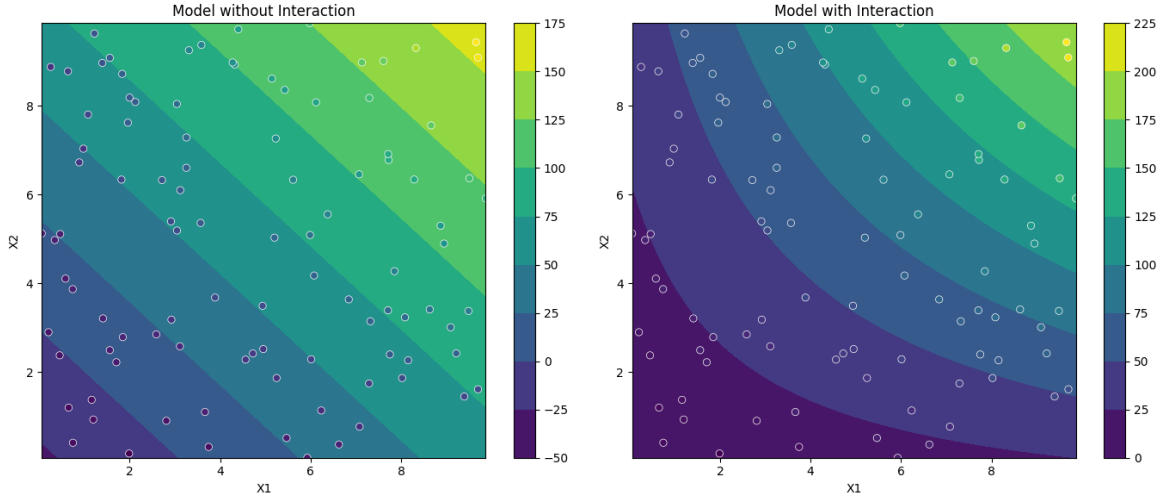


Figure 2.2.: Example of how data points are better assigned to their true labels with an interaction model rather than a linear one. The gradient levels represent taste ratings from 1 to 8. The actual data points are scattered over the contour plots in both subplots, with their true $\mathbf{Y}$ values indicated by color.

2.4.3. Hierarchy condition

When adding interaction terms, we often want to impose a hierarchy condition. The hierarchy condition has two levels; it can be either strong or weak. *Strong hierarchy is the rule that requires an interaction term to be included only if both of its covariates are already included as main effects.* On the other hand, a weak hierarchy requires at least one of its covariates to be included in the model. We want to impose it because it rarely makes sense not to do it [20] and because when including interactions, the number of covariates to consider is growing very fast (with p main effects, we would have $p + \frac{p(p-1)}{2}$ interactions). The hierarchy condition saves us computation time.

2.5. Minimization problem & Penalization method

2.5.1. Introduction

A minimization problem is the process of minimizing a function J , subject to some inequality and/or equality constraints. The process is denoted mathematically by $\min_{\beta \in \mathbb{R}^n} J(\beta)$, where J is called the cost function. In the context of regression, that process would minimize the error of the prediction of the chosen regression model. In other words, it applies to plain linear regression and linear regression with interactions.

A penalization method is the set of constraints we pose in a minimization problem. For this study, we analyze Lasso and variations of it.

2.5.2. Lasso

The Lasso constraint is:

$$\sum_{j=1}^n |\beta_j| \leq \tau, \quad (2.7)$$

where $\tau \geq 0$ is a threshold that limits the sum of the absolute values of the linear regression coefficients. The term on the left-hand side of the inequality represents the Lasso penalty, commonly referred to as the L1 penalty.

Example

Consider the true model:

$$Y = 1 + 7X_1 + 7.5X_2 + \varepsilon,$$

where $\varepsilon_i \sim N(0, 4)$. The OLS estimation gives a $\hat{\beta}_{OLS}$ closer to the true β . On the other hand, the Lasso estimation $\hat{\beta}_{Lasso}$ gives a smaller $\hat{\beta}_1$ and a $\hat{\beta}_2$ close to 0 due to the constraint, as we can see in Figure 2.3. Intuitively, we aim to obtain a predictive model that uses only the necessary information, which mathematically translates to using smaller β 's. This property is not needed for this simple example with two covariates, but in a high dimensional setting, it helps both the performance in unseen data and the time consumption for the model buildup. These arguments will be further supported in section 2.6 where we will describe how we get the Lasso estimation.

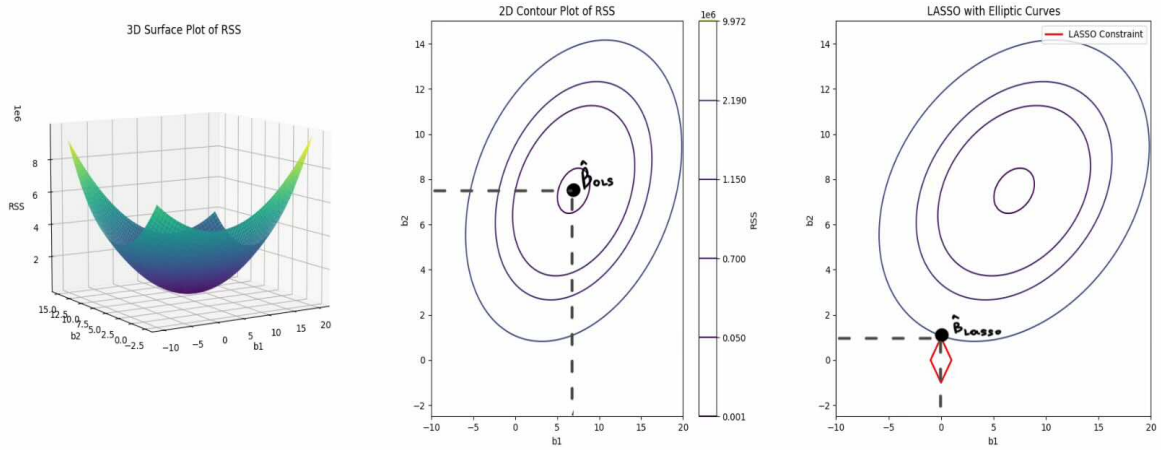


Figure 2.3.: Example of the effect of Lasso penalty/constraint on $\hat{\beta}$ when we have two covariates. The left plot depicts the convex RSS function. The middle plot depicts the contour curves of the RSS function projection on the β_1, β_2 plane and the OLS β estimator at the center. The right plot shows how β is shrunk towards lower values and β_1 became 0 to meet the diamond shape corresponding to the Lasso constraint (with $t=1$).

From now on, we will work with the penalized form of Lasso, which is equivalent to its constrained form. Concretely, instead of

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{N} \|\mathbf{Y} - X\beta\|_2^2 \quad \text{subject to 2.7}$$

we use the following representation to describe the linear regression error minimization problem with the Lasso penalty and the Lasso estimator, respectively as⁴:

$$\min_{\beta \in \mathbb{R}^p} \left\{ \frac{1}{N} \underbrace{\|\mathbf{Y} - X\beta\|_2^2}_{RSS} + \underbrace{\lambda \|\beta\|_1}_{\text{Lasso Penalty (L1)}} \right\}, \quad (2.8)$$

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^p} \left\{ \frac{1}{N} \|\mathbf{Y} - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}, \quad (2.9)$$

where RSS stands for Residual Sum of Squares and N is a constant helping with computations⁵. The above formula describes the goal of finding the best estimation of β for minimizing the errors between the fit and observations while penalizing β not to be very large with penalization degree λ . The penalization degree λ controls how much we suppress the coefficients.

The Lasso method is advantageous because it shrinks less important or highly collinear coefficients exactly to zero. This leads to a sparse model that retains only the most significant predictors, performing feature selection. Lasso is advantageous when we expect only a subset of the features to be truly relevant, as it simplifies the model and makes it

⁴We assume that both X and $\mathbf{Y}$ have been mean-centered, so the intercept is zero and need not be included in the penalized parameter vector.

⁵Specifically, $N = 2n$. dividing by 2 helps with computations when derivating RSS. Dividing by n helps us interpret RSS independently of the sample size.

more interpretable.

2.5.3. Group Lasso

Another relevant penalization method is group Lasso. As the name suggests, group Lasso distributes the regression coefficients in groups where

$$\boldsymbol{\beta} = (\beta_1, \dots, \beta_p) = (\beta^{(1)}, \dots, \beta^{(K)})$$

and K is the number of groups. This method will be useful later when extending to one of the methods we analyze. Mathematically it is represented as a minimization problem with the following expression:

$$\min_{\boldsymbol{\beta} \in \mathbb{R}^p} \left\{ \frac{1}{N} \|\mathbf{Y} - X\boldsymbol{\beta}\|_2^2 + \alpha\lambda \|\boldsymbol{\beta}\|_1 + (1 - \alpha)\lambda \sum_{j=1}^K \sqrt{\omega_j} \|\boldsymbol{\beta}^{(j)}\|_2, \right\} \quad (2.10)$$

where α is a chosen constant that determines how much we weigh the constraint on individual weights over the group constraint. All the initial coefficients are distributed in groups without any pair of groups containing the same coefficient. Finally, ω_j is the number of predictors/coefficients in group j .

The purpose of applying the extra penalty is to shrink to zero groups of coefficients in the estimated $\boldsymbol{\beta}$. In the trivial case, where every group contains only one variable, it is identical to Lasso. As is stated in [30] this method is particularly useful in biological studies as, in that case, it might be more meaningful to shrink all features involved in a specific biological pathway.

2.5.4. Purpose of penalization & conclusion

In this paragraph, we argue why we need penalization. Interaction models can capture more complex data structures [28]. But if we do not add the penalty term, we might create a model that overfits the sample data and has lower predictive ability in unseen (new) data. The danger of overfitting is greater when dealing with high-dimensional data because multicollinearity is present [16]. In these cases, adding penalization to the model coefficients becomes a necessary step. Notice that we do not include β_0 in the L1 penalty since the purpose of Lasso penalization is to avoid overfitting and perform feature selection. These are achieved through adjusting the coefficients of the covariates, and the intercept β_0 is not interfering.

Given the case-study involving microRNAs, which are often multicollinear because of their functional relationships and their co-expression patterns, a suitable penalization method must be carefully chosen. We take the previous sentence as a given and explain more about microRNAs in section 4.2, where we analyze the case-study dataset. The simplicity and effectiveness of Lasso make it a good choice for our purpose. Lasso not only treats multicollinearity by shrinking redundant microRNA coefficients to zero, but it also enhances model interpretability as it performs feature selection and keeps only significant predictors.

2.6. Lasso Regression Solution

2.6.1. Introduction

The Lasso minimization problem we want to solve is, as in equation 2.8:

$$\min_{\beta \in \mathbb{R}^p} \left\{ \frac{1}{N} \underbrace{\|\mathbf{Y} - X\beta\|_2^2}_{RSS} + \lambda \underbrace{\|\beta\|_1}_{R(\beta)} \right\},$$

where by $R(\beta)$ we denote the penalization part. Now, the corresponding cost function is:

$$J(\beta) = \frac{1}{N} \|\mathbf{Y} - X\beta\|_2^2 + \lambda \|\beta\|_1. \quad (2.11)$$

We can now consider the existence and uniqueness of the solution to the above minimization problem. Lasso regression solution exists even in the HDD case [30]. The Lasso cost function combines the RSS part with the $R(\beta)$ part. The RSS part is a strictly convex function when we have a high-dimensional dataset and convex when we have a low-dimensional dataset. Strictly convex intuitively translates to having the line segment between any two points of the graph above the graph. The $R(\beta)$ is convex but not strictly convex. Overall, we have a convex minimization problem. A convex but not strictly convex minimization problem can have multiple minima. The set of solutions for a convex minimization problem is convex (Theorem 9.4.1, [11]). This means that if $\beta_1(\lambda)$ and $\beta_2(\lambda)$ are solutions, $\kappa\beta_1(\lambda) + \omega\beta_2(\lambda)$ will also be a solution for any two constants κ, ω . So, the solution for beta is not unique, but $X\beta$ is [tibshirani2013Lasso]. This means we will not get a different prediction if we use another solution from the convex set of solutions.

To minimize J, we would want to take its gradient and set it to zero, but the R part is not differentiable everywhere. It is differentiable everywhere except at $\beta = \mathbf{0}$ and therefore, J is not differentiable locally at zero⁶. As a result, we have to look for alternatives. We will take cases for the design matrix X and solve the problem accordingly.

2.6.2. Orthonormal design matrix

An orthonormal design matrix is a design matrix that satisfies the property $X^\top X = I$, where

$$I = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

is a $p \times p$ dimensional matrix, and each column of X has norm 1. When this property is satisfied, the coefficients are decoupled and the solution for each β_j can be calculated independently (this is explained further in appendix A).

The solution for the orthonormal case is:

⁶With 0 in bold we denote the zero vector, $\mathbf{0} = (0, \dots, 0)$.

$$\beta_j = \text{sign}(u) \max(|u| - \lambda, 0) \quad (2.12)$$

where $u = \frac{\mathbf{X}^\top \mathbf{Y}}{n}$.

The function $\text{sign}(x)$ denotes the sign function:

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

where $x \in \mathbb{R}$.

The function $\max(a, b)$ is:

$$\max(a, b) = \begin{cases} a & \text{if } a > b \\ b & \text{otherwise} \end{cases}$$

where $a, b \in \mathbb{R}$.

The calculations that lead to the solution are displayed in appendix A.

KKT conditions for Lasso

Another way to approach the solution is through the Karush-Kuhn-Tucker (KKT) conditions. These conditions have to be satisfied for a solution in a minimization problem with a constraint to be optimum. We will provide the final form of the conditions when applied to Lasso minimization. For further reading on KKT conditions, see [13]. By the optimality conditions, as is intuitive, zero should be part of the set of solutions of the derivative of the minimization function. That translates into the following formula:

$$0 \in \frac{1}{n} \mathbf{X}_j^\top (\mathbf{Y} - \mathbf{X} \boldsymbol{\beta}) - \lambda \cdot \partial |\beta_j|, \quad (2.13)$$

where $\partial(\cdot)$ is called the "subgradient". We can think of it as the gradient counterpart when f is convex but not necessarily differentiable (a formal definition is here [6]). The derivative of the cost function is called the subdifferential because it involves a subgradient. The subgradient differs from its counterpart only where the function is non-differentiable. The subgradient at a non-differentiable point is the set of all slopes of lines that lie below the function and touch it at that point. Mathematically, we write:

$$\partial \beta_j = \begin{cases} 1 & , \beta_j > 0 \\ -1 & , \beta_j < 0 \\ [-1, 1] & , \beta_j = 0. \end{cases}$$

Using the equation for Lasso and solving for $\partial \beta_j$ we get the final form of KKT conditions for Lasso:

- For $\beta_j > 0$:

$$\frac{1}{n} \mathbf{X}_j^\top (\mathbf{Y} - \mathbf{X} \boldsymbol{\beta}) = \lambda \quad (2.14)$$

- For $\beta_j < 0$:

$$\frac{1}{n} \mathbf{X}_j^\top (\mathbf{Y} - \mathbf{X} \boldsymbol{\beta}) = -\lambda \quad (2.15)$$

- For $\beta_j = 0$:

$$\left| \frac{1}{n} \mathbf{X}_j^\top (\mathbf{Y} - X\boldsymbol{\beta}) \right| \leq \lambda \quad (2.16)$$

In the orthonormal design case, where $X^\top X = I$, these conditions simplify because $\mathbf{X}_j^\top X\boldsymbol{\beta} = \beta_j$, and we can write:

$$\frac{1}{n} \mathbf{X}_j^\top \mathbf{Y} = \beta_j + \lambda \cdot \text{sign}(\beta_j).$$

When the KKT conditions are checked for the coefficient of each new variable added to the model, we can assess its contribution towards explaining the unexplained variance of $\mathbf{Y}$. Specifically, we use KKT to guarantee that the amount of variance that the new variable explains is not less than the regularization parameter λ . Otherwise, we remove the variable from the model by shrinking its coefficient to 0. The function of performing this check and, whenever needed shrinkage, we call it soft-thresholding.

Soft-thresholding function (SF)

Definition 2.1 (Soft Thresholding Function). The soft thresholding function is defined for a scalar $x \in \mathbb{R}$ and a threshold parameter $\lambda \geq 0$ as [10]:

$$S(x, \lambda) = \begin{cases} x - \lambda & \text{if } x > \lambda \\ 0 & \text{if } |x| \leq \lambda \\ x + \lambda & \text{if } x < -\lambda \end{cases}$$

Alternatively, the soft thresholding function can be expressed as:

$$S(x, \lambda) = \text{sign}(x) \cdot \max(|x| - \lambda, 0)$$

Therefore the Lasso solution for β_j 2.12 can also be expressed as:

$$S\left(\frac{\mathbf{X}_j^\top \mathbf{Y}}{n}, \lambda\right)$$

by the definition of the soft thresholding function $S(x, \lambda)$.

Now, we can review helpful information about interpreting the soft thresholding function. The SF function can be understood as a shrinkage operator applied to the first argument, using the second argument as the shrinkage parameter. The scaling of the first argument is crucial for the operation to be performed consistently among all coefficients. If we divide $\mathbf{X}_j^\top \mathbf{Y}$ by $\|\mathbf{X}_j\|_2^2$, we ensure that all features are treated evenly regardless of their magnitude. $\|\mathbf{X}_j\|_2^2$ is approximately equal to n , provided that all features are standardized (meaning $\forall j \in \{1, \dots, n\} : E(\mathbf{X}_j) = 0, \text{Var}(\mathbf{X}_j) = 1$). That is true since,

$$\|\mathbf{X}_j\|_2^2 = \sum_{i=1}^n X_{ij}^2 \approx n \cdot \text{Var}(\mathbf{X}_j) = n.$$

The larger the sample size n the better the approximation.

A practical read on what the SF function is doing on the coefficients can be found in Figure 3.1. What can be observed in Figure 3.1 is that the coefficients that were originally below the shrinkage parameter, $\lambda = 0.5$, were shrunk to 0. The rest were re-weighted accordingly (e.g., assuming the vector of coefficients before thresholding is $(-1, 0.3, 2.2)$, it will become $(-0.5, 0, 1.7)$).

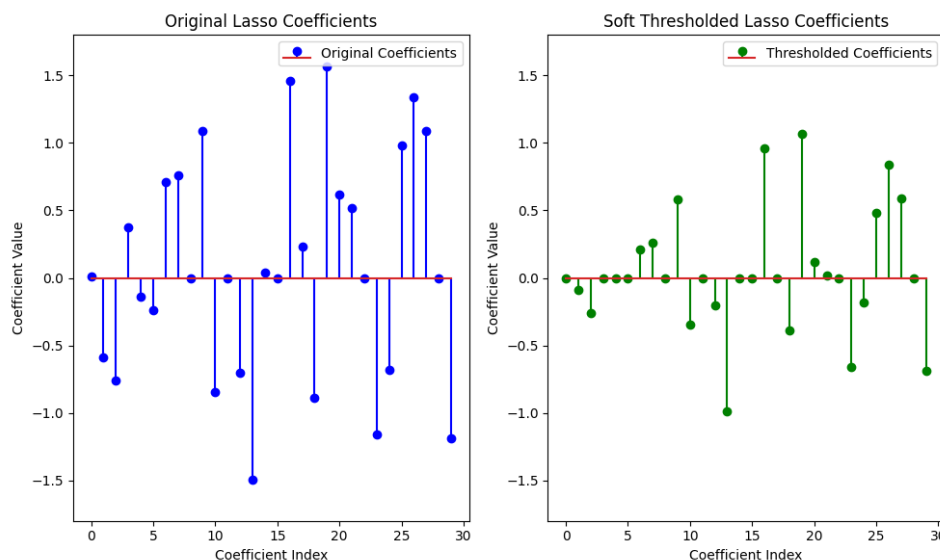


Figure 2.4.: Soft thresholded coefficients with shrinkage step $\lambda = 0.5$. Notice that a general pattern in the coefficients is maintained and some of them are shrunk to 0.

2.6.3. General design matrix

When columns of X are correlated, there are dependencies between columns, and unfortunately, we cannot give a definite estimate for each β_j , but there are good approximation methods. Some of the most commonly used methods involve:

1. Coordinate descent: updates for one coordinate at a time while holding other coordinates fixed.
2. Least Angle Regression (LARS): Iteratively adds predictors to the model based on their correlation with the residual.
3. Gradient-based methods: Updates all coordinates simultaneously and always involves the gradient in the update rule.

Coordinate Descent

For the Lasso solution, coordinate descent is the method that is used in the R package *glmnet* [32]. By updating one coefficient at a time and considering all others fixed, we compromise the amount of information we consider in each step. This approach simplifies the problem into a single-variable optimization, which has a closed-form solution, and the end product β succeeds in minimizing the overall error.

To update one coordinate at a time, even though the effect of covariates on $\mathbf{Y}$ is interconnected with the effects of other covariates, we shift our focus. Instead of evaluating how much of $\mathbf{Y}$ is explained by $\mathbf{X}_j$, we assess how much of the residual from a model excluding covariate j is explained when $\mathbf{X}_j$ is added. Mathematically, the mentioned residual is represented as

$$\mathbf{r}_{(-j)} = \mathbf{Y} - \sum_{\substack{k=1 \\ k \neq j}}^p \mathbf{X}_k \beta_k. \quad (2.17)$$

Or in matrix form $\mathbf{r}_{(-j)} = \mathbf{Y} - \mathbf{X}\boldsymbol{\beta} + \mathbf{X}_j\beta_j = \mathbf{Y} - \mathbf{X}_{-j}\boldsymbol{\beta}_{-j}$ and it is called the partial residual of j . The exact algorithm is described on the algorithmic table 1.

Algorithm 1 Coordinate Descent for Lasso Regression

- 1: **Initialize:** Set coefficients $\beta_j = 0$ for all $j = 1, 2, \dots, p$
- 2: Set convergence tolerance ε
- 3: Standardize the data $\mathbf{B}_j = \frac{\mathbf{X}_j - \mu_j}{\sigma_j}$ for all $j = 1, 2, \dots, p$
- 4: **Repeat until convergence:**
- 5: **for** each $j = 1, 2, \dots, p$ **do**
- 6: Compute partial residual $\mathbf{r}_{(-j)} = \mathbf{Y} - \mathbf{B}_{-j}\boldsymbol{\beta}_{-j}$
- 7: Update β_j using ST:

$$\beta_j \leftarrow S\left(\frac{\mathbf{B}_j^\top \mathbf{r}_{(-j)}}{n}, \lambda\right)$$

8: **end for**

9: $|\beta_j^{\text{new}} - \beta_j^{\text{old}}| < \varepsilon$ for all j

10: **Return** the final coefficients $\boldsymbol{\beta}$

Example & Useful terminology

Think of minimizing:

$$\|\mathbf{Y}_{\text{centered}} - \mathbf{X}_1 b_1 - \mathbf{X}_2 b_2\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1,$$

where $\mathbf{Y}_{\text{centered}}$ is $\mathbf{Y}$ minus the mean of $\mathbf{Y}$. This is a way to remove the intercept from our model. For different λ 's we would get different optimal solutions. Think of the level curves of the RSS graph as ellipses shrinking inwards from infinity (see on Figure 2.5 the gradient of red curves). The optimal solution for the current λ is found on the first point, where one of these ellipses barely touches the constraint region, L1, of the current λ .

We begin by addressing how to choose the grid of λ values. We want to select the minimum λ that is big enough to shrink all coefficients to zero, and then gradually decrease λ to allow more coefficients into the model. According to the soft-thresholding solution for the Lasso in the general design matrix case, a coefficient β_j is zero if

$$\left| \frac{\mathbf{B}_j^\top \mathbf{r}_{(-j)}}{n} \right| \leq \lambda.$$

To ensure this holds for all j , we define

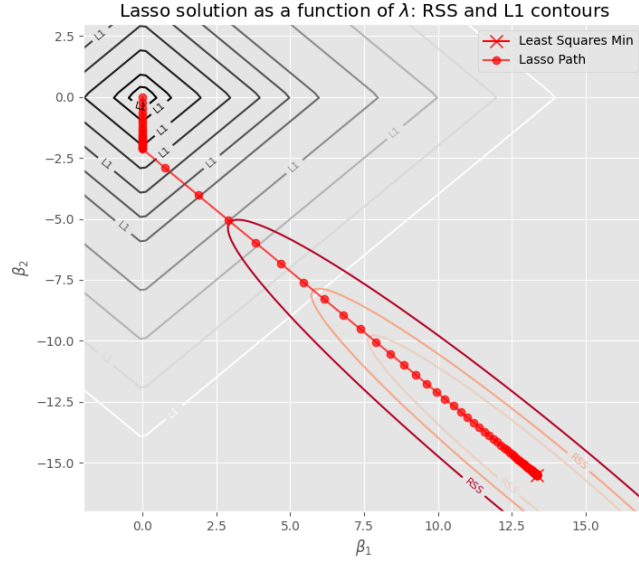


Figure 2.5.: Lasso solution (red dot) for different lambdas. As lambda changes, the size of the diamond (constraint) also changes. The solution always lies within or on the constraint. For $\lambda = 0$ we get the RSS solution (blue dot) and as $\lambda \rightarrow \infty, \beta \rightarrow 0$.

$$\lambda_{\max} = \max_j \left| \frac{\mathbf{B}_j^\top \mathbf{r}(-j)}{n} \right|. \quad (2.18)$$

Next, we set

$$\lambda_{\min} = \varepsilon \lambda_{\max}, \quad (2.19)$$

where ε is a small positive constant. The density of λ values in the grid can be adjusted, and using more λ values may help obtain an optimum solution with lower prediction errors. Our choice for ε and the number of values on the grid are 0.001 and 100, respectively. This choice is a typical choice for ε and the number of values on the grid[12]. The best λ is finally chosen via cross-validation. For a detailed explanation of cross-validation, see [25].

We now analyze Figure 2.5. In Figure 2.5, each red dot corresponds to an optimal solution for a particular value of λ . The collection of all optimal solutions forms the *solution path*, which is defined as

$$P(\boldsymbol{\beta}) = \{\boldsymbol{\beta}(\lambda_{\max}), \dots, \boldsymbol{\beta}(\lambda_{\min})\}. \quad (2.20)$$

Each red dot represents the first point at which one of the RSS level curves touches the diamond-shaped constraint corresponding to the current λ . Not all constraints for all λ 's and all level curves are depicted to simplify the graph. Notice that as $\lambda \rightarrow 0$, the Lasso solution converges to the RSS solution, which is equivalent to the linear regression solution without regularization. Conversely, as $\lambda \rightarrow \lambda_{\max}$, the coefficients tend to zero. In particular, for larger values of λ , the solution path becomes vertical with respect to the x -axis, indicating that for a certain λ and any larger value, the coefficient β_1 is zero.

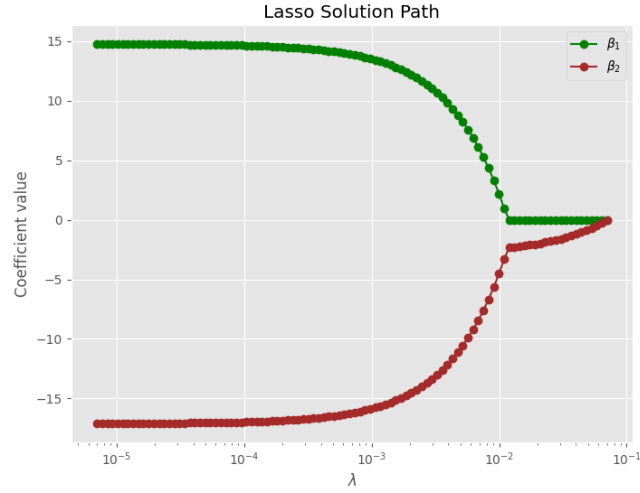


Figure 2.6.: Lasso solution path showing how each coefficient changes as a function of the regularization parameter λ .

What we just observed is the *sparsity* induced by the Lasso constraint.

Figure 2.6 shows the same solution path as in Figure 2.5 but with different axes. Each green bullet and its vertical brown bullet corresponds to an optimal solution for β . In this plot, we can connect the choice of $\lambda_{\max}$ with its reasoning, as indeed, both coefficient values are 0 in the beginning.

Later, we will have many more coefficients when working with real datasets. Therefore, we will use from now on, a simplified version of the solution path plot that no longer displays a bullet for each tested λ . This is the plot shown in Figure 2.7. In this plot, each vertical line represents a different final model, and we often use a vertical line to highlight the chosen model that returns optimal predictions.

We will also introduce additional terminology in parallel with the example. This terminology will be useful for keeping track of the large number of coefficients and covariates that exist in real datasets. We refer to the covariates included in the design matrix as *Candidate covariates* because they are candidates for entering the model the moment their coefficient is no longer 0. The set including the candidate covariates is called *Candidate set*, and we denote it by C . Here, $C = \{\mathbf{X}_1, \mathbf{X}_2\}$. It is worth noting that the solution path changes if we modify the candidate set/design matrix. To signify that dependency, when C varies, we will write:

$$P(\beta, C) = \{\beta(\lambda_{\max}, C), \dots, \beta(\lambda_{\min}, C)\}. \quad (2.21)$$

In this example, C does not vary. As the solution path is built while we lower λ , more coefficients become non-zero. When this happens, these coefficients enter the *Active set*, and we denote this set with the letter A . Here, the dashed line in Figure 2.8 gives a model with $A = \{X_2\}$.

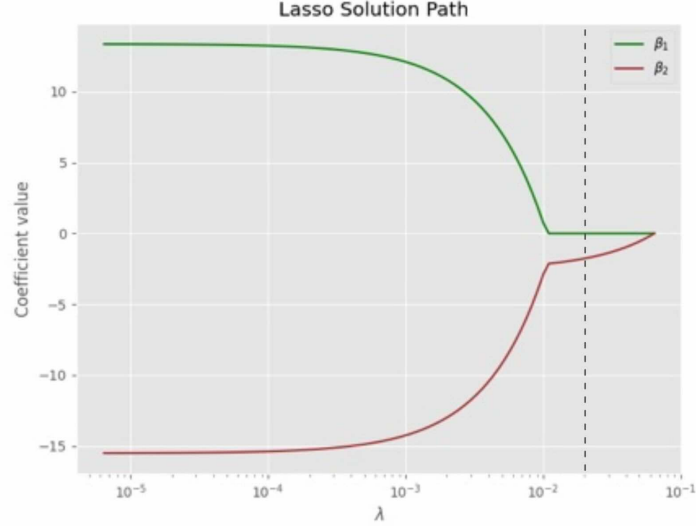


Figure 2.7.: Simplified representation of the Lasso solution path shown in Figure 2.6. The dashed line indicates the selected model.

2.7. Extension to logistic regression

We will now introduce the logistic regression model. It is a model used to explain a binary outcome variable. We denote the outcome for the i -th observation as Y_i , where Y_i can take on values of either 0 or 1 and $\mathbf{X}_{i,*}$ the row vector that we store the i -th sample for $i = 1, \dots, n$. So the domain of the response variable Y_i is $\{0, 1\}$. Since we are dealing with a binary response, we cannot directly model Y_i with a linear form like $Y_i = \mathbf{X}_{i,*}\boldsymbol{\beta} + \varepsilon_i$, as in linear regression. Our data, though, are still stored in a design matrix $\mathbf{X}$ where $\mathbf{X}_{i,*}$ and $\boldsymbol{\beta}$ are vectors in $\mathbb{R}^p$. Since the right-hand side of the previous equation is in $\mathbb{R}^p$, we need to use a transformation that restricts these vectors to the domain of response. We can accomplish that by the following transformation that maps measurements to 0 or 1 and explains the probability of success $p_i = P(Y_i = 1)$:

$$p_i = \frac{e^{\mathbf{X}_{i,*}\boldsymbol{\beta}}}{1 + e^{\mathbf{X}_{i,*}\boldsymbol{\beta}}} \quad (2.22)$$

The above function will eventually be used to map the linear combination of $\mathbf{X}_{i,*}$ and the estimated coefficients to the probability of success (see left plot of figure 2.8).

The general form of equation 2.22 is:

$$p_i = g^{-1}(\mathbf{X}_{i,*}, \boldsymbol{\beta}) = \frac{e^{t_i}}{1 + e^{t_i}}, \quad (2.23)$$

where t_i is the score calculated from the linear model that has to be binarized and $g(\mathbf{X}_{i,*}, \boldsymbol{\beta})$ is the *link function* that links the score t_i with the probability p_i . Other link functions could be used, but all of them have to be invertible. Regardless of the choice of the link function, the binary single outcome is modeled by the binomial distribution [30]:

$$Y_i \sim \mathcal{B}(g^{-1}(\mathbf{X}_{i,*}, \boldsymbol{\beta}), 1) \text{ or } \mathcal{B}(p_i, 1),$$

where the first argument is the probability of success and the second argument is the number of trials.

Equivalently to equation 2.22 we have:

$$\log\left(\frac{p_i}{1-p_i}\right) = \mathbf{X}_{i,*}\boldsymbol{\beta} \quad (2.24)$$

We call the right-hand side the "linear predictor", and we denote:

$$t_i = \mathbf{X}_{i,*}\boldsymbol{\beta} \quad (2.25)$$

Equations 2.24 and 2.25 show that the log-odds of success are modeled as a linear function of the explanatory variables.

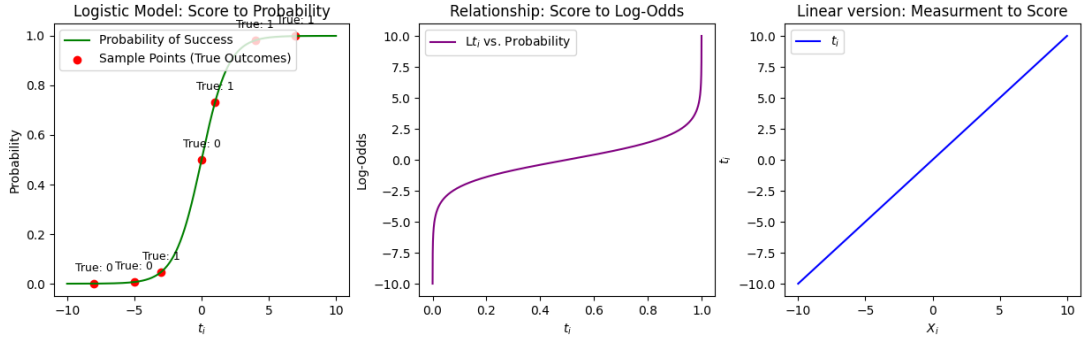


Figure 2.8.: Left plot: Fitted line trained on real data (red dots) to map samples to a probability of success (as described by equation 2.22. Annotations indicate the true outcomes with 1 to denote success and 0 to denote failure, allowing us to see how well the model aligns with the observed data. Middle plot: Describes the relationship between the probability of success and log odds ($t_i = \log\left(\frac{p_i}{1-p_i}\right)$), which shows the large variations in log-odds as probabilities approach 0 or 1. Right plot: Describes the linear form of logistic regression where Y describes the log-odds of the probability of success (corresponds to equation 2.25).

The logistic model's coefficients represent the log odds of the outcome and are estimated by the Maximum Likelihood Estimator.

- If $\beta_i = 0.5$: A one-unit increase in X_i increases the log-odds of the outcome by 0.5 .
- If $\beta_i = -0.5$: A one-unit increase in X_i decreases the log-odds of the outcome by 0.5 .

Another thing to consider is the separability of the labels (success/failure). As shown in the first plot of Figure 2.8, some samples, despite being used in training, are not clearly assigned to the correct label and have a probability of success close to 0.5. In such cases, we would expect even more misclassifications when new data is introduced. In high-dimensional data (HDD) scenarios, it is likely that the model will attempt to incorporate

additional covariates to make it easier to separate the labels. This makes it even more critical to use a regularization method like Lasso, which performs feature selection and helps prevent overfitting by selecting only the most relevant predictors.

Coefficient estimation

The solution will combine the method of solving the unregularized part with the method for solving the regularized part. As mentioned, the coefficients of the logistic regression model (without regularization) are estimated by the Maximum Likelihood estimator, found by maximizing the logarithm of the likelihood function $\mathcal{L}(\boldsymbol{\beta}) = \log(\ell(\boldsymbol{\beta}))$. Additionally $Y_i \sim \mathcal{B}(p_i, 1)$. Combining the above, we want to maximize:

$$\mathcal{L}(\boldsymbol{\beta}) = \log\left(\prod_{i=1}^n f(\mathbf{X}_{i,*}, \boldsymbol{\beta})\right), \quad (2.26)$$

where $f(\cdot, \cdot)$ is the probability density function (pdf) of the binomial distribution. After calculations (that are displayed in A) we find:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^n [Y_i \mathbf{X}_{i,*} \boldsymbol{\beta} - \log(1 + e^{\mathbf{X}_{i,*} \boldsymbol{\beta}})]. \quad (2.27)$$

Maximizing $\mathcal{L}(\boldsymbol{\beta})$ is equivalent to minimizing $-\mathcal{L}(\boldsymbol{\beta})$.

To succeed in the aforementioned, a generally used method is the *Newton-Raphson's* (NR) algorithmic method. The NR method finds the zeros of $\mathcal{L}(\cdot)$, provided that $\mathcal{L}(\cdot)$ is smooth enough [30]. The update for the NR method generally is:

$$\mathbf{w}^{new} = \mathbf{w}^{old} - \frac{\mathbf{d}}{H}, \quad (2.28)$$

where the gradient vector is

$$\mathbf{d} = \nabla f(\mathbf{w}) = \begin{pmatrix} \frac{\partial f(\mathbf{w})}{\partial w_1} \\ \frac{\partial f(\mathbf{w})}{\partial w_2} \\ \vdots \\ \frac{\partial f(\mathbf{w})}{\partial w_p} \end{pmatrix}$$

and the Hessian matrix is

$$H = \nabla^2 f(\mathbf{w}) = \begin{pmatrix} \frac{\partial^2 f(\mathbf{w})}{\partial w_1^2} & \frac{\partial^2 f(\mathbf{w})}{\partial w_1 \partial w_2} & \dots & \frac{\partial^2 f(\mathbf{w})}{\partial w_1 \partial w_p} \\ \frac{\partial^2 f(\mathbf{w})}{\partial w_2 \partial w_1} & \frac{\partial^2 f(\mathbf{w})}{\partial w_2^2} & \dots & \frac{\partial^2 f(\mathbf{w})}{\partial w_2 \partial w_p} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{w})}{\partial w_p \partial w_1} & \frac{\partial^2 f(\mathbf{w})}{\partial w_p \partial w_2} & \dots & \frac{\partial^2 f(\mathbf{w})}{\partial w_p^2} \end{pmatrix}_{p \times p}.$$

We take the quadratic approximation of $\mathcal{L}(\cdot)$ via the Taylor approximation around a point $\mathbf{w}_0 \in \mathbb{R}^p$. The approximation ensures smoothness and, therefore, convergence. The

choice of a quadratic approximation (instead of a different order) ensures computational efficiency. We now want to minimize:

$$\mathcal{L}(\mathbf{w}) \approx \mathcal{L}(\mathbf{w}_0) + \nabla \mathcal{L}(\mathbf{w}_0)^\top (\mathbf{w} - \mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top \nabla^2 \mathcal{L}(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0). \quad (2.29)$$

We choose to update one coordinate at a time since calculating and storing the whole Hessian matrix would not be efficient, especially in the HDD case. As a result, the previous equation is more useful to us in the following form:

$$\mathcal{L}(w_j) \approx \mathcal{L}(w_j^{old}) + \nabla \mathcal{L}(w_j^{old})^\top (w_j - w_j^{old}) + \frac{1}{2}(w_j^{old})^\top (w_j - w_j^{old})^\top \nabla^2 \mathcal{L}(w_j^{old})(w_j - w_j^{old}), \quad (2.30)$$

or

$$\mathcal{L}(w_j) \approx \mathcal{L}(w_j^{old}) + d_j(w_j - w_j^{old}) + \frac{1}{2}H_{jj}(w_j - w_j^{old})^2. \quad (2.31)$$

Here, $d_j = [\nabla \mathcal{L}(w_j^{old})]_j$ is the j -th component of the gradient, and $H_{jj} = [\nabla^2 \mathcal{L}(w_j^{old})]_{jj}$ is the j th diagonal element of the Hessian matrix. If we minimize $\mathcal{L}(\cdot)$, we will get a solution to the unregularised part of the problem, namely $\mathbf{w}^{\text{unreg}}$. To find the general solution, we want to incorporate the solution for L1 regularization. As we have shown, that comes from the soft-thresholding function. We have shown how to apply the SF function when the format of the minimization problem is "square form" plus "L1 regularization". Gladly, after calculations and a completion of squares (that are displayed in A), we have:

$$\mathcal{L}(w_j) \approx \frac{1}{2}H_{jj}(w_j - w_j^{\text{unreg}})^2. \quad (2.32)$$

To conclude, for the overall minimization problem

$$\min_{w_j} \left(\frac{1}{2}H_{jj}(w_j - w_j^{\text{unreg}})^2 + \lambda |w_j| \right), \quad (2.33)$$

the update would be:

$$w_j^{\text{new}} = S \left(w_j^{\text{unreg}}, \frac{\lambda}{H_{jj}} \right). \quad (2.34)$$

The whole procedure described above results in a sequence of nested loops for the implementation of coefficient estimation:

- OUTER LOOP: Decrement λ .
- INNER LOOP: Update the quadratic approximation 2.32 with NR method.
- OUTER LOOP: Run the coordinate descent algorithm on the updated minimization problem with update w_j^{new} as specified in 2.33.

From now on, we will discuss methods to help us build a model. The model we will build will help us to predict the log odds of the probability of success. We can use equations (2.2) and (2.1) to determine the probability of success. Finally, we can set a threshold probability to determine from which point onward it is considered safe to predict success.

3. Methods

In this chapter, we introduce the three methods under study, detailing their mechanisms to support the subsequent analysis and interpretation of the simulation outcomes and the results obtained on the TGS dataset. For the following sections, when we refer to the error term ε , we assume that $\varepsilon_i \sim N(0, \sigma^2)$ unless specified otherwise. Also, sometimes we will interchange the terms covariate and variable to refer to a column from a design matrix $\mathbf{X}_i$. Finally, when we write "hierarchy condition", we refer to the strong hierarchy condition that is imposed on the first two methods.

3.1. Forward Selection Method (FS)

We will go through the skeleton of a generalized forward selection method. Forward Selection is a simple yet effective method to pick which variables will be included in the linear regression model. In the beginning, all models with a single variable are fitted. A "selection criterion" has been predefined, and based on that, we can choose which variable will be the first to be added to the model. The selection criterion has to be a measure of the goodness of fit of the model. Keeping the first chosen variable, we can proceed to do exactly the same for adding a second variable. Iteratively, this continues until the "stopping criterion" directs the algorithm to finish. The stopping criterion generally is a mark that the model performance does not improve by adding any new variable. In a high-dimensional data case, multiple stopping criteria can be combined to ensure we avoid over-fitting the model.

Below, we will list popular selection and stopping criteria, while the specific choice for our implementation will be discussed later.

1. Add the predictor with "lowest p-value" and lower than a cutoff α (usually lower than 0.05).
2. Choose the predictor that results in a model with "minimal AIC" (Akaike Information Criterion) or "minimal BIC" (Bayesian Information Criterion). Both AIC and BIC balance goodness of fit while penalizing models with more predictors, but BIC has a stronger penalty for additional predictors.
3. Choose the predictor that results in a model with "Maximal adjusted R^2 " (maximizing the proportion of variance explained by the model).
4. Choose the predictor that results in a model with "minimal RSS". RSS stands for Residual Sum of Squares and, in contrast with adjusted R^2 , measures the unexplained variance.

Based on the provided dataset, one could pick the most suitable selection criterion. Popular stopping criteria involve:

FS example with 5 variables:

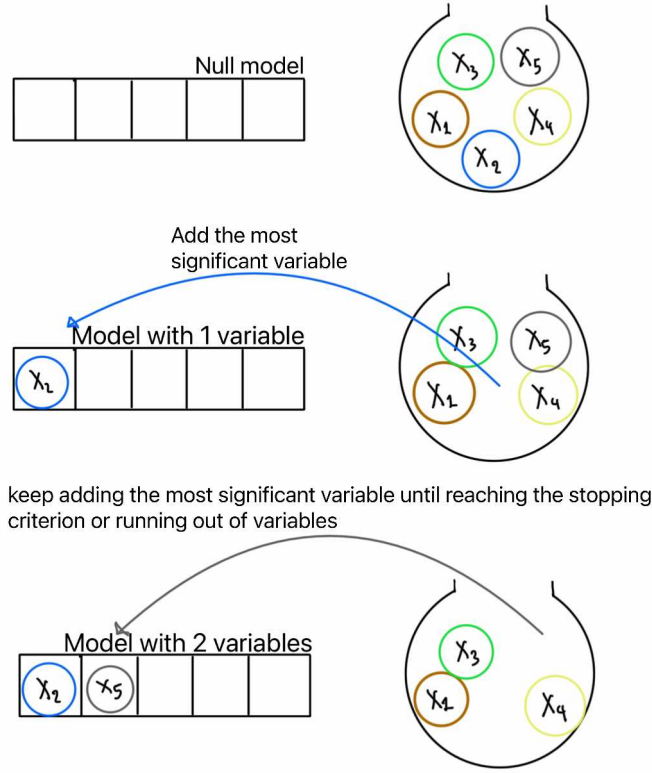


Figure 3.1.: Forward Selection example with 5 variables. The blue arrow indicates the selection process that is performed using the RSS criterion. After adding the first variable, we continue until the number of chosen variables is at most as big as the number of samples ($k \leq n$).

1. Stop when a "predetermined number of predictors" has entered the model.
2. Stop when "no significant p-value" occurs in any of the newly proposed predictors.
3. Stop when the improvement of AIC/BIC/adjusted R^2 /RSS is minimal based on a certain threshold.
4. Stop when all predictors are evaluated.

As with the selection criteria, we can decide based on the dataset properties and any prior knowledge related.

Overall, the pros and cons of this algorithm make it a default baseline algorithm. On the one hand, it is straightforward to implement, understand, and interpret. Its complexity is $\mathcal{O}(np)$, which makes it feasible for HDD application (for the related calculations see A and for a definition of $\mathcal{O}(\cdot)$ see [9]). On the other hand, it is picking local optimal solutions at each iteration and is biased towards early added covariates. In other words, covariates selected in the initial steps can influence the selection of subsequent covariates. Therefore, we would expect it to identify strong connections, but there are more sophisticated algorithms for maximizing accuracy.

3.1.1. Forward Selection (without interaction terms)

Having set the ground for how FS works, we can now go through an example with more mathematical details. Assume the model :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon.$$

Additionally, assume the helping vector:

$$\vec{v} = (v_1, v_2, \dots, v_n) \quad , \quad v_j = \begin{cases} 1 & \text{if } X_j \text{ is included in the model} \\ 0 & \text{if } X_j \text{ is not included in the model} \end{cases} \quad \text{for } j = 1, 2, \dots, n.$$

where $\vec{v}$ describes which variables are included in the model. Another way to refer to those variables is the *candidate model*, denoted by m . With the candidate model, we address any model examined by FS that potentially gives the best selection criteria. If, for example, we examine the model $Y = \beta_1 X_1 + \beta_2 X_2$, then $m = \{1, 2\}$. When we refer to the candidate set, we address the covariates that are included in the corresponding candidate model, and the number of covariates in the candidate set is the $\sum_{i=1}^n v_i$. Therefore the vector $(0, 0, \dots, 0)$ represents the null model.

First, let us introduce a few useful symbols:

a_k : The index of the variable added at the k-th step.

P : The set of indices of all predictors

s_k : The set that includes the indices of the variables in the solution of the k-th iteration.

c_k : "*indices of candidate set*" The set that includes the indices of the candidate variables.

$m_{i,j}$: "*indices of candidate model*" The set that includes the indices of the candidate model at step j plus index i c_k .

We will follow a simple example with $n = 3$. Consider the selection criterion to be how low $RSS = \sum_{i=1}^2 (y - \hat{y})^2$ is. Suppose a candidate model involving only X_1 and not X_2, X_3 gives the lowest RSS among all models with a single term. In the next step of the solution path, we would pick $(1, 0, 0)$. Then, an illustration of how the model would learn with FS can be found in Figure 3.2. Assuming all variables added value to the model and no additional stopping criteria were forced (other than stopping when all predictors are evaluated), following the FS algorithm (see algorithm 2), we end up with the full model $(1, 1, 1)$. In a non-simple example, the only difference would be having more variables and imposing one or more stopping criteria.

3.1.2. interaction Forward Selection (iFOR)

Now we are ready to dive into the concept of FS with interactions that adhere to the hierarchy condition. When we include interactions, especially in the HDD case, the number of variables to consider is rising quickly, and the need for computational efficiency becomes more important in order for the algorithm to converge. Below, we will analyze the adjustments suggested by the authors of "Interaction screening for ultra-high dimensional datasets" on the FS algorithm to tackle the aforementioned problem. These adjustments

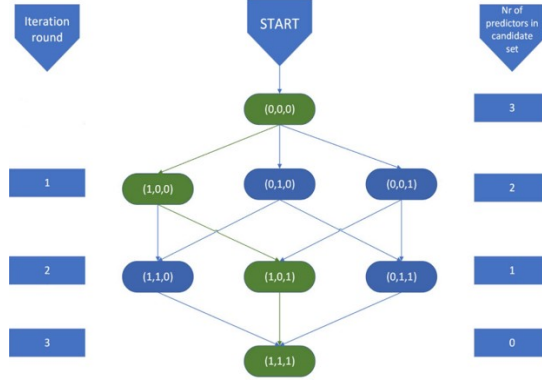


Figure 3.2.: Forward Selection example solution in green.

Algorithm 2 Forward Selection (FS)

```

1: Initialize:  $k = 1$ ,  $s_0 = \emptyset$  ▷ No variables are selected yet
2: while  $k \leq n$  do
3:   for each  $i \in c_k$  do ▷ For every  $i$  in the set of candidate indices
the ones we have already chosen
4:     Form candidate model  $m_{i,k-1} = s_{k-1} \cup \{i\}$  ▷ Make a set  $m$  that
includes current set plus new  $i$ 
5:     Compute  $RSS_{m_{i,k-1}}$  ▷ Compute the RSS for this model
6:   end for
7:   Find  $a_k = \arg \min_{i \in c_k} RSS_{m_{i,k-1}}$  ▷ Find  $i$  that gives minimum RSS
8:   Update  $s_k = s_{k-1} \cup \{a_k\}$ 
9:    $k \leftarrow k + 1$ 
10: end while

```

result in two algorithms. A simpler version called iFORT and the final version called iFORM.

Specifically, in this section, we will briefly mention the adjustments by listing them. We will further analyze their value in the dedicated section of the algorithms, iFORT and iFORM, that will follow. The adjustments involve:

1. Stopping criteria used: We predetermine the maximum number of iterations to be bounded by $k < n$. Note that in this algorithm, the number of iterations k is equivalent to the number of covariates added to the model. That criterion is necessary to avoid collinearity. The second criterion is $k < \frac{n}{\log(n)}$ that results in a model with fewer covariates, which is desirable in an HDD case.
2. Selection criteria used: The purpose of this change is to employ a criterion that allows us to penalize larger models, more effectively than RSS , now that the number of all possible candidate predictors is $p + \frac{p(p-1)}{2}$ together with a large number of samples.

BIC_1 : A criterion advantageous over RSS in avoiding overfitting.

$$BIC_1 = \ln(\hat{\sigma}^2) + \frac{k}{n} \ln(n), \quad (3.1)$$

where k is the number of selected variables in the model and n is the number of samples. It involves RSS in the formula

$$\hat{\sigma}^2 = \frac{RSS}{n - k - 1}$$

that gives us the goodness of fit measure plus the overfitting penalization term $\frac{k}{n} \ln(n)$.

BIC_2 : This is the enhanced BIC. That criterion is stronger in helping us avoid overfitting, which can be caused by a large number of predictors.

$$BIC_2 = \ln(\hat{\sigma}^2) + \frac{k}{n}(\ln(n) + 2\log(d)) \quad (3.2)$$

Where, d denotes the total number of covariates at the start of FS.

3. How interactions enter the model:

iFORT: In a separate second stage of the algorithm

iFORM: Together with the main effects

iFORT

Below, we will describe iFORT and give a simple example. iFORT, with interactions and the hierarchy condition, is a two-stage algorithm. FS without interactions is the first step, which is the procedure described in the preceding section. The second stage builds on the model from stage one, which includes the selected main effects. It builds on the stage-one model by performing FS with all the interactions that follow the hierarchy condition in the candidate set.

1st stage: The first stage uses FS to identify the main effects in the model. The selection criterion is BIC_2 , and the stopping criteria are as outlined in the previous section. We pick the model with the lowest BIC_2 . We will now support the choice of BIC_2 as the selection criterion. As mentioned, using BIC_2 allows us to account for the overfitting caused by a large number of predictors. This is desired since this algorithm is directed at HDD cases. We avoid overfitting with BIC_2 because of the term $\frac{k}{n}(\ln(n) + 2\log(d))$ (notice the part $\frac{k}{n}2\log(d)$ does not exist in BIC_1). The purpose of the term is to penalize the ratio of the selected variables (k) over the maximum number of covariates to select (n). We want to have the most suitable weight for this penalization. The BIC_2 criterion weighs that penalization with the term $\ln(n) + 2\log(d)$. Including that weight makes the penalization more strict as the number of samples increases (due to $\ln(n)$) and as the number of predictors increases (due to $2\log(d)$). Notice that the penalization is stronger towards the increasing number of predictors. Moreover, it has been shown by Chen and Chen that adding the extra predictor's penalization term reduces the False Discovery Rate (FDR) [8]. The FDR, here, is a measure of how many variables were mistakenly chosen to enter the model. The aforementioned makes BIC_2 a suitable selection criterion when we have many predictors to choose from.

2nd stage: The second stage is FS, aiming to identify interaction effects. The selection criterion is BIC_1 , and the stopping criteria remain the same. Below, we will argue

why we can now switch to BIC_1 . At this stage, the main effects have already been selected, and due to the hierarchy condition, the number of feasible interactions is restricted. So, we can effectively use BIC_1 that drops the penalty for the large number of predictors. This is shown through simulations to be a safe option as we expect the number of main effects picked from stage 1 to be minimal [14].

To clarify the process, we will go through an iFORT algorithm example. Consider the full model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_1^2 + \beta_5 X_1 X_2 + \beta_6 X_1 X_3 + \beta_7 X_2^2 + \beta_8 X_2 X_3 + \beta_9 X_3^2 + \varepsilon$$

Now, assuming X_1 and X_2 were selected by FS as main effects from the first stage, we are ready to expand for interactions. When following the hierarchy condition, $(X_1^2, X_1 X_2, X_2^2)$ are all the possible interactions. Consider the current indicator vector $(1, 1, 0, 0, 0)$. This is the starting point of stage 2 of iFORT and denotes that we start with X_1 and X_2 and without $X_1^2, X_1 X_2, X_2^2$ as can be seen in Figure 3.3. The algorithm progresses following the yellow arrows based on the selection criteria, which now becomes BIC_1 .

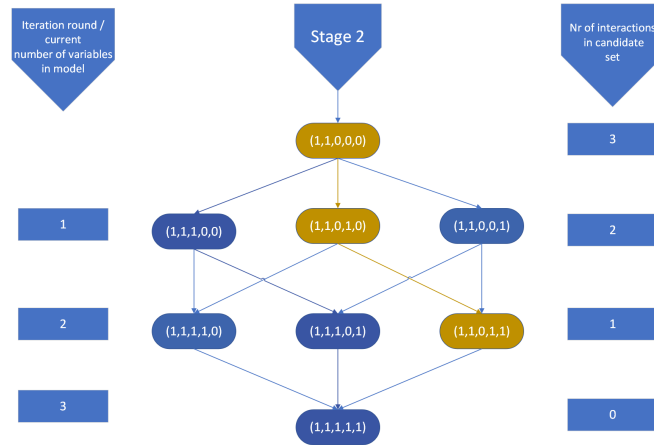


Figure 3.3.: interaction FS example solution in yellow.

iFORM

Now we move to iFORM, an improved version of iFORT, which is what we will be using for later analyses. iFORM works in one stage. This is advantageous because in the two-stage version, there is a bias towards the early-added coefficients' main effects. This is especially problematic for interaction screening, because an interaction term might explain the response better than adding another main effect, but (when we prioritize adding main effects), adding that main effect could overshadow the more informative interaction.

Below, we describe the procedure of iFORM. First, we specify the criteria of this method. The Stopping criteria remain the same as iFORT, and the selection criterion is the "enhanced BIC ", namely BIC_2 . Then, assuming the full model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_1 X_2 + \beta_5 X_2^2 + \varepsilon,$$

we show how iFORM works through a simple example. Suppose the first covariate added to the model is X_2 . Notice in the illustration 3.4 the covariate X_2^2 appearing in

the pool of available coefficients together with the remaining main effects (in this simple example, it is just X_1). Then, X_2^2 is added. After another main effect is added, we can see more interactions entering the pool. The process is analogous for a larger number of covariates and follows the steps outlined in the pseudo Algorithm 3.

Algorithm 3 iFOR under marginality principle (iFORM)

- 1: **Step 1 (Initialize):** $s_0 = \emptyset$, $m_0 = \emptyset$, $c_0 = P$ ▷ Initialize sets of indices
 - 2: **Step 2 (Build Solution path):**
 - 3: **for** each t in $\{1, \dots, D\}$ **do** ▷ D chosen constant below or equal n
 - 4: Given s_{t-1} , c_{t-1} , and m_{t-1} , use FS to select +1 predictor from $c_{t-1} \setminus s_{t-1}$ into the model
 - 5: Add the +1 into s_{t-1} to get s_t
 - 6: **if** +1 is a main effect **then**
 - 7: Update c_t and m_t
 - 8: **else**
 - 9: $c_t = c_{t-1}$
 - 10: $m_t = m_{t-1}$
 - 11: **end if**
 - 12: **end for**
 - 13: **Step 3:** Calculate BIC_2 for each $s_t \in \{s_1, \dots, s_D\}$ models in the solution path and choose the one with the lowest BIC_2
-

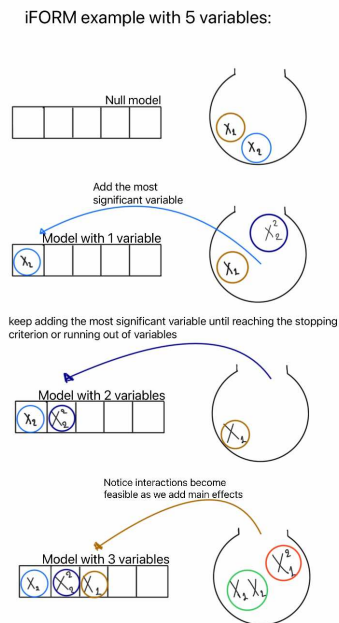


Figure 3.4.: Illustration of iFORM's example with 5 variables. In this Figure we see how interactions become available as soon as their main effects are in the model.

Comparison of iFORM with interaction FS

In this section, we highlight the strengths of iFORM in comparison to traditional interaction Forward Selection (iFS).

First, we will go through the setup of the comparison. A script was run to compare the performance of iFS over iFORM. The script was run on the simulated datasets of different structures for comparison. The dataset structures are described in section B.2 of the appendix. The criterion of comparison was RSS. The specifications of the local machine can be found in Table 3.1.

Component	Specification
Operating System	Windows (Laptop)
Processor (CPU)	AMD Ryzen 7 4000 Series
Memory (RAM)	15.7 GB
Graphics Card (GPU)	AMD Radeon (TM)

Table 3.1.: Hardware specifications of the system used for the simulation.

We can draw conclusions from the resulting Figure 3.5. As we can see, for datasets with a large number of predictors, we do not have results from the iFS model. That is because for these datasets, we run into the error "cannot allocate vector of size 58.4 Gb". This error occurs when R runs out of memory while trying to create a vector of big size during the iFS step. When $p = 500$, the number of interactions is $\binom{500}{2} = 124.50$, which leads to a design matrix of $124.750 + 500$ columns in total. Another observation is that the performance of iFORM improved with the larger number of predictors, showcasing that iFORM's adjustments are effective.

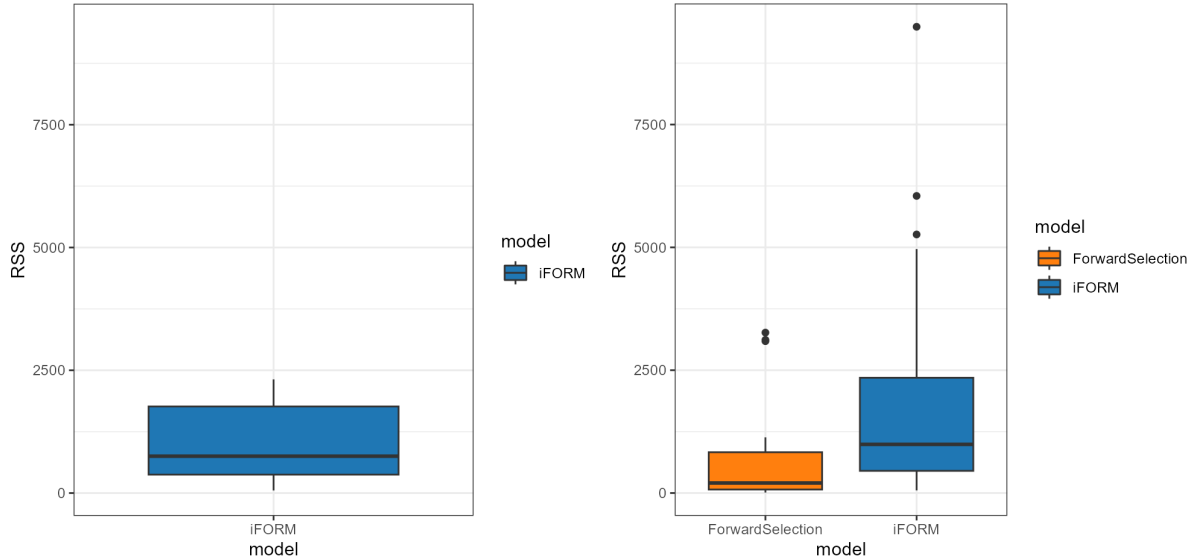


Figure 3.5.: Performance of plain interaction FS(orange) over iFORM(blue). The right photo shows results from fits on datasets with $p=500$, while the left photo shows results from datasets with $p=20$.

Theoretical properties

Ning Hao and Hao Helen Zhang were excited to report a key property of their adjusted Forward Selection (FS) method: as the number of samples increases, their approach guarantees the selection of the correct variables. This ensures that there exists a large enough number of samples so that the method will recover the true model. Remarkably, this is achieved while keeping computational complexity linear in p in both iFORT and iFORM, even in HDD settings, provided certain key conditions hold. These four conditions are:

1. **Well-behaved errors:** The residuals should not have extreme outliers or high variance that could disrupt model selection.
2. **Strong signal strength:** The true coefficients must be large enough to be distinguishable from noise, ensuring reliable selection.
3. **Well-conditioned covariance matrix:** The covariance matrix describes the collinearity of covariates. An ill-conditioned covariance matrix would mean a large condition number. A large condition number indicates that the result of the algorithm is not stable (e.g., slight changes in the data could give a very different final model).
4. **Sparsity of the true model:** The true model should include only a relatively small number of relevant variables compared to the total number of predictors.

Under these conditions, the iFORT/iFORM algorithm is guaranteed to converge to the true model in

$$O(n^2p + n^4)$$

steps [14]. A detailed mathematical analysis of this result can be found in Appendix A.

3.1.3. Extension to logistic regression

We will now sketch how iFORM extends when we fit a logistic instead of a linear regression model with interactions.

All forward selection type of algorithms generally do:

1. Start with no predictors
2. For every candidate variable not yet in the model:
 - Fit the model with that variable added.
 - Compute some score (goodness of fit/predictive performance).
3. Add the variable that improves the score the most (maybe adding more selection criteria, such as a coefficient threshold)
4. Repeat until no remaining variable improves the model “enough”.

The above logic is identical for linear and logistic regression; however, we need to be careful with the selection criterion. BIC also works for logistic regression, but we need to adjust it as RSS is no longer suitable. That is because RSS is defined to sum the error distances from the true data points to the fitted line (red lines in Figure 2.1). Therefore, we will replace the first term of equation 3.2 with the log likelihood function as given in

section 2.7.

Another difference is that we no longer have a closed-form solution as in linear regression. For logistic regression, we use maximum likelihood estimation and rely on iterative numerical methods like the one described in section 2.7. Because we've moved from a closed-form solution to an iterative one, forward selection becomes computationally heavier.

3.2. Backtracking with Lasso (BT)

Now, we want to do the same: build a predictive model, but include penalization and interaction effects to make it more efficient. Backtracking can help us build models efficiently. We will briefly mention the advantages, selection, and stopping criteria of this method, then give a detailed example, and finally discuss the insights of Backtracking with Lasso. For the rest of the analysis, we will sometimes refer to Backtracking with Lasso as BT.

3.2.1. Overview

The backtracking variation of Lasso is a worthy method to consider because of the following advantages:

- A1. It examines different starting setups (Candidate set C and λ -grid) to build a linear regression model augmented with interaction terms of the following form¹

$$\mathbf{Y} = X_C \boldsymbol{\beta}_C + \boldsymbol{\epsilon}.$$

This aims to maximize the chance of finding a model with minimal noise variables and maximal predictive accuracy, more effectively than classic Lasso.

- A2. It uses previously done computations and parallel computing processes to make the algorithm more time-efficient by applying the KKT condition in an efficient way.
- A3. It is a dynamic method that can add and remove variables to the model before it concludes.

We can start analyzing how it works by listing the selection and stopping criteria.

1. Selection criterion, a variable is selected to enter the model when, given the current iteration k , candidate set C_k and value of λ , the coefficient becomes nonzero when solving the Lasso minimization problem:

$$J(\boldsymbol{\beta}) = \frac{1}{N} \|\mathbf{Y} - X_{C_k} \boldsymbol{\beta}_{C_k}\|_2^2 + \lambda \|\boldsymbol{\beta}_{C_k}\|_1. \quad (3.3)$$

Specifically, we use KKT conditions to identify where the solution path starts to deviate from the one in the previous iteration. When it deviates, it means a new coefficient has become non-zero. Recall the KKT inequality 2.16. If 2.16 holds for all newly entered $\mathbf{X}_j$'s in the candidate set of the current iteration (C_k), we know

¹as discussed in Section 2.4.1.

$\hat{\beta}$ remains unchanged from the previous iteration. We will denote the part of the design matrix with the new $\mathbf{X}_j$'s as $X_{C_k \setminus C_{k-1}}$. Equivalently to checking KKT for all new $\mathbf{X}_j$'s we use the norm $\|\mathbf{x}\|_\infty = \max(|x_1|, \dots, |x_2|)$ to group all the checks in one. From the above, we conclude we want to find the smallest λ such that

$$\frac{1}{n} \|X_{C_k \setminus C_{k-1}}^\top (\mathbf{Y} - \mathbf{X}_{C_k} \hat{\beta}(\lambda, C_{k-1}))\|_\infty \leq \lambda \quad (3.4)$$

does not hold anymore. The above means at least one of the $\mathbf{X}_j$'s doesn't fall in the shrinkage area, and we have a new non-zero coefficient:

$$\hat{\beta}_{C_k \setminus C_{k-1}}(\lambda, C_k) \neq \mathbf{0}$$

and the rest of the solution path is the same,

$$\hat{\beta}_{C_{k-1}}(\lambda, C_k) = \hat{\beta}(\lambda, C_{k-1}).$$

Lastly, we will underline some technical notes. First, the term $X_{C_k \setminus C_{k-1}}^\top$ is not in bold and $\hat{\beta}_{C_k \setminus C_{k-1}}$ is in bold cause we refer to the general case where more than one new candidate variables entered the augmented design matrix. Second, this selection criterion holds from the second iteration. The first iteration is just regular Lasso.

2. Stopping criteria:

- We limit the maximum number of interactions that can be augmented per step to avoid a large number when the active set is very large.
- We limit the maximum number of iterations.
- Once all λ 's on the grid have been tested without a new interaction appearing as a candidate covariate, we finish the process.

3.2.2. Example

We will now go through an example that showcases the mentioned advantages. When explaining the steps, we will pinpoint sentences that show the mentioned advantages by mentioning the corresponding advantage in parentheses. The example and related figures are taken from the paper "*Modeling Interactions in High-dimensional Data with Backtracking*". Given the true model:

$$\mathbf{Y} = \sum_{i=1}^6 \beta_i \mathbf{X}_i + \beta_7 \mathbf{X}_1 \mathbf{X}_2 + \beta_8 \mathbf{X}_3 \mathbf{X}_4 + \beta_9 \mathbf{X}_5 \mathbf{X}_6 + \varepsilon \quad (3.5)$$

Assume variable 5 only contributes to the model, provided interactions $\mathbf{X}_1 \mathbf{X}_2$ and $\mathbf{X}_3 \mathbf{X}_4$ are already included. We can identify cases where a different algorithm, other than BT, would not yield good results. If we were to find first the main effects to build a hierarchical model, we would get a solution path as in Figure C.1(a). Notice that covariate 5 is falsely missing from the solution path, and noise is included. If we were to add all interactions in the candidate set from the beginning, we would get a solution path as in Figure C.1(b). Notice again how main effects 4- 5- 6 are neglected (and therefore, all the models with their interactions involved are not feasible since the final model will

follow the hierarchy condition), and noise is included. From the above, we can conclude that the solution path can change significantly with a change in the candidate set.

Below we will go through the steps of the algorithm, and we will refer to λ_x^j , where x will be an action we take at that λ , and j will be the corresponding iteration at which the action happens.

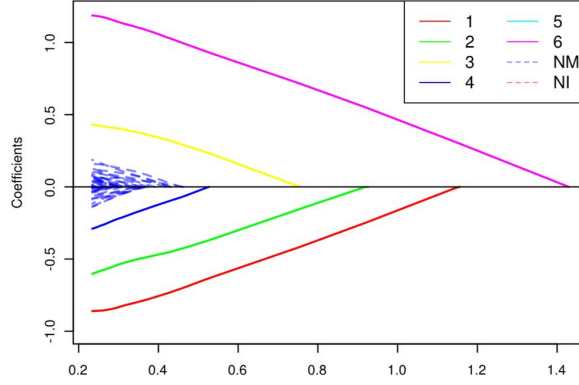


Figure 3.6.: Step 0: Regular Lasso. The plot illustrates the solution path of coefficients versus λ values. Each $\mathbf{X}_i$ is color-coded by assigning a unique color to covariate i . The NM and NI labels correspond to noise related to main and interaction effects, respectively.

Step 0: Lasso with all main effects in the candidate set C_1 . We obtain $\hat{\beta}(\lambda, C_1)$. (see Figure 3.6).

Step 1 (backtracking actually starts):

In this step the algorithm performs three tasks:

- (1) a new main effect enters the model and selects the first interaction to augment the design matrix of Step 2,
- (2) selection criterion 3.4 fails and picks an efficient value of λ at which to begin Step 2, and
- (3) computes the Lasso estimates $\hat{\beta}$ under the current candidate set for all λ 's on the λ -grid.

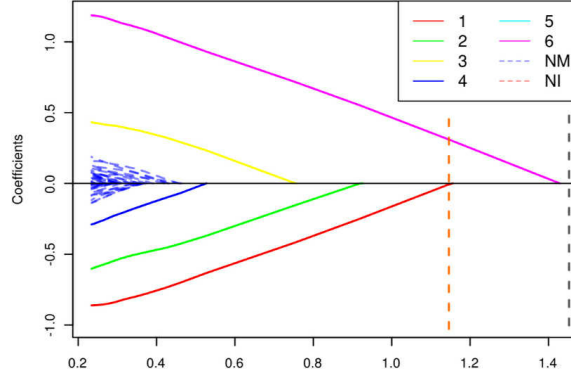


Figure 3.7.: (a) Step 1: We iterate on the λ -grid until we have two main effects, and therefore we can add the first interaction in the candidate set. Vertical dashed gray and orange lines give the values of $\lambda_{start}^1 = \lambda_{max}$ and λ_{add}^1 respectively.

Here we will analyze task **(1)**: We begin with only the main effects in the candidate set C_1 . Iterating over the grid of λ values², we stop at the λ that allows the first two main effects ($\mathbf{X}_6, \mathbf{X}_1$) to enter the model (orange dashed line in Figure 3.7). This allows us to add the first interaction $\mathbf{X}_1\mathbf{X}_6$, subject to the hierarchy condition, into the candidate set of the next step C_2 (this corresponds to advantage A1). We complete task (1) by recording that λ and call it λ_{add}^1 .

Now we proceed to task **(2)**: This check exploits the fact that if no new interaction from the candidate set enters the model (selection criterion fails) at a given λ , then the solution path from λ_{max} down to that λ remains unchanged. Consequently, in the next step, we can begin our search at this smaller λ instead of restarting from λ_{max} . Because with λ_{add}^1 the selection criterion for the new interaction variable is not passed, we make λ_{add}^1 the starting λ of the next step and call it λ_{start}^2 of step 2 (task (2) is done). To conclude, we underline that by the nature of the selection criterion 3.4 we know that if it is not passed for λ_{add} then it will not be passed for any stricter (larger) λ either and therefore the solution path of step 2 will coincide with the solution path of the current step up to λ_{add} . As a result, we can set

$$\lambda_{start}^2 = \lambda_{add}^1.$$

This way, we have a narrowed grid to examine for step 2, and we have saved computations by not refitting the Lasso models for the λ 's in $[\lambda_{add}^1, \lambda_{max}]$ (advantage A2).

Finally, action **(3)** is to finish the solution path by calculating $\hat{\beta}$ for the rest of the λ 's in $[\lambda_{min}, \lambda_{add}^1]$. That is the area on the left of the orange dashed line in Figure 3.7.

Step 2:

Next, we start Step 2 with the updated candidate set (C_2) and λ_{start}^2 (see the gray dashed line in Figure 3.8). In this step, the algorithm performs three tasks:

²Picked as described by 2.18, 2.19.

- (1) a new main effect enters the model and selects more interactions to further augment the design matrix of Step 3,
- (2) selection criterion 3.4 succeeds for one interaction and searches for an efficient value of λ at which to begin Step 3, and
- (3) computes the Lasso estimates $\hat{\beta}$ under the current candidate set for all λ 's on the updated λ -grid.

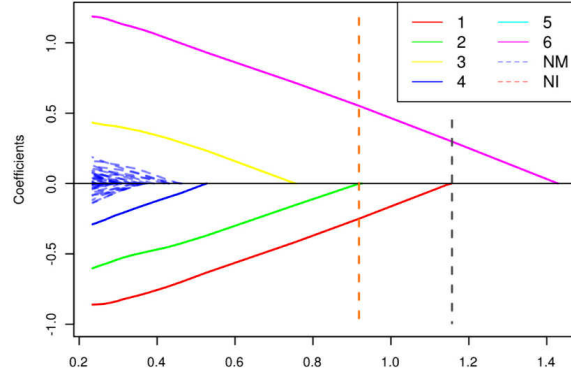


Figure 3.8.: Step 2: We iterate on the remaining λ -grid until we have a new main effect to make interactions with.

Below we will expand on each task:

(1) We iterate over the remaining of the λ -grid until main effect $\mathbf{X}_2$ enters the model and interactions $\mathbf{X}_1\mathbf{X}_2$, $\mathbf{X}_2\mathbf{X}_6$ enter the candidate set of step 3, C_3 .

(2) We check the selection criterion, and one new interaction covariate is selected. We can see it becoming non-zero in the solution path of the next step in Figure 3.9. We conclude that λ_{start}^3 will be the smallest $\lambda \in [\lambda_{add}^2, \lambda_{max}]$ such that the selection criterion for the new covariate is not yet satisfied (we backtrack in the solution path). That is the last lambda on the grid for which the interaction covariate is zero in the solution path, indicated by the gray line in Figure 3.9(a).

(3) Finally, we obtain $\hat{\beta}$ for the remaining of the solution path for C_2 .

Step 3 and Beyond:

Continuing on the example and generalizing for all steps, the algorithm performs the following tasks:

- (1) selects a new main effect and augments the design matrix of the next step with the new feasible³ interactions,
- (2) checks the selection criterion 3.4 for all new feasible interactions. If all of them fail the criterion at step k , $\lambda_{start}^{k+1} = \lambda_{add}^k$. If at least one doesn't, it searches for an efficient value of $\lambda \in [\lambda_{add}^k, \lambda_{max}]$ at which to begin step $k + 1$, and

³Under hierarchy condition.

- (3) computes the Lasso estimates $\hat{\beta}$ under the current candidate set for all λ 's on the remaining λ -grid.

Below we will expand on each task:

- (1) Every time, the algorithm, as soon as we reach the lambda of the grid where a new main effect enters the model, it adds the new feasible interactions to the candidate set of the next step and keeps that λ_{add}^k .

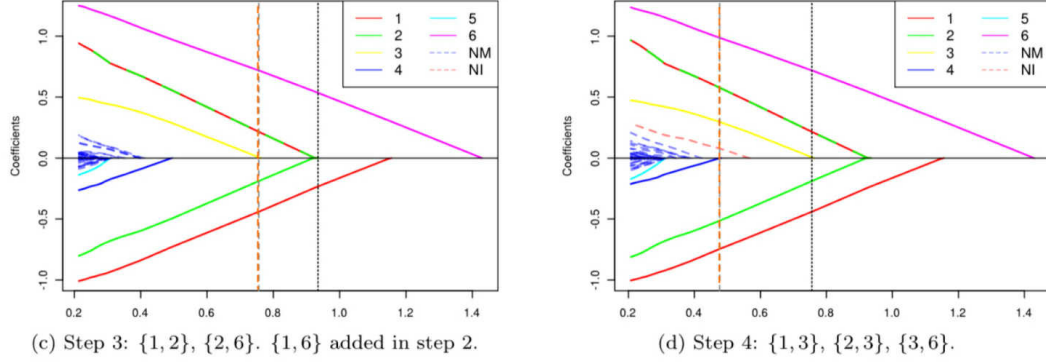


Figure 3.9.: (b) Step 3&4: Example of how a failed selection criterion on step 3 affects the starting λ of step 4.

(2) If none of the new interactions explains sufficient amount of unexplained variance of residuals then the lambda grid is updated to values from $[\lambda_{min}, \lambda_{add}^k]$. For example, see Figure 3.9. As soon as main effect 3 entered the model (given by orange line of step 3 in Figure 3.9), interactions $\mathbf{X}_1\mathbf{X}_3$, $\mathbf{X}_2\mathbf{X}_3$, and $\mathbf{X}_3\mathbf{X}_6$ were added to C_4 but none of them satisfied the selection criterion so λ_{start}^4 (given by gray line of step 4 in Figure 3.9) was equal to λ_{add}^3 (given by orange line of step 3 in Figure 3.9).

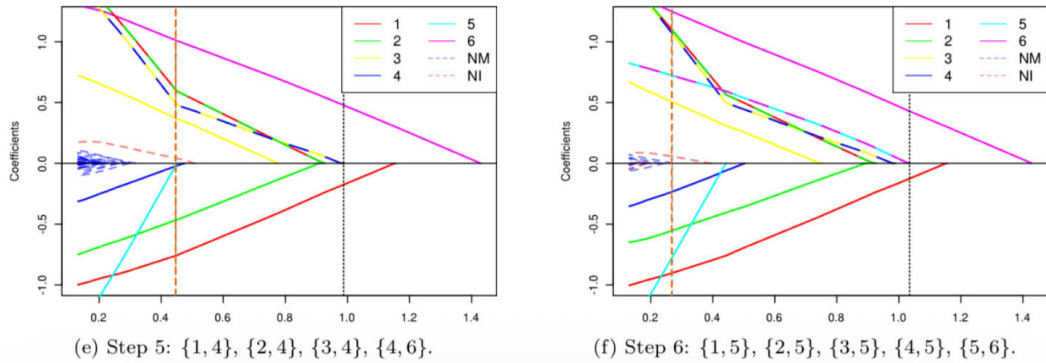


Figure 3.10.: (b) Step 5&6: Example of how a successful selection criterion on step 5 affects the starting λ of step 6.

And, how noisy terms are reduced from step 5 to step 6.

If at least one of the interactions does not fall in that category (does get selected) then we backtrack among the interval $[\lambda_{add}^k, \lambda_{max}]$ to find the smallest λ for which all the new interactions are shrunk to 0 and we make that the λ_{start} of the new iteration. For example, see Figure 3.10. As soon as main effect 5 entered the model (given by orange line of step 5 in Figure 3.10), we added all the feasible interactions to C_5 , interaction

$\mathbf{X}_5 \mathbf{X}_6$ was satisfying the selection criterion so we had to backtrack to find the smallest λ for which the criterion is not satisfied anymore and therefore λ_{start}^6 (given by gray line of step 6 in Figure 3.10) was different than λ_{add}^5 .

(3) After deciding on the λ_{start}^{k+1} , the starting λ of the next step, we finish the solution path.

We continue performing tasks (1),(2),(3) until the stopping criteria are met. Notice that at the end of the example algorithm (see Figure 3.10 (f)), the solution path includes all the true variables (in equation 3.5), and with the right choice of λ we can get a very accurate estimation with minimal noise.

3.2.3. Review

Now we can compare backtracking with iFOR. While iFOR is an excellent choice when simplicity is the goal, Backtracking offers several advantages over iFOR. One key difference is that iFOR's choice of variables to enter the model depends strongly on the order in which they are selected. This results in two problems that are alleviated in BT. First, any variable can enter the model more easily in the initial steps of iFOR's algorithm. This problem is less severe in the current algorithm because the decision on whether the variable enters the model or not depends on the Candidate set and λ , which get frequently updated also going backward (thus the algorithm is called backtracking), and we can revisit the same grid of lambdas at any iteration if necessary. Second, once a variable enters the model, it cannot be removed in iFOR. Meanwhile, with Lasso Backtracking, even if a variable enters the model at a certain λ , it can be removed in the next step. An illustration of the previous point can be found in Figure 3.10. Notice in (e) that a noise variable (red dashed line) has entered the model at $\lambda \approx 0.5$ while in (f), for the same λ , the noise variable is no longer part of the model. Consequently, for the Lasso model with $\lambda \approx 0.5$, a noise variable was selected on step 5, but it was removed on step 6 for that lambda. The only remaining practical challenge is that we cannot give a strict upper bound on runtime (unlike iFORM), since the number of backtracking rounds (and hence the total cost) depends on how often new variables enter as we move through the penalty grid.

3.2.4. Extension to logistic regression

Here we give the general idea of how Backtracking for Lasso extends to logistic regression. The minimization function is the negative log-likelihood as given in section 2.3, plus the L1 penalty on coefficients, and it is always evaluated using the candidate covariates in the design matrix. The KKT conditions have the same form as in the linear Lasso case, the only difference is how residuals are represented. Now residuals are calculated in terms of the difference of the true $\mathbf{Y}$ (which is a vector of n entries for a binary class) minus p_i , the probability that observed sample i is in class 1.

Algorithm 4 Iterative Backtracking Algorithm with Stopping Criteria

Require: $C_0 = \{1, \dots, p\}$, $\max_interactions = T$

```

1:  $\lambda_{start}^0 \leftarrow \lambda_{max}$ 
2: for  $k = 0$  to  $T - 1$  do
3:   (1) Select and augment: Scan  $\lambda$  from  $\lambda_{start}^k$  down to  $\lambda_{min}$  using Lasso on  $C_k$ 
      until the first new interaction (satisfying hierarchy) enters.
4:   Record this value as  $\lambda_{add}^k$  and set  $C_{k+1} \leftarrow C_k \cup \{\text{new interaction}\}$ .
5:   if no interaction entered then
6:     terminate  $\triangleright$  no new interactions detected
7:   end if
8:   (2) Check selection criterion 3.4 and determine next  $\lambda_{start}$ :
9:   if no interaction is selected at  $\lambda_{add}^k$  then
10:    Set  $\lambda_{start}^{k+1} = \lambda_{add}^k$ 
11:   else
12:    Search (by backtracking) for the smallest  $\lambda_{start}^{k+1} \in [\lambda_{add}^k, \lambda_{max}]$  that does not
      satisfy 3.4 and make it  $\lambda_{start}^{k+1}$ .
13:   end if
14:   (3) Compute solution path: Finish  $\hat{\beta}(\lambda, C_{k+1})$  for all  $\lambda \in [\lambda_{min}, \lambda_{start}^{k+1}]$ .
15: end for

```

3.3. Pliable Lasso (PL)

3.3.1. Method Introduction

Now, we will consider an algorithmic method of covariate selection and coefficient estimation on a slightly different model called the pliable Lasso (PL). With PL, we will examine a different way of incorporating interaction covariates and, therefore, a slightly different coefficient estimation method than regular Lasso. We will utilize certain covariates to build interactions, that we will call modifying covariates. Assuming we have n samples of k modifying covariates ($\mathbf{Z}_i$) and p covariates ($\mathbf{X}_j$), the model is:

$$Y_i = \beta_0 + \mathbf{Z}_i^\top \boldsymbol{\theta}_0 + \mathbf{X}_i^\top \boldsymbol{\beta} + \sum_{j=1}^p (X_{ij} \mathbf{Z}_i^\top) \boldsymbol{\theta}_j + \varepsilon_i, \quad (3.6)$$

or else,

$$\mathbf{Y} = \beta_0 \mathbf{1} + \mathbf{Z} \boldsymbol{\theta}_0 + \mathbf{X} \boldsymbol{\beta} + \sum_{j=1}^p \mathbf{W}_j \boldsymbol{\theta}_j + \boldsymbol{\varepsilon} \quad (3.7)$$

where

$$\mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n \times 1}$$

and

$$\mathbf{W}_j = \mathbf{X}_j \odot \mathbf{Z}$$

is the matrix created by the "Hadamard product" of $\mathbf{X}_j, Z$.

$$\text{Let } \mathbf{X}_j = \begin{bmatrix} X_{1j} \\ X_{2j} \\ \vdots \\ X_{nj} \end{bmatrix}, \quad Z = \begin{bmatrix} Z_{11} & Z_{12} & \cdots & Z_{1m} \\ Z_{21} & Z_{22} & \cdots & Z_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ Z_{n1} & Z_{n2} & \cdots & Z_{nm} \end{bmatrix}.$$

The Hadamard product $\mathbf{X}_j \odot Z$ is then given by:

$$\mathbf{X}_j \odot Z = \begin{bmatrix} X_{1j} \cdot Z_{11} & X_{1j} \cdot Z_{12} & \cdots & X_{1j} \cdot Z_{1m} \\ X_{2j} \cdot Z_{21} & X_{2j} \cdot Z_{22} & \cdots & X_{2j} \cdot Z_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ X_{nj} \cdot Z_{n1} & X_{nj} \cdot Z_{n2} & \cdots & X_{nj} \cdot Z_{nm} \end{bmatrix}.$$

Here, $\odot$ represents the element-wise multiplication, where each element of the i -th row of Z is multiplied by the i -th element of $\mathbf{X}_j$. Each column of the matrix W_j is a covariate-modifier pair, which we will sometimes refer to as just "interaction".

We will now analyze the Pliable Lasso model and its components in the order they appear in the formula. The Formula 3.6 includes:

- The intercept terms $\beta_0, \mathbf{Z}_i^\top \theta_0$. Where $\mathbf{Z}_i$ is the $1 \times m$ dimensional row vector of the modifiers for the i -th sample and θ_0 is the $1 \times m$ dimensional vector with the corresponding coefficients.
- The linear combination of the sample i of covariates $\mathbf{X}_j$ with the coefficients β_j . That controls the contribution of main effects in the final prediction for sample i .
- The sum of the linear combination of θ_j with the i -th measurement of covariate $\mathbf{X}_j$ influenced by the multiplication with the vector of modifying covariates for sample i , $\mathbf{Z}_i$. That controls the contribution of modifying variables in the final prediction.
- The error term ε_i that accounts for the amount of unexplained variance in the i -th prediction.
- The parameters n, p, m represent the number of samples, the number of predictors, and the number of modifiers, respectively.

For this study, we will consider measurements and relevant modifying covariates to be known. That means the matrices X and Z are given, and we try to estimate the coefficients β_j and $\theta_j \forall j \in \{1, \dots, p\}$ by minimizing the following cost function:

$$J(\boldsymbol{\beta}, \boldsymbol{\Theta}) = \underbrace{\frac{1}{2n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}_A + \underbrace{(1 - \alpha) \lambda \sum_{j=1}^p ((\|\beta_j\|_2 + \|\theta_j\|_2))}_B + \underbrace{\alpha \lambda \sum_{\substack{j \in \{1, \dots, p\}, \\ k \in \{1, \dots, m\}}} |\theta_{jk}|}_C, \quad (3.8)$$

where

$$\Theta = \begin{bmatrix} \theta_{11} & \theta_{12} & \cdots & \theta_{1p} \\ \theta_{21} & \theta_{22} & \cdots & \theta_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \theta_{m1} & \theta_{m2} & \cdots & \theta_{mp} \end{bmatrix}.$$

We will analyze the terms of the cost function formula. Term A is the RSS term that we introduced in the introductory chapter. Minimizing RSS improves the fit of the model. Term B is designed to promote a hierarchical relationship between the main-effect coefficient β_j and the modifier coefficient $\boldsymbol{\theta}_j$. In particular, when $\beta_j = 0$, the j -th contribution to B satisfies

$$\|(\beta_j, \boldsymbol{\theta}_j)\|_2 + \|\boldsymbol{\theta}_j\|_2 = \|(0, \boldsymbol{\theta}_j)\|_2 + \|\boldsymbol{\theta}_j\|_2 = 2\|\boldsymbol{\theta}_j\|_2,$$

so keeping $\boldsymbol{\theta}_j \neq 0$ is relatively costly unless it yields a sufficiently large reduction in the residual sum of squares. Hence, the penalty encourages solutions consistent with a "*weak asymmetric hierarchical condition*". It is weak because an interaction $\mathbf{X}_i \mathbf{Z}_j$ doesn't require $\mathbf{Z}_j$ to be a main effect of the model. It is asymmetric because the terms address to

$$Z\boldsymbol{\theta}_j \neq 0 \implies \beta_j \neq 0 \text{ but not necessarily to } \beta_j \neq 0 \implies Z\boldsymbol{\theta}_j \neq 0.$$

Meaning, if the fitted effect of the modifiers on $\mathbf{X}_j$ is non-zero, then β_j is non-zero. Equivalently, we have that $\beta_j = 0 \implies Z\boldsymbol{\theta}_j = 0$ also holds. That means when β_j is 0, the overall effect of modifiers in $\mathbf{X}_j$ is 0, but we can still have individual θ_{jk} 's being non-zero. In other words, $Z\boldsymbol{\theta}_j = 0$ does not necessarily mean $\boldsymbol{\theta}_j = 0$ even in the non-trivial case that Z is not a zero matrix. That occurs when the columns of Z are collinear (if the concept of a collinear matrix is unknown to the reader we refer them to [31]), and in that case we do not have a unique solution for $\boldsymbol{\theta}_j$. Otherwise, $Z\boldsymbol{\theta}_j \neq 0 \implies \beta_j \neq 0$ is equivalent to $\boldsymbol{\theta}_j \neq 0 \implies \beta_j \neq 0$, and we can give a meaning to the effect of the modifier on covariate $\mathbf{X}_j$. If the covariate $\mathbf{X}_j$ is modified, term C controls the effect that is applied by the k -th modifying variable to be minimum. Finally, λ, α are tuning parameters.

In the final solution, we can have θ_{jk} 's being non-zero, even when β_j is 0 and Z has collinear columns. In an optimal solution, this would not happen. The term $Z\boldsymbol{\theta}_j$ would be 0, so the non-zero θ_{jk} 's would not contribute to minimizing the residual sum of squares and the $\boldsymbol{\theta}_j$ would be penalized strongly as a vector and with its individual entries through term C. Although, as we discuss later, we approximate the solution and we use tolerances. If θ_{jk} 's are non-zero in our final solution, they will still be very small (e.g. $\theta_{jk} = 10^{-6}$ and the tolerance to consider a number 0 is chosen to be 10^{-8}).

In this paragraph, we will explain the role of λ, α in the equation. Constant $\lambda \in [0, +\infty)$ and controls the level of regularization. Constant $\alpha \in [0, 1]$ moderates whether the model prioritizes "the selection of which covariates get modified" over "how they are modified" (e.g., selecting which specific modifier-covariate interactions matter). For instance, a *low* α results in a model that prioritizes selecting which covariates are worth being involved in modifications and a matrix Θ with many zero-vector columns. A *high* α means the model prioritizes selecting modifier-covariate pairs, and the matrix Θ would have many non-zero columns but with few non-zero entries each. Choosing a very high α can encourage non-hierarchical interactions to enter the model.

The pliable Lasso offers several advantages over regular interaction Lasso, particularly in settings where some variables naturally act as modifiers, as is often the case in biology. In such contexts, subtle conditional effects may be missed by the interaction Lasso, which treats all variables symmetrically. Conceptually, this lets you ask “how does the effect of $\mathbf{X}_j$ change with $\mathbf{Z}_k$?”. This directional question is more precise than the general one of “Does $\mathbf{X}_j$ interact with $\mathbf{Z}_k$?” and therefore can be advantageous. Additionally, interaction Lasso estimates $p + \frac{p(p-1)}{2}$ coefficients, including all pairwise interactions. In contrast, pliable Lasso estimates only $p + pm$ coefficients. While m could theoretically be up to $\frac{p(p-1)}{2}$, in practice it is much smaller. That is because modifiers are chosen based on contextual relevance, and θ_j undergoes a double penalty in pliable Lasso, which keeps a minimal number of θ_{jk} ’s non-zero, greatly reducing model complexity. However, in the special case $Z \subseteq X$ you end up with both $\mathbf{X}_1\mathbf{X}_2$ and $\mathbf{X}_2\mathbf{X}_1$ in the design matrix, which are numerically identical and perfectly collinear columns. That raises the concern of how the model can decide which covariate to attribute the variation in the response.

Here, we will discuss the special case where Pliable Lasso allows two perfectly collinear covariates. It allows separate coefficients θ_{12} for the interaction $\mathbf{X}_1\mathbf{X}_2$ and θ_{21} for $\mathbf{X}_2\mathbf{X}_1$. Coordinate-descent adjusted for the Pliable Lasso algorithm that we will discuss later, arbitrarily splits the interaction effect between the identical terms, depending on the order it encounters their coefficients. Neither θ_{12} nor θ_{21} individually has a clear interpretation, only their sum, $\theta_{12} + \theta_{21}$, is meaningful. Since the cost function remains convex (but not strictly convex), multiple combinations of these two coefficients give the same prediction (similarly to the regular Lasso case discussed in Section 2.6). In practice, coordinate descent first checks if the total correlation between $\mathbf{X}_1\mathbf{X}_2$ and the residual exceeds a specified threshold. If it does, it thresholds the first coefficient it encounters. When it subsequently encounters the identical second term, its correlation typically has been reduced by 1 threshold unit. Therefore, unless the amount of explained variance is high enough to survive a second thresholding after the main effects are also already in the model explaining part of the variance, the second coefficient is likely to be shrunk to zero. Thus, while the individual split is arbitrary and uninterpretable, the sum $\theta_{12} + \theta_{21}$ correctly represents the combined, once-shrunk interaction effect used in the prediction.

3.3.2. Simple example to understand modifiers

This is a hypothetical example of a clinical trial aimed at providing an understanding of the pliable Lasso model and the difference between regular and modifying covariates⁴. Suppose we have $n = 3$ observations, $p = 2$ predictors $\mathbf{X}_1$, $\mathbf{X}_2$, and $K = 1$ modifier. We will not use any units for simplicity. Suppose $\mathbf{X}_1$ represents the concentration of a biomarker, $\mathbf{X}_2$ represents the concentration of the drug dosage, and $\mathbf{Y}_i$ is a treatment response score for patient i . Specifically, $Z = X_2$. Our design matrix is:

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \quad , \quad Z = \mathbf{X}_2 = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

⁴In reality, we would need to estimate the coefficients, and the numbers would result in more complicated calculations.

We define the pliable Lasso model (same as 3.6 but $\mathbf{Z}$ is a vector instead of a matrix and therefore Z_i, θ_i, θ_0 are scalars for this simplified example):

$$\hat{Y}_i = \beta_0 + Z_i\theta_0 + \sum_{j=1}^2 X_{ij}(\beta_j + Z_i\theta_j) \quad (3.9)$$

Let the coefficients be:

$$\beta_0 = 0, \quad \theta_0 = 0, \quad \beta = [1, 2], \quad \theta = [0.5, -0.2]$$

For each observation, we compute the predictions:

$$\hat{Y}_i = \beta_0 + Z_i\theta_0 + X_{i1}(\beta_1 + Z_i\theta_1) + X_{i2}(\beta_2 + Z_i\theta_2) \quad (3.10)$$

Equation 3.10 highlights the difference of modifying covariates between regular ones. Modifying covariates act on the slope of regular ones. Regular ones have a constant effect on the response among patients. A modifying covariate can make regular effects vary on the response as the modifying covariate(s) varies.

For $i = 1$:

$$\hat{Y}_1 = 1(1 + 2 \cdot 0.5) + 2(2 + 2 \cdot (-0.2)) = 1(2) + 2(1.6) = 2 + 3.2 = 5.2$$

For $i = 2$:

$$\hat{Y}_2 = 2(1 + 3 \cdot 0.5) + 3(2 + 3 \cdot (-0.2)) = 2(2.5) + 3(1.4) = 5 + 4.2 = 9.2$$

For $i = 3$:

$$\hat{Y}_3 = 3(1 + 4 \cdot 0.5) + 4(2 + 4 \cdot (-0.2)) = 3(3) + 4(1.2) = 9 + 4.8 = 13.8$$

We will now interpret the coefficients of the modifiers.

$$\theta_1 = 0.5$$

- The effect of the biomarker X_1 **increases** as the drug dosage (modifier) $Z = X_2$ increases.
- Increasing $\mathbf{X}_2$ from $2 \rightarrow 4$, increased slope from $1 + 2 \cdot 0.5 = 2$ to $1 + 4 \cdot 0.5 = 3$

$$\theta_2 = -0.2$$

- The effect of X_2 **decreases** with its own value (self-modifying). In other words, the effect of increasing dosage of drug X_2 decays as its value increases.
- Increased $\mathbf{X}_2$ from $2 \rightarrow 4$ decreased slope from $2 + 2 \cdot (-0.2) = 1.6$ to $2 + 4 \cdot (-0.2) = 1.2$

3.3.3. Estimating coefficients

Notice that if θ_j is zero, then the current cost function is the same as the regular Lasso cost function. In that case, the solution is:

$$\hat{\beta}_j = \left(\frac{n}{\|\mathbf{X}_j\|_2^2} \right) S \left(\frac{\mathbf{X}_j^\top \mathbf{r}_{(-j)}}{n}, (1 - \alpha)\lambda \right), \quad (3.11)$$

where $S(\cdot, \cdot)$ is the soft-thresholding operator, identical to that used in the regular Lasso solution with tuning parameter $(1 - \alpha)\lambda$. The additional factor $\frac{n}{\|\mathbf{X}_j\|_2^2}$ accounts for cases where the covariate $\mathbf{X}_j$ is non-standardized. If the data are standardized, then (as shown in section 2.6.2), $\frac{n}{\|\mathbf{X}_j\|_2^2} \approx 1$. Otherwise, the term $\mathbf{X}_j^\top \mathbf{r}_{(-j)}$ is effectively being scaled by $\|\mathbf{X}_j\|_2^2$ instead of n . In 3.11 the residual is as defined by 2.17 because we assumed $\theta_j = \mathbf{0}$ but the general form $\forall L \in \{1, \dots, n\} \setminus \{j\}$ is:

$$\mathbf{r}_{(-j)} = \mathbf{Y} - \sum_{L \neq j} [\mathbf{X}_L \hat{\beta}_L + W_L \hat{\theta}_L]. \quad (3.12)$$

The general approximate solution comes from the update rule:

$$\begin{aligned} \hat{\beta}_j &= \frac{\beta_0 - t \nabla_{\beta_j} L}{c_1} \\ \hat{\theta}_j &= \frac{\|S(\theta_0 - t \nabla_{\theta_j} L, t\alpha\lambda)\|_2}{c_2} \end{aligned} \quad (3.13)$$

where L is the loss function defined as $L = \frac{1}{2n} RSS$. We will show how we arrive to that solution in the next section.

We choose which solution is suitable to assign among Equation 3.11 and 3.13 by checking the thresholds of the soft-thresholding function for θ_j . Before we analyze the algorithm, we note that β_0 and θ_0 are optional components in the model, and are unpenalized when included. Hence, for simplicity of notation, we omit them from consideration: at the beginning of each pass over the predictors, we estimate β_0 and θ_0 by regressing the current residual on the vector $(1, Z)$ [29]. In that case, the outcome $\mathbf{Y}$ appearing in the cost function 3.8 can be replaced by $\mathbf{Y} - \hat{\beta}_0 \mathbf{1} - Z \hat{\theta}_0$.

3.3.4. Pliable Lasso Algorithm

We will analyze the three cases for the pair $(\hat{\beta}_j, \hat{\theta}_j)$ to follow the corresponding update rules in the algorithm. One is the trivial solution where β_j and θ_j are both zero. Another is when β_j is non-zero and θ_j is zero, and the third is when both are non-zero. We will split the analysis into the first two cases where $\theta_j = \mathbf{0}$ and $\theta_j \neq \mathbf{0}$. The latter case is more tricky and requires special consideration.

Identifying $\theta_j = \mathbf{0}$ cases:

First, we want to identify conditions under which both β_j and θ_j are shrunk exactly to zero. Specifically, we split the problem into two simpler cases:

1. Assume now $\beta_j = 0$. That means both have to fall into the shrinkage area that corresponds to each for the specific scenario. That translates to the regular Lasso shrinkage area for β_j . Specifically, the shrinkage area for β_j is:

$$\left| \frac{\mathbf{X}_j^\top \mathbf{r}_{(-j)}}{n} \right| \leq (1 - \alpha)\lambda. \quad (3.14)$$

For $\boldsymbol{\theta}_j$, the problem resembles a group Lasso type of penalty on the vector $\boldsymbol{\theta}_j$, which generally does not yield a closed-form solution. However, we can derive the boundary condition defining the shrinkage area through subgradient optimality conditions (see Appendix A for detailed derivation). Intuitively, the resulting criterion involves a *nested soft-thresholding condition*, reflecting the simultaneous presence of a group-level (vector norm) threshold and an element-wise threshold.

Specifically, the shrinkage region for θ_j is:

$$\left\| S \left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, \alpha\lambda \right) \right\|_2 \leq 2(1 - \alpha)\lambda. \quad (3.15)$$

2. Assume now $\beta_j \neq 0$. We again find β_j similarly to regular Lasso through equation 3.11 and $\boldsymbol{\theta}_j$ must satisfy, similarly to the above case, the following shrinkage condition:

$$\left\| S \left(\frac{W_j^\top (\mathbf{r}_{(-j)} - \mathbf{X}_j \hat{\beta}_j)}{n}, \alpha\lambda \right) \right\|_2 \leq (1 - \alpha)\lambda. \quad (3.16)$$

Nested Soft-Thresholding

We will analyze the nested soft-thresholding condition 3.15 with the help of a numerical example.

Given:

- $n = 3$ observations, $m = 2$ modifiers
- Partial residual vector:

$$\mathbf{r}_{(-j)} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

- Matrix $W_j = \mathbf{X}_j \circ Z$:

$$W_j = \begin{bmatrix} 0.6 & -0.1 \\ -0.2 & 1.2 \\ 0.4 & -0.3 \end{bmatrix}$$

- Parameters: $\alpha = 0.6$, $\lambda = 1$

Step 1 We compute $\frac{W_j^\top \mathbf{r}_{(-j)}}{n}$. We will call this the signal vector, as it measures if the signal of the modified $\mathbf{X}_j$ is strong enough to survive penalization. The computation follows:

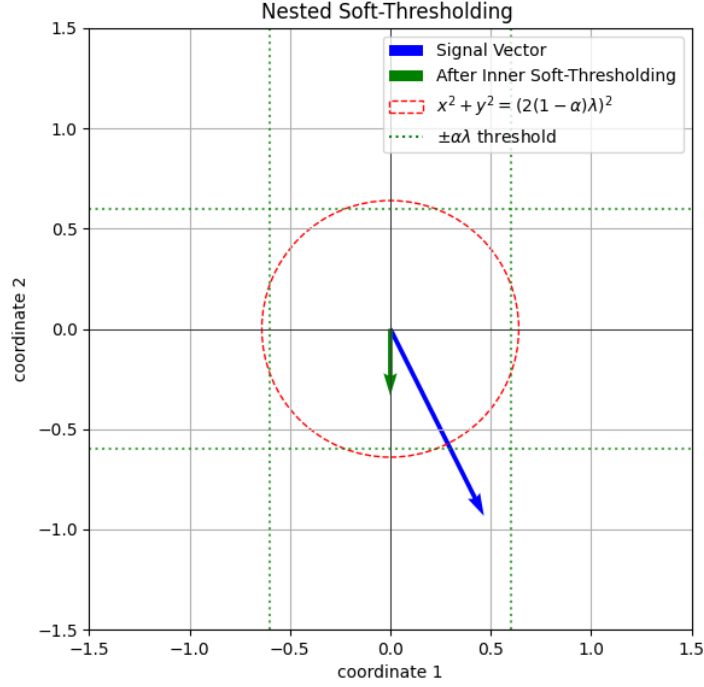


Figure 3.11.: The illustration depicts how the Nested soft Thresholding affects the signal vector. The green dotted lines represent the shrinkage boundary forced by the inner soft thresholding function. The red dashed circle represents the outer inequality boundary as specified by Inequality 3.14.

$$\frac{W_j^\top \mathbf{r}_{(-j)}}{n} = \frac{1}{3} \begin{bmatrix} 0.6 & -0.2 & 0.4 \\ -0.1 & 1.2 & -0.3 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1.4 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.467 \\ -0.933 \end{bmatrix}$$

Step 2 We apply element-wise soft-thresholding at $\alpha\lambda = 0.6$

We apply $S(\cdot, 0.6)$ to each coordinate:

$$S(0.467, 0.6) = 0, \quad S(-0.933, 0.6) = -0.333$$

So the vector after soft-thresholding becomes:

$$S\left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, 0.6\right) = \begin{bmatrix} 0 \\ -0.333 \end{bmatrix}$$

In Figure 3.11, we depict the above vector in green, as it "survived" soft thresholding and did not become zero.

Step 3 We check the group-level shrinkage condition:

$$\left\| S\left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, \alpha\lambda\right) \right\|_2 = \left\| \begin{bmatrix} 0 \\ -0.333 \end{bmatrix} \right\|_2 = 0.333 < (2(1 - 0.6))$$

Since $0 \leq 0.64$, the condition holds and the vector is eventually shrunk to zero.

Conclusion: The nested soft-thresholding condition is satisfied, so the optimal solution is:

$$\theta_j = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This example can help us intuitively understand how hierarchy condition is applied indirectly. We do not force hierarchy condition but when β_j is zero the penalty applied on θ_j is so strong that it becomes unlikely that a signal will be so strong it will survive (especially since the covariate alone did not have a strong signal in the first place). As we can see in the Figure 3.11, we would need a signal vector way further outside⁵ of the first boundary (green dotted box). For many dimensions, we cannot visualize it, but is analogous.

$\theta_j \neq 0$ case:

When θ_j is non-zero, we need to approximate the solution because there is no closed-form solution (similarly to group-Lasso type of algorithms with non-smooth regularization part). Below, we mention the four main actions we need to take and later we analyze them further. We need to:

1. parametrize β_j, θ_j with $\gamma_j = (\beta_j, \theta_j)$ because they are coupled in the penalty term.
2. majorize the approximation of L as this is done by Taylor, and Taylor tends to underestimate in its approximations.
3. use gradient descent to find the least squares solution $\tilde{\gamma}_j$.
4. use proximal gradient descent to find the regularized solution γ_j and then infer β_j, θ_j .

Our goal is to approximate:

$$J(\beta, \Theta) = \underbrace{\frac{1}{2n} \sum (Y_i - \hat{Y}_i)^2}_L + \underbrace{(1 - \alpha)\lambda \sum_{j=1}^p ((\|\beta_j\|_2 + \|\theta_j\|_2)) + \alpha\lambda \sum_{j,k} |\theta_{jk}|_1}_R.$$

The L part is smooth, and we can apply Taylor approximation. The second-order Taylor approximation of L around γ_0 is :

$$L(\gamma_j) \approx L(\gamma_0) + \nabla L(\gamma_0)^\top (\gamma_j - \gamma_0) + \frac{1}{2} (\gamma_j - \gamma_0)^\top H_L(\gamma_0) (\gamma_j - \gamma_0). \quad (3.17)$$

It is called a second-order approximation because we use up to the second-order derivatives of L . Taylor's theorem provides the above Taylor approximation, along with a bound for the error of the approximation. For further reading on Taylor approximation for convex functions, see the paper "Convex Functions and Taylor's Theorem"[22]. The mentioned paper informs us that Taylor's approximation tends to underestimate convex functions, such as L . This claim is easy to follow for a first-order Taylor approximation if we consider the characterization that given $f : \mathbb{R}^n \rightarrow \mathbb{R}$, f differentiable, then f is convex if and only if:

$$f(\mathbf{x}) \geq f(\mathbf{a}) + \nabla f(\mathbf{a})^\top (\mathbf{x} - \mathbf{a}) \quad \text{for all } \mathbf{x}, \mathbf{a} \in \text{dom}(f).$$

⁵Specifically, the signal vector should be larger than $\alpha\lambda + (2(1 - \alpha)\lambda)^2$.

To undo the underestimation effect of Taylor, we replace the second-order-derivatives term with another term that majorizes the approximation of the cost function. So, we introduce a majorizing function $M(\gamma)$, a surrogate that upper-bounds the original function while matching it at a specific point. Therefore, the majorized function is always equal to or greater than the original function we wanted to minimize. Minimizing a majorized surrogate is the concept of proximal gradient descent. The resulting majorized function is:

$$M(\gamma_j) = L_0 + (\gamma_j - \gamma_0)^\top \nabla L + \frac{1}{2t} \|\gamma_j - \gamma_0\|_2^2 + (1 - \alpha)\lambda(\|\gamma_j\|_2 + \|\theta_j\|_2) + \alpha\lambda \sum_{j,k} |\theta_{jk}| \quad (3.18)$$

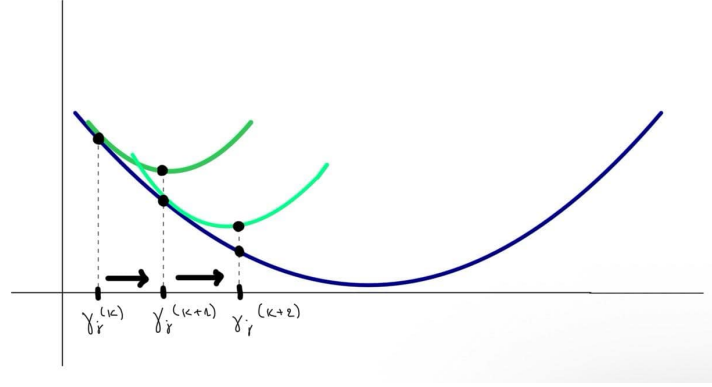


Figure 3.12.: The illustration describes the basic concept of proximal gradient descent. Instead of minimizing J (blue curve), for each j , we iterate over k and for each k we minimize a simpler function $\tilde{M}$ (green curves). The simpler function is above J and shares one common point. The γ_j^k that minimizes the simpler function is becoming the starting point that the new simpler function shares with J . We iterate over k until γ_j does not change much with new iterations.

Proximal gradient descent is an approximation method that simplifies the problem of minimizing the complex J by allowing us to minimize a simpler function in each iteration. We find the easier problems to solve by making numerical manipulations to upper bound the function, and that is how we go from J to M . The related numerical manipulations are further analyzed in the paper "*MM optimization: Proximal distance algorithms, path following, and trust regions*"[18]. And as long as γ_j is separable (there are no products of $\gamma_i \gamma_j$ in the equation), the occurring M is also a separable function, and that is why we can solve $M(\gamma_j)$ (instead of $M(\gamma)$). For more on proximal gradient for a general $J = L + R$, where R is a general penalty function, read "*Proximal Distance Algorithms: Theory and Practice*"[15].

After numerical manipulations on equation (3.4), the resulting majorized function is :

$$\tilde{M}(\gamma_j) = \frac{1}{2t} \|\gamma_j - \tilde{\gamma}_j\|_2^2 + (1 - \alpha)\lambda(\|\gamma_j\|_2 + \|\theta_j\|_2) + \alpha\lambda \sum_{j,k} |\theta_{jk}| \quad (3.19)$$

where γ_j is found by gradient descent with the following formula.

$$\tilde{\gamma}_j = \gamma_0 - t \nabla L$$

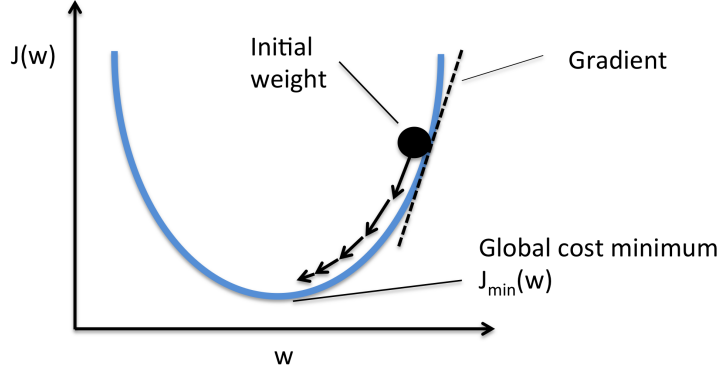


Figure 3.13.: Illustration of the general gradient descent algorithm. It minimizes J by finding the coefficients that assign the right weight to each covariate so that the cost function is minimized. It starts with a random initial guess and uses a step size that is multiplied by the size of the gradient, so that when the gradient is small (meaning we are close to the minimum), we take smaller steps so that we can precisely spot the minimum. The illustration uses J and w as generic notation for a cost function and an explanatory variable. In our case, gradient descent is applied to the least-squares component of the $\tilde{M}$ objective, with $\tilde{\gamma}_j$ as the corresponding explanatory parameter.

In order to find the γ_j that minimizes $\tilde{M}$ we have to:

1. follow an inner loop of iterations to first find $\tilde{\gamma}_j^{(k)}$, that minimizes the least squares part of $\tilde{M}$, through gradient descent.
2. use this $\tilde{\gamma}_j^{(k)}$ for the outer loop that finds $\gamma_j^{(k+1)}$ which minimizes $\tilde{M}$
3. continue iterating over k until $\|\gamma_j^{(k+1)} - \gamma_j^{(k)}\|_2 < \varepsilon$.

Specifically $\tilde{\gamma}_j$ satisfies (detailed calculations supporting the claim can be found in appendix A)

$$\begin{aligned} \left(1 + t(1 - \alpha)\lambda \frac{1}{\|\hat{\gamma}_j\|_2}\right) \hat{\beta}_j &= \tilde{\beta}_j \\ \left(1 + t(1 - \alpha)\lambda \left(\frac{1}{\|\hat{\theta}_j\|_2} + \frac{1}{\|\hat{\gamma}_j\|_2}\right)\right) \hat{\theta}_j &= S(\tilde{\theta}_j, t\alpha\lambda), \end{aligned} \quad (3.20)$$

where $\tilde{\beta}_j = \beta_0 - t \frac{\partial L}{\partial \beta_j}$ and $\tilde{\theta}_j = \theta_0 - t \nabla L$ are the gradient updates.

Finally, because the norm of gamma depends on both beta and theta we cannot directly find the estimate of beta and theta from the system of equations (3.6) and so we first estimate the norms with the following equations

$$\begin{aligned} \left(1 + t(1 - \alpha)\lambda \frac{1}{\sqrt{a^2 + b^2}}\right) a &= \tilde{\beta}_j \\ \left(1 + t(1 - \alpha)\lambda \left(\frac{1}{b} + \frac{1}{\sqrt{a^2 + b^2}}\right)\right) b &= S(\tilde{\theta}_j, t\alpha\lambda), \end{aligned} \quad (3.21)$$

and then we substitute them to (3.6) to end up with the final form of solution 3.13 for constants c_1, c_2 .

Algorithm 5 Pliable Lasso (Coordinate Descent Algorithm)

Require: Data matrix $X \in \mathbb{R}^{n \times p}$, modifiers $Z \in \mathbb{R}^{n \times m}$, response $\mathbf{Y} \in \mathbb{R}^n$.

Require: Regularization parameters $\lambda > 0$, $\alpha \in [0, 1]$ and tolerance threshold 0.05.

- 1: Initialize parameters: $\beta_j = 0$, $\theta_j = 0$ for all j , and intercepts β_0, θ_0
 - 2: **repeat** until convergence
 - 3: **for** each covariate $j = 1$ to p **do**
 - 4: Compute partial residual $\mathbf{r}_{(-j)}$ from equation 2.17
 - 5: Check soft thresholding conditions 3.14, 3.15 for $(\hat{\beta}_j, \hat{\theta}_j) = \mathbf{0}$. If zero, move to $(j + 1)$ -predictor.
 - 6: If $(\hat{\beta}_j, \hat{\theta}_j) \neq \mathbf{0}$: compute $(\hat{\beta}_j)$ from equation 3.11 and check updated soft-thresholded condition 3.16 for $\hat{\theta}_j$. If $\hat{\theta}_j = \mathbf{0}$, move to $(j + 1)$ -predictor.
 - 7: Otherwise, if $\hat{\beta}_j \neq 0$ and $\hat{\theta}_j \neq \mathbf{0}$,
 - 8: **repeat**
 - 9: Compute the norms from the system of equations 3.21.
 - 10: update the vectors from equations 3.20.
 - 11:
 - 12: **until** convergence
 - Ensure:** Estimated coefficients β_j, θ_j
-

3.3.5. Extension to logistic regression

To extend the pliable Lasso to the logistic setting, we make a few adjustments. Likewise to the extension of BT to logistic regression, we minimize the sum of a fit plus a penalty term, where now the fit term becomes a negative log likelihood function instead of a least-squares term. At each iteration of the algorithm, we approximate the log-likelihood using a partial quadratic (Taylor) expansion. This yields a weighted least-squares problem with the pliable Lasso penalty, which is directly analogous to the minimization problem in the standard pliable Lasso. We can then apply the existing algorithm to this weighted problem. The reader can view the related equations at the paper "*Pliable Lasso for the multinomial logistic regression*" [2].

4. Results & Discussion

4.1. Introduction

In this chapter, we present the results of the thesis and discuss what we can conclude from both the simulations and the case-study dataset. We start by describing the real data and the preprocessing steps required before model fitting, including scaling, handling missing values, and addressing class imbalance. Next, we report the simulation outcomes and use them to compare the three studied methods.

After the simulation study, we move to method selection and explain which approach is most suitable for the case-study objective, where interpretability and stability of selected variables matter. Since the true signal is unknown in real data, we complement cross-validation with a stability analysis to identify which variables and interactions are selected consistently. Finally, we summarize the modeling choice for the company use-case and provide practical deliverables and takeaways, including what the fitted models can (and cannot) support in terms of prediction and variable selection/interpretation.

Note that sometimes we will refer to the microRNAs with their index and sometimes with their name. To avoid confusion we have created a correspondence table, Table C.4.

4.2. Data

The data originate from Maastricht University Medical Center and, to protect patient privacy, all covariate names and patient IDs have been anonymized. The measurements were obtained using the Livak and S method for gene expression analysis, taking blood samples and using the plasma of the blood. The csv dataset contains 554 patient samples and 20 variables: one binary response column (1 = Mild Cognitive Impairment (MCI), 0 = no impairment) and 20 microRNA expression measurements. MicroRNAs are small RNA regulatory molecules whose dysregulation can indicate underlying disease processes, such as MCI.

In this paragraph, we outline the data analysis and preparation steps taken before applying our learning algorithms. First, we scaled (standardized) the measurements, which is a useful step to help reduce multicollinearity when introducing interactions, and for the methods BT and PL that use gradient descent to minimize the error in the coefficient estimation, it helps avoid convergence issues [21]. In addition, for the microRNAs we had missing values, we imputed them using the minimum observed value of the corresponding microRNA across all samples, following the company’s advice that the Livak and S method returns NA when the concentration of a given microRNA is very low. Next, we assessed class balance and found that 80% of samples belonged to the non-MCI class (label 0) and 20% to the MCI class (label 1), as is often the case with

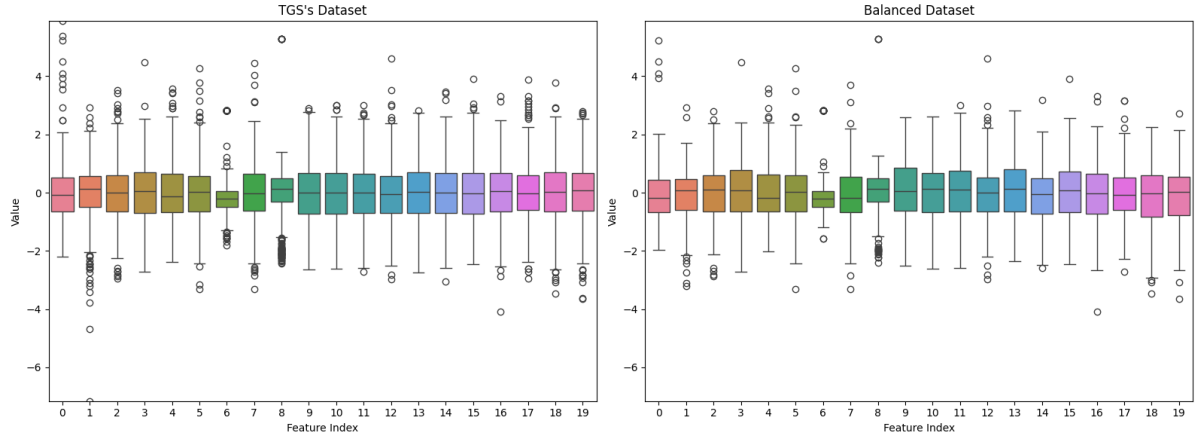


Figure 4.1.: Boxplots of the data distribution before (TGS's Dataset) and after down-sampling (Balanced Dataset)

biomedical datasets. Because such an imbalance can bias models toward the majority class, we chose to down-sample the majority class to balance the dataset. The previous steps were taken to make the fit easier, but we need to be careful on how to interpret the results given these adjustments. This topic will be further discussed in the deliverables to the company section. Finally, at the Table 4.1 and Figure 4.1, we get an overview of the dataset. Notice that the distribution of any microRNA does not change evidently as we move from the original to the down-sampled. The previous could indicate low signal, as for a microRNA M that $P(M|Y)$ is equal to $P(M)$ would indicate no relationship between them. This does not prove anything but is an indication to consider.

hsa_miR_X1a_3p	...	hsa_miR_X21t_5p	MCI
0.571	...	0.348	0
-0.607	...	0.449	1
-0.592	...	-1.000	1
-0.205	...	0.123	1
0.079	...	0.134	0

Table 4.1.: Overview of the case-study dataset.

4.3. Method Selection

4.3.1. Introduction

We will show how we selected the best method to apply to the company's dataset. To do so, we address two key questions: *"Which method is best at recovering the true effects?"* and *"Which method minimizes the prediction error?"*.

Our answers rely on a simulation study based on 54 datasets that vary in sample size, number of predictors, noise level, and correlation structure among predictors. A detailed description of the simulation design can be found in Appendix.

4.3.2. Metrics

We will quantify performance on true effect recovery using the metric called $F1$ that includes two other metrics called $Precision(P)$ and $Recall(R)$ that are defined as follows:

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Figure 4.2.: Illustration of the relationship between True Positives(TP), True Negatives(TN), False Positives(FP) and False Negatives(FN).

Precision

$$P = \frac{TP}{TP + FP} \quad (4.1)$$

“Of all predicted positives, what proportion were correct?”

Recall

$$R = \frac{TP}{TP + FN} \quad (4.2)$$

“Of all actual positives, what proportion did we correctly identify?”

F1 score

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} \quad (4.3)$$

“The harmonic mean of precision and recall, which increases as we correctly predict positives while keeping the number of false negatives low.”

This is an especially suitable metric in medical studies where a false negative is a more severe error than a false positive.

4.3.3. Model Application

Here, we explain how we trained the examined models and list the parameters used.

iFORM

We trained the iFORM model described in Section 3.1.2 on all simulated datasets, forcing hierarchy condition and allowing only two-way interactions. For each dataset, we performed an outer 5-fold cross-validation (Figure 4.3). Specifically, we split the samples into 5 folds, fitted the model on 4/5 of the data (training set), and used the remaining 1/5 to apply the fitted model and evaluate its performance (test set). We repeated this procedure over all folds and averaged the resulting test performances to obtain the mean test MSE for that dataset.

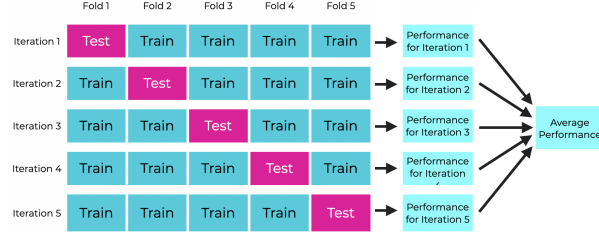


Figure 4.3.: Outer 5-fold cross-validation procedure.

Here, we outline how the scores reported in Tables 4.2 and C.1 were obtained. After cross-validation, we fitted an iFORM model to the full dataset and computed precision, recall, and F1-score for recovery of the true covariates. These values are reported in Table C.1, together with the MSE of this final fit. The average MSE in Table 4.2 is obtained by averaging the cross-validated mean MSEs over all simulated datasets, providing a clearer picture of the MSE we can expect in a broader, generalized study.

BT

For every BT model fitted as BT was described in Section 3.2 we used the following parameters. Regarding creating the solution path of the algorithm, a convergence threshold of 10^{-8} was used and the maximum number of iterations was set to 70. Also, 100 lambdas were picked, forming a decreasing log scale from $\lambda_{\max}$ to $\lambda_{\min}$. Log scale was used as small changes in small lambdas can make a bigger difference than the same changes in larger lambdas. The choice of $\lambda_{\min}$, $\lambda_{\max}$ was made based on the corresponding formulas of section “2.6:Lasso Regression Solution: Example & Useful terminology” with

$$\varepsilon = \begin{cases} 0.01, & n < p, \\ 10^{-4}, & n \geq p. \end{cases}$$

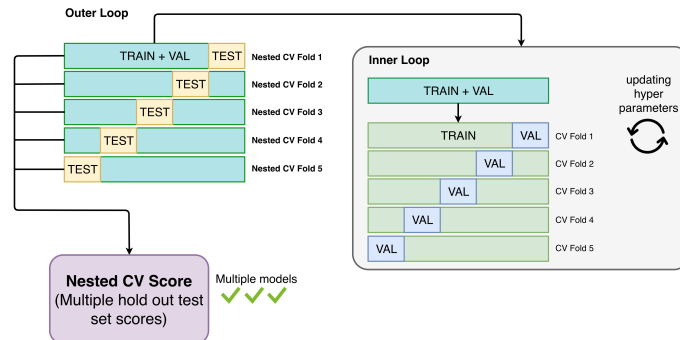


Figure 4.4.: nested 5-outer-folds and 5-inner-folds cross-validation procedure.

To obtain the scores in Tables 4.2 and C.2, we used the following procedure. We performed nested cross-validation with 5 outer and 5 inner folds, as illustrated in Figure 4.4. For each outer training set, we carried out an additional 5-fold split to select the

value of λ that returned the lowest MSE, and used this value for the corresponding outer training. At the end, we computed the average MSE reported in Table 4.2 and refitted the BT model on the full dataset to obtain the F1-related metrics on each dataset as reported in Table Y and their averages as reported in Table 4.2.

PL

The parameters used for each fit of the algorithm were the following. Regarding the design matrices, X contained all the simulated microRNA columns, while Z contained a subset of X . The matrix Z contained all the columns from X that participated in the interaction involved in the model (since we are exploring the Z -known scenario). In other words, the indices of the true interaction were known to the model, but not the exact true interactions. Regarding the model parameters, we used the default value of $\alpha = 0.5$, a convergence threshold of 10^{-4} , a maximum number of interaction terms at 70, and a maximum number of iterations of coordinate sweeps per λ at 100. The λ -path was built using 100 λ 's with $\lambda_{\min} = 0$ and $\lambda_{\max}$ slightly different that the formula given in paragraph 2.6. On the same basis as discussed in chapter 2.6, we want to pick the minimum $\lambda_{\max}$ that keeps all coefficients to 0, and by the soft-thresholding conditions of the pliable Lasso model, it occurs that

$$\lambda_{\max} = \frac{|X_j^T r^{(-j)}|}{n(1 - \alpha)}.$$

Regarding the score tables, MSE is obtained using the same 5×5 nested cross-validation scheme as for BT. The inner folds are used for λ tuning, and the outer folds return an average MSE for each dataset. These cross-validated MSE scores are then averaged across all datasets to compare the goodness of fit of all and reported in Table 4.2. The F1-related scores are computed separately from a single fit on the full dataset and reported in Table C.3.

4.3.4. Analyzing outcomes

Table 4.2 shows a trade-off between variable-selection quality and prediction error. iFORM achieves by far the highest precision for both main and interaction effects (0.887 and 0.778), with moderate recalls (0.663 and 0.400). Thus, when iFORM selects a covariate, it is very likely to be truly active, but a non-negligible portion of the true effects is missed. This conservative behavior gives relatively high F1 scores and a medium average outer CV MSE of 3.63.

Table 4.2.: Average covariate recovery metrics for main and interaction Effects across all datasets and average outer cross-validation MSE.

Method	Main P	Main R	Main F1	Inter P	Inter R	Inter F1	MSE
iFORM	0.887	0.663	0.752	0.778	0.400	0.580	3.630
BT	0.355	0.856	0.471	0.020	0.093	0.102	3.104
PL	0.392	0.867	0.485	0.312	0.820	0.408	6.543

Both BT and PL are less conservative on the main effects. They have similar, high main-effect recall (0.856 for BT and 0.867 for PL), but lower precision (0.355 and 0.392),

meaning that they tend to include most true main effects together with several noise variables. PL strikes a slightly better balance and therefore attains a larger main-effect F1 score (0.485). For interactions, PL clearly outperforms BT: it attains interaction precision 0.312 and recall 0.820, with an interaction F1 of 0.408, so it recovers a large fraction of the true interactions while keeping false interactions relatively under control. In contrast, BT performs very poorly on interactions on average (precision 0.020, recall 0.093, F1 0.102), essentially failing to identify the interaction structure. Despite this, BT still attains the lowest average outer CV MSE (3.10), slightly better than iFORM and substantially better than PL (6.54). This indicates that in the present design, the response is driven mainly by main effects: getting those roughly right is enough for reasonably accurate prediction, even if the interactions are almost completely missed. Which is true since we simulated main effects to contribute more towards the response with their coefficients (see appendix for simulation process). PL, on the other hand, provides a decent overall covariate recovery, is not ideal for prediction, likely due to its large coefficient shrinkage, resulting in the largest MSE.

Conclusion

The simulations differ from the case-study setting in two important ways, consequently results should be interpreted primarily as directional evidence. First, the simulations were run for a continuous response with a linear model, whereas the case-study involves a binary outcome, and we fit logistic models. Second, in the simulations, we have definite knowledge of the true active main and interaction effects, so we can evaluate variable selection directly, while in the case-study, we do not know the definite ground truth. We start with features that the company suggests are likely relevant, but we do not know if or how many are involved in interactions, and we use a stability analysis to decide which effects to keep. Although performance on the linear simulations does not translate one-to-one to the logistic case, both models are Generalized Linear Models (GLMs) fitted with the same penalty term, so we expect broadly similar relative behavior between the methods in terms of sparsity and covariate recovery, even if the goodness-of-fit metrics differ.

Taking the above into account, the simulations still provide a useful guide for method selection, and we draw some conclusions. iFORM consistently shows high precision and good F1 scores for both main and interaction effects, while PL offers decent overall recovery of the true interaction structure, although with a higher prediction error. By contrast, BT performs poorly in recovering true effects, especially interactions, even when its predictive error is competitive. Since a key goal of the case-study is to obtain interpretable models and hints of causal relationships rather than purely optimal prediction, we therefore focus on applying iFORM and PL to the case-study dataset and do not pursue BT further.

4.4. Best Method

In this section, we compare the models that performed well in the simulation study, along with the simple L1-penalized logistic regression model introduced in Section 3.2.4. Based

on this comparison, we determine which model best fits the case-study dataset and is most likely to be informative about the potential true covariates.

4.4.1. Stability Analysis

We perform the stability analysis method known as CPSS (Complementary Pairs Stability Selection) to help us safely select the best method. We will later use CPSS to build the matrix of modifiers for Pliable Lasso and compensate for not knowing the true interaction effects on the case-study dataset. Additionally, CPSS allows us to see which model consistently pinpoints the same covariates and therefore can be more reliable in concluding the potential true covariates.

Process

In this paragraph, we write about how CPSS works and how we applied it. Complementary Pairs Stability Selection (CPSS) is used to assess how stable a method is in selecting specific covariates. It does this by reporting, for each covariate, the frequency with which it is selected across many subsamples, and we then compare that frequency to a predefined acceptance threshold. Concretely, CPSS splits the original dataset into $\mathcal{B}$ complementary pairs of half-samples and fits the model separately in each half-sample (in $2\mathcal{B}$ half-samples in total). A covariate $\mathbf{X}_i$ is assigned a selection frequency of 0.72 if it was selected in 72% of those $\mathcal{B}$ rounds on both half-samples of the round. If this frequency exceeds the chosen acceptance threshold (e.g., 0.7), we regard $\mathbf{X}_i$ as a stable selection. In our analysis, we chose the default $\mathcal{B} = 50$ and 0.9 as a threshold.

Theoretical Selection Boundary of Low Selection Probability covariates

According to Theorem 1 of CPSS theory, given q is the average number of selected variables per half-sample, we obtain the following upper bound on the expected number of variables that have base-procedure-selection (e.g. Pliable Lasso) probability at most $\frac{q}{p}$ but are nevertheless selected by CPSS. Assuming the expected number of variables that have base-selection probability at most $\frac{q}{p}$ is F and the CPSS threshold is τ , then:

$$\mathbb{E}[F] \leq \frac{\frac{q}{p}q}{(2\tau - 1)} = \frac{q^2}{p(2\tau - 1)}. \quad (4.4)$$

Results

From Figure 4.5, we see that iFORM yields no stable interactions and only a single stable main effect, whereas PL and L1 both retain many stable interactions. PL appears to have more stable main effects but fewer interactions than L1. Interestingly, PL also reveals a pattern in which microRNAs with indices 3, 7, and 12 are involved in all interactions, suggesting a potential modifier role.

4.4.2. Case-study Modeling Choice

Followed Scheme

We will further examine this potential modifier structure and finalize our modeling approach for the case-study data. To do that, we will compare the fits of 6 models.

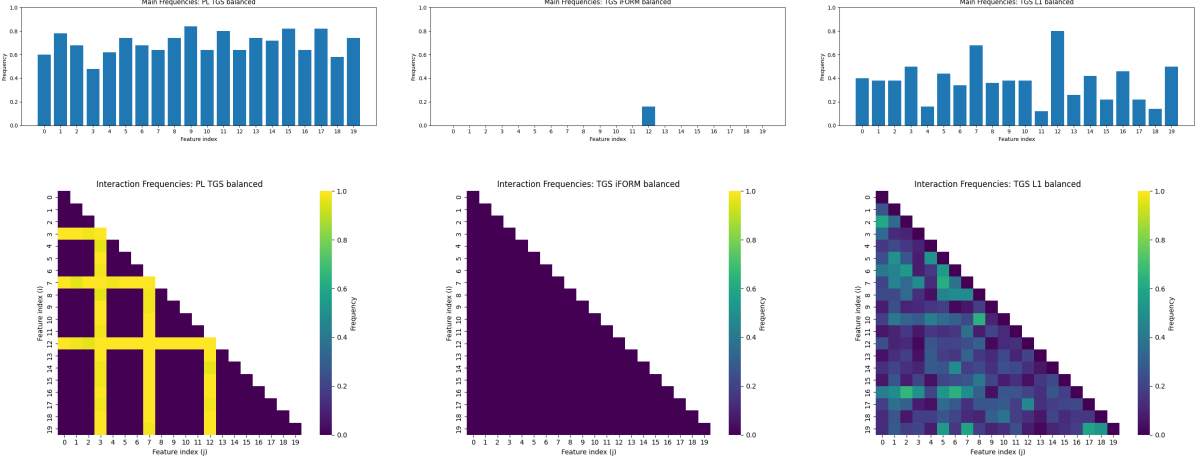


Figure 4.5.: Selection frequency plots of stability analysis. The first column of plots is the frequency of main and interaction covariates respectively, using Pliable Lasso method. The second column consists of the same plots for iFORM and the third is for L1.

We will fit an L1-penalized logistic regression model using only the stable covariates (“L1_stab”) as well as one using the full design matrix (“L1_full”) with all possible covariates. For iFORM, we used the full design matrix, as no stable covariates were identified. For the pliable Lasso, we consider three versions: a full model (“mpl_full”), a model including the stable main covariates and microRNAs with indices 3, 7, and 12 as modifiers (“mpl_fixed”), and a model with stable main effects and modifiers coming from the indices involved in stable interactions (“mpl_stab”).

Initially, we fit each model once on the full dataset to obtain a first impression of their performance, using 5-fold stratified cross-validation to tune λ (except for iFORM, which does not have a λ parameter). We then perform a 5-5 nested cross-validation scheme, identical to that used in the simulations, but with deviance as the evaluation metric in the inner folds instead of MSE. From the outer folds we report deviance, PR AUC and ROC AUC.

Deviance (D) is a goodness-of-fit metric:

$$D = -2 \sum_{i \in \text{outer test}} [\mathcal{L}(\beta)]_i. \quad (4.5)$$

When the log-likelihood $\mathcal{L}(\beta)$ decreases, the model fits the data worse and D increases. We know β depends on λ so we can rewrite equation 4.5 as :

$$D(\lambda) = -2 \sum_{i \in \text{outer test}} [\mathcal{L}(\beta(\lambda))]_i. \quad (4.6)$$

In the inner fold of our CV we choose the λ that gives the lower D.

The chosen model parameters are as follows. For pliable Lasso and iFORM, we kept them identical to the simulations. For L1 we used 100 λ ’s to build the solution path.

Comparison with 5-fold stratified cross-validation

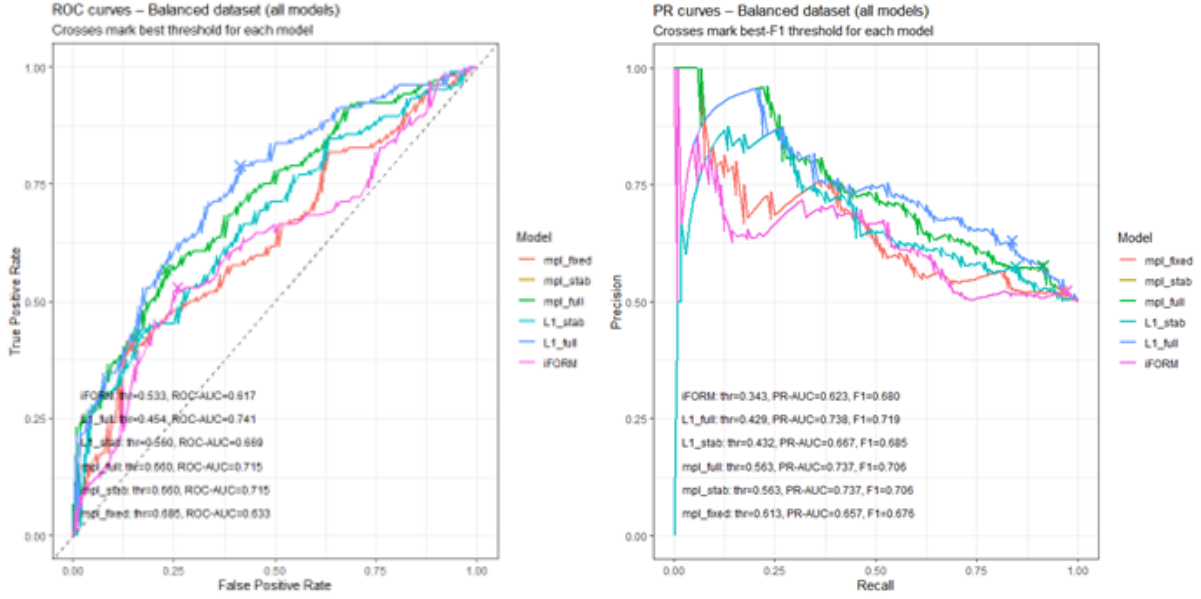


Figure 4.6.: ROC and PR AUC plots.

To obtain the results, we used 5-fold stratified cross-validation. In each fold, we trained all models on the training split and evaluated them on the corresponding test split. For every value of λ along the solution path, we computed PR AUC, ROC AUC, and deviance. We then averaged each metric across the five folds for every λ .

To generate Plots 4.6, we used these averaged metrics to select the λ that achieved the best mean performance (for the corresponding metric). We then refit the model on the full dataset using the selected λ and produced the final curves. For the Receiver Operating Characteristic curve, the axes are True Positive Rate versus False Positive Rate, and the ROC AUC is the area under this curve. For the Precision–Recall curve, the axes are precision versus recall, and the PR AUC is the corresponding area under the curve. In both cases, the curve is traced out by varying the decision threshold from 0 to 1, with each point representing the metric values at a specific threshold. The threshold that returns the best trade-off is marked with a cross, and its value is reported in the bottom-left corner of each plot alongside the AUC score.

To generate Plot 4.7, we similarly averaged deviances across folds (using the test sets) for each λ , and plotted the resulting mean deviance curve.

Overall, we observe modest to decent performance across all models. L1_full achieves the highest area under the curve for both the precision–recall (PR) curve and the ROC curve. Close behind are mpl_stab and mpl_full (again for both curves), with only a tiny difference between them. The mpl_stab curve is not shown because it is identical to mpl_full. This is expected: we used a stability threshold of 0.6 for all models, and for mpl, that threshold retains all features. mpl_fixed performs next best, while iFORM ranks last on both metrics.

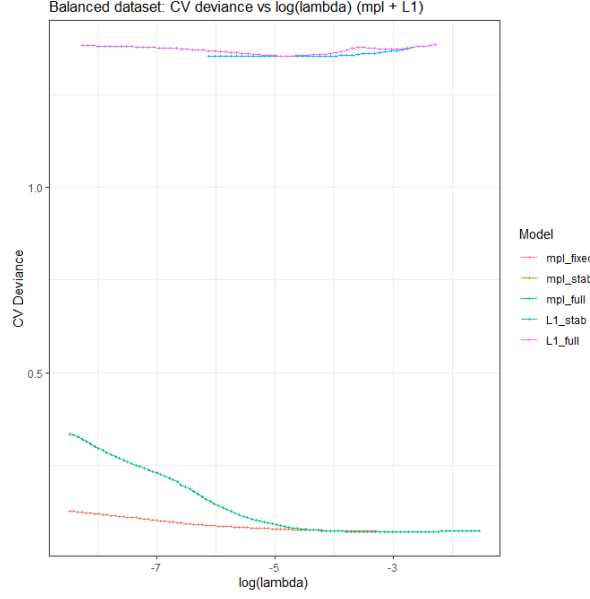


Figure 4.7.: Average deviance across cross-validation folds versus $\log(\lambda)$.

From Plot 4.7 we conclude the following. iFORM is missing because it is not dependent on λ . Mpl models have lower deviance than L1 models. Keep in mind that we fit the models with different lambdas in AUC Plots 4.6 and deviance Plot 4.7 but the fact that we do not see the same trend leads us to the following. L1 models seem to have a better ability to be trained to rank higher actual positives compared to actual negatives but likely do not have calibrated probabilities and perhaps assign extreme probabilities. This would explain why mpl models can have lower deviance, as deviance increases with "confident mistakes". For example, given a single deviance contribution is D_i :

$$D_i = -2 \left(Y_i \log(p_i) + (1 - Y_i) \log(1 - p_i) \right)$$

Supposing the true label is negative ($Y_i = 0$), and the predicted probability is $p_i = 0.99$:

$$D_i = -2 \log(1 - 0.99) = -2 \log(0.01)$$

$$\log(0.01) \approx -4.60517 \Rightarrow D_i \approx -2(-4.60517) = 9.21034$$

Now, assuming again a negative true label $Y_{i+1} = 0$, and a smaller mistake $p_{i+1} = 0.60$:

$$D_{i+1} = -2 \log(1 - 0.60) = -2 \log(0.40)$$

$$\log(0.40) \approx -0.91629 \Rightarrow D_{i+1} \approx -2(-0.91629) = 1.83258$$

This example illustrates how high-confidence errors can substantially inflate the individual deviance, and consequently the mean deviance as well. It also highlights the important distinction between deviance and AUC-based metrics: even if we had many instances like case $i + 1$, as long as the model assigned higher predicted probabilities to positive cases than to negative ones, we could still select a decision threshold that gives a strong AUC score.

Comparison of the Nested CV Results

In this section, we will dive deeper into the investigation of the most suitable modeling scheme, using nested cross-validation to test models also on unseen data.

To generate the boxplots, we used cross-validation procedures designed to tune λ and obtain a more generalizable estimate of model performance. Specifically, for all models except iFORM, we applied a 5-5 stratified nested cross-validation scheme, using the five inner folds to fine-tune λ and the five outer folds to collect goodness-of-fit measurements. For iFORM, a standard 5-fold stratified cross-validation produced the same goodness-of-fit metrics. Consequently, each boxplot is based on those five measurements (of ROC AUC, PR AUC, and deviance). While more than five measurements would be preferable, the sample size is limiting, we have 102 healthy and 102 MCI samples. That corresponds to roughly $\frac{102}{5} \approx 20$ samples per class in each split, and reducing this further would make our conclusions unstable.

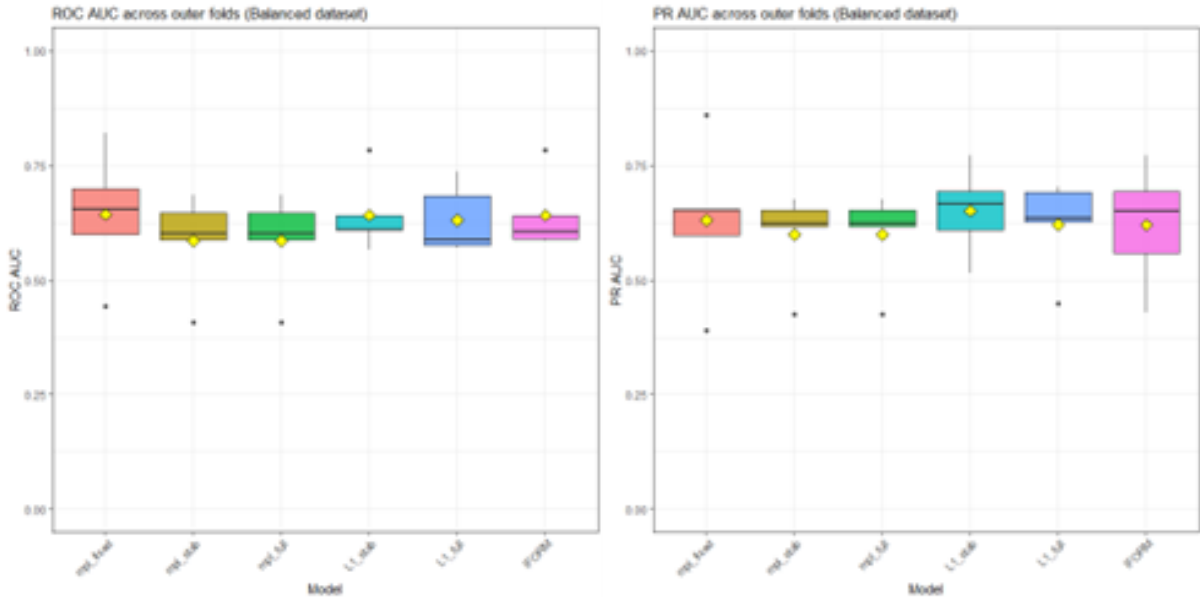


Figure 4.8.: PR AUC and ROC AUC boxplots. The yellow diamond represents the score’s mean.

Here we summarize the boxplot results. The strongest performance is observed for `mpl_fixed` and `L1_stab`. `mpl_stab` and `mpl_full` are identical because all covariates were stable. `mpl_fixed` appears best on the ROC AUC plot, whereas `L1_stab` performs best on PR AUC. Intuitively, ROC AUC reflects how often, across all thresholds, a randomly chosen true MCI sample receives a higher predicted score than a randomly chosen healthy sample, while PR AUC emphasizes how separable the predicted MCI’s are across thresholds. This suggests that `mpl_fixed` achieves better overall separation, but its positive predictions can be noisier at certain thresholds. Regarding deviance, iFORM gives the lowest, but with a low ROC AUC score. Among those with the best AUC scores, `L1_stab` gives a lower deviance.

We explain the calibration plot shown in Figure 4.10. To construct it, we pool the predicted probabilities (p_i) from all outer test folds together with their true labels (Y_i).

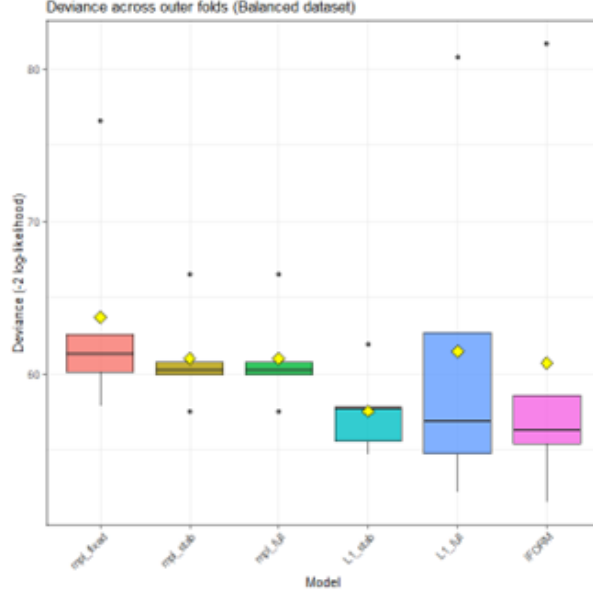


Figure 4.9.: Deviance boxplots. The yellow diamond represents the score's mean.

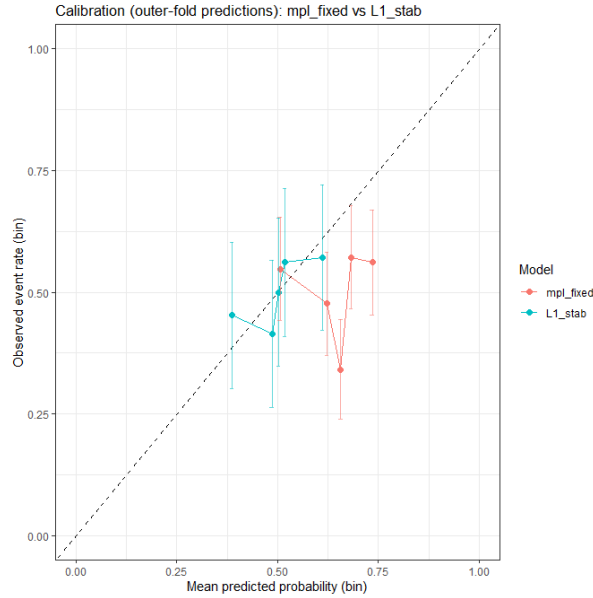


Figure 4.10.: Calibration plot with quantile bins.

This yields $5 \times 40 = 200$ out-of-fold predictions in total. For each model, we then sort the (p_i) values and split them into five equal-sized bins (quantiles). Each point on the plot corresponds to one quantile bin: the x -coordinate is the mean predicted probability within the bin, and the y -coordinate is the observed event rate, $\hat{p} = \frac{1}{n} \sum_{i=1}^n Y_i$, i.e., the mean of the observed outcomes within the same bin. The vertical interval represents an approximate 95% confidence interval for the bin-wise event rate, computed using a normal approximation. The lower and upper limits, respectively, are

$$\max \left(0, \hat{p} - 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \right) \quad \text{and} \quad \min \left(1, \hat{p} + 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \right), \quad (4.7)$$

where n is the number of observations in the bin.

From the calibration plot in Figure 4.10, we draw the following conclusions. `L1_stab` appears to produce better-calibrated probabilities, in the sense that the mean predicted probability generally increases as the observed event rate increases. However, this apparent calibration may be partly driven by probability shrinkage toward 0.5: the model rarely outputs extreme probabilities (close to 0 or 1), which reduces the influence of outliers on the bin averages. In addition, `L1_stab` shows wider confidence intervals, which is likely due to ties in the predicted probabilities for the `mpl` model (another reason calibration can look deceptively good). When many observations share the same $\hat{p}$, changes in $\hat{p}$ have less impact on the confidence interval limits given in Equations 4.7. For example:

$$\begin{aligned}\hat{p} = 0.50 : \quad & 1.96\sqrt{\frac{0.25}{40}} \approx 0.155, \\ \hat{p} = 0.60 : \quad & 1.96\sqrt{\frac{0.24}{40}} \approx 0.152, \\ \hat{p} = 0.35 : \quad & 1.96\sqrt{\frac{0.2275}{40}} \approx 0.148.\end{aligned}$$

In contrast, `mpl_fixed` tends to assign higher probabilities, with mean predictions roughly in the range 0.5 to 0.75, while the corresponding observed event rates are about 0.35 to 0.57. This is not necessarily problematic: in biomedical settings, overestimation is often preferable to underestimation, provided the model still ranks cases well (e.g. true negatives may receive relatively high probabilities, but typically lower than those assigned to true positives).

4.5. Deliverables to the Company

4.5.1. Interpretation of the balanced dataset analysis

Two preprocessing choices are important for interpreting our results. We will discuss how each affects variable selection and prediction probabilities. First, we standardized all covariates before fitting the Lasso–logistic models. This improves numerical stability and ensures that the Lasso penalty is applied on a comparable scale across covariates. As a result, standardization primarily affects variable selection by making it fairer across features, while it does not affect predictions (as long as new data are preprocessed the same way). Second, we trained the models on a balanced dataset (equal numbers of 0/1 outcomes), whereas the target population prevalence is approximately 20/80. Balancing can improve learning and feature selection, but it also gives probability estimates that do not reflect true population frequencies. So predicted probabilities should not be interpreted as absolute risk without correction. For deployment, probability outputs should therefore either be recalibrated (e.g., via Platt scaling, treating the current outputs as scores and mapping them to calibrated probabilities consistent with population prevalence) or the model should be refit using class weights or on the naturally imbalanced dataset once more samples are collected. For variable selection, there is no theoretical guarantee that selections will match those from an imbalanced fit, but in practice there is often substantial overlap, and applying stability selection further mitigates this issue [23].

From our previous analysis, we draw several practical takeaways for the company. L1_stab appears to be the strongest model for prediction, and variable selection even though its selected features are unstable. From the Ineq.4.4, the bound for low selection probability variables included in the model are:

$$\mathbb{E}[F_{L1_stab}] \leq \frac{27^2}{\binom{20}{2}(2 \cdot 0.6 - 1)} \approx 17.36$$

while for mpl it was

$$\mathbb{E}[F_{mpl_fixed}] \leq \frac{130^2}{\binom{20}{2}(2 \cdot 0.9 - 1)} \approx 100.6.$$

That means L1_stab guarantees to select at most 17 low selection probability variables out of the 210 in total, while mpl_fixed guarantees at most 100. The mpl_fixed model did not give better stability bounds, as one might intuitively expect, given that it had a more stable selection. That is because of the square in the formula. The average number of selected variables per sample contributes more. One caveat when comparing L1_stab to mpl_fixed is that the software library underlying L1_stab is substantially more optimized than the current mpl implementation, which may partly explain the performance gap.

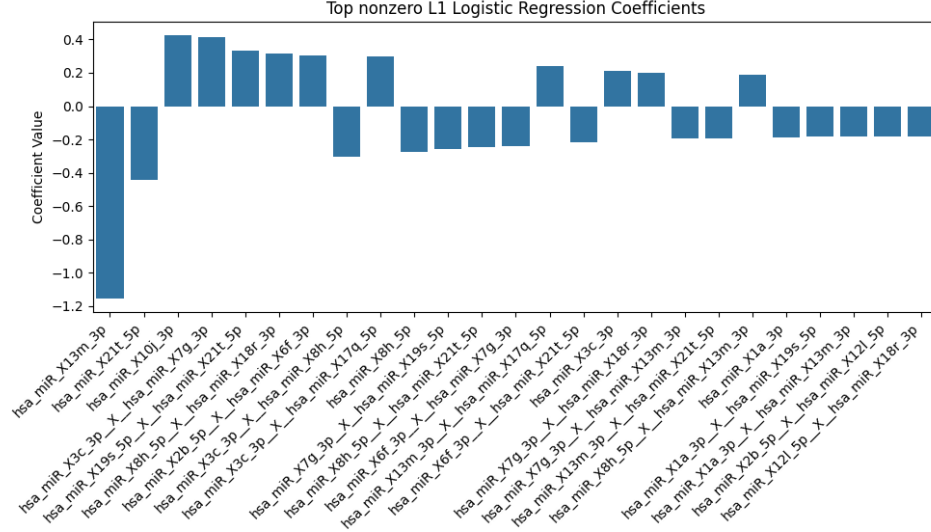
4.5.2. L1_stab's fit on the unbalanced data

We fit the L1_stab model on the unbalanced data. The covariates with non-zero coefficients are shown in Figure 4.11a.

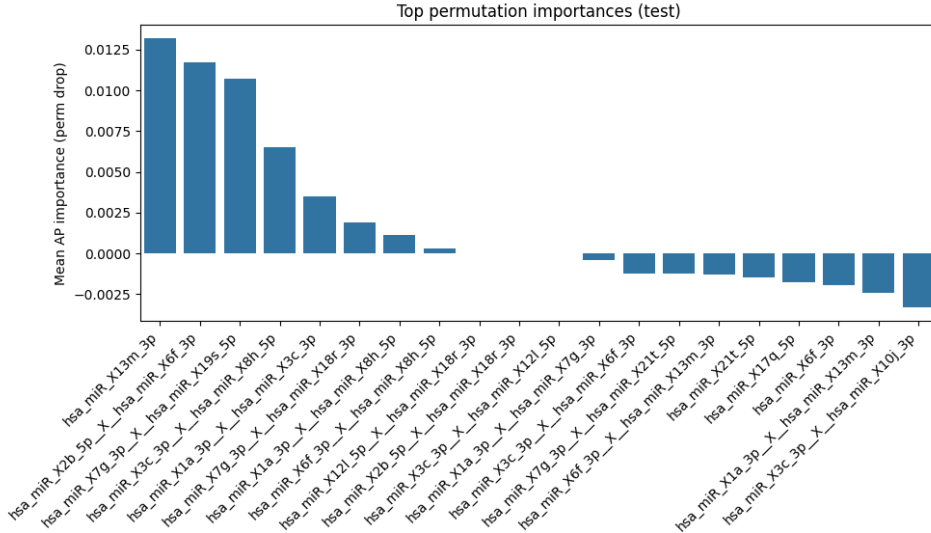
To understand which covariates actually matter for predictive performance, we permuted the values of each covariate and measured the resulting change in PR AUC (Figure 4.11b). Covariates on the left are the most influential, while those on the right are more likely to be noise variables. Notably, no single covariate causes a large drop in PR AUC, suggesting that performance is not driven by one dominant feature or that performance is low overall.

Another important observation on the Figures 4.11a and 4.11b can be made regarding microRNAs hsa_miR_X13m_3p and hsa_miR_X21t_5p. Because the model was fit on standardized covariates, we would typically expect larger coefficient magnitudes to correspond to higher importance, so the top covariates in Figure 4.11a should also appear near the top of Figure 4.11b. This is true for hsa_miR_X13m_3p (first in both plots) but not for hsa_miR_X21t_5p, which indicates the relationship of hsa_miR_X21t_5p with the unexplained variance of Y is likely correlational but not causal.

To analyze the performance, we look at Figure 4.12. Plot 4.12c shows that the model achieves a PR AUC of 0.23, only slightly better than the random baseline of 0.19 (the dashed line, determined by the dataset prevalence). It also shows poor performance, not due to a non-polynomial relationship in the data, as the popular machine learning algorithm Random Forest did not perform satisfactorily either. For a read on the random forest algorithm, we refer the reader to [4]. Plots 4.12a and 4.12b indicate that this is not simply a matter of selecting the right threshold: in (a), the predicted probability distributions for class 0 and class 1 overlap substantially, and in (b) the precision-recall trade-off



(a) Non-zero coefficients' magnitude in descending order of absolute value.



(b) Illustration of how much the model's test-set Average Precision (AP) drops (y-axis) when you randomly permute the covariate values (x-axis).

Figure 4.11.: Covariate importance.

implies we can achieve either high precision or high recall, but not both simultaneously. Finally 4.12d shows that after shuffling the response labels of the samples pr auc score occurring from the correct labels is close to the mean of the distribution, instead of on the far right, indicating that there is little learnable signal.

4.5.3. Takeaway

In this section, we will give our suggestions to the company and the project -related code.

Our analysis revealed little learnable signal in the dataset. In general, microRNA studies face tougher signal-to-noise ratio [24]. We therefore advise the company to reconsider its biomarker choice if possible.

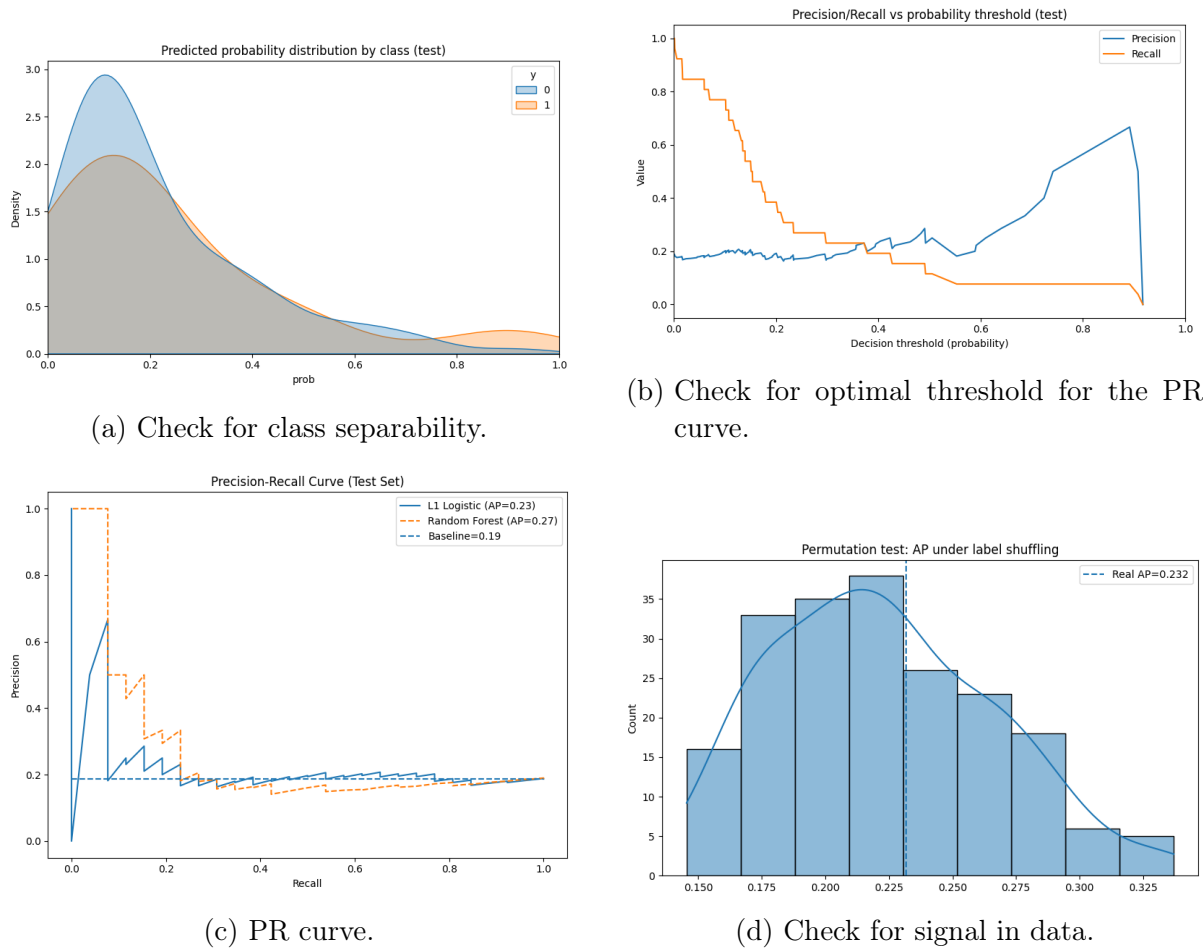


Figure 4.12.: L1_stab's performance analysis. The PR AUC is reported as AP because scikit-learn computes the area under the precision–recall curve using average precision.

If microRNAs are to remain the biomarkers, the pronounced class imbalance means that substantially more samples would be required to increase the number of cross-validation folds without making the conclusions unstable. If acquiring additional data is not feasible, a last-resort option would be to apply one of the rebalancing techniques mentioned in section 4.5.1 to the L1_stab and mpl_fixed models in an attempt to keep performance closer to the balanced score. However, given the weak signal in the data, we would not expect such an approach to generalize well to new samples.

Finally, the consistent involvement of microRNAs 4, 8, and 13 (or hsa_miR_X4d_3p, hsa_miR_X8h_5p and hsa_miR_X13m_3p) across interactions under the mpl model remains an interesting observation. Even more for microRNA hsa_miR_X13m_3p, which was influential also for the L1_stab model fit with the unbalanced data. We recommend collecting additional data to enable more cross-validation folds safely. Since the company has access to the true microRNA identifiers, the biological relevance of these candidates can then be investigated further.

The related code can be accessed through: <https://github.com/enthelgi/Thesis>.

5. Conclusion

In this section, we revisit the purpose of the study, the questions posed, and the answers provided. We will refer to the methods as iFORM, BT, PL, and L1, always meaning their versions for a binomial response.

This thesis was conducted in collaboration with ToxGenSolutions B.V. to help identify patterns in their dataset, as well as its limitations, for explaining a patient’s mild cognitive impairment.

In Chapter 4, we effectively asked four questions and answered them in sequence: (1) which method can recover main effects and, especially, interaction effects when the ground truth is known (simulations), (2) how does the recovery compares to predictive performance, (3) what carries over to the balanced case-study dataset when we require both stable selection and reasonable prediction, and (4) what remains true once we step back from the balanced setting and consider the original class imbalance again.

A summary of the answers is given here. The simulations revealed iFORM had decent performance regarding variable recovery and prediction error, while the best method (PL) showcased clear differences in interaction recovery and prediction performance, with the highest error among the three (PL, BT, iFORM). BT had very low interaction recovery performance so we did not examine it further. In the case-study, where the ground truth regarding explanatory variables is not known, the emphasis shifted towards the stability of variable selection and out-of-sample classification metrics. There, iFORM and PL were compared to the standard logistic regression model with an L1 penalty. iFORM performed poorly, both in selection stability and in classification. PL and L1 achieved similarly moderate classification performance, but differed in stability: PL was highly stable yet selected too many variables on average across the 50 CPSS fits, whereas L1 was less stable (with few variables selected more than 60% of the time) but kept the number of selected variables moderate, making its selections more reliable in practice. We therefore proceeded only with L1 on the imbalanced version of the company’s dataset, which ultimately suggested that the dataset contains limited learnable signal.

Concerning individual microRNAs, indices 3, 7, and 12 stood out as particularly interesting. Under PL, these microRNAs were repeatedly selected as modifiers, suggesting a potential modifier relationship in the data. However, this evidence is not conclusive, since PL achieved only moderate performance and was applied only to the balanced version of the case-study dataset. We also have additional evidence that these microRNAs were among the most stable main effects under L1 stability selection. Moreover, microRNAs 7 and 12 appear among the most influential covariates in Figures 4.11a and 4.11b from the fit on the unbalanced data. Taken together, these signals suggest that these microRNAs remain promising candidates and are worth investigating further.

Another noteworthy conclusion is that PL performed well on the balanced dataset. Its performance was close to L1, despite L1 being implemented in a much more optimized software library. Moreover, microRNAs highlighted by PL as potentially important were also highlighted by L1 in the imbalanced fit (indices 3, 7, 12 for PL versus 7, 12 for L1 on imbalanced), even though the overall signal in the data appears weak. Taken together, this suggests that PL is worth further optimization and/or reapplying once a higher-quality dataset is available.

For the company, future work should focus on expanding and improving the quality of the dataset, and on investigating the biological relationships underlying the microRNAs that showed particularly interesting behavior in our analyses.

6. Final Note

6.1. Code

The related code can be found in the following link: <https://github.com/enthelgi/Thesis>.

6.2. Acknowledgments

To conclude, I would like to sincerely thank my university supervisor, Wessel, for the role he played in helping me mature in the way I communicate mathematical ideas. I also want to express my gratitude to my company supervisors, Keano and Stan, for their positive attitude and encouragement throughout the process.

I am especially grateful to my family, Rena, Michalis, and Yorgos, for their unconditional support and belief in me during my studies. Their encouragement has been invaluable, not only during this thesis but throughout every challenge.

Finally, I would like to thank Kostas, Christina, Dora, Vasilis, Ellie, and Marisini, as well as Emma, for their contributions to this project in various ways. Whether through discussions, feedback, or simply by offering support along the way, their help and presence made the completion of this thesis possible.

Bibliography

- [1] Julia Alles et al. “An estimate of the total number of true human miRNAs”. In: *Nucleic acids research* 47.7 (2019), pp. 3353–3364.
- [2] Theophilus Quachie Asenso, Hai Zhang, and Yong Liang. “Pliable lasso for the multinomial logistic regression”. In: *Communications in Statistics-Theory and Methods* 51.11 (2022), pp. 3596–3611.
- [3] Scott Baskerville and David P Bartel. “Microarray profiling of microRNAs reveals frequent coexpression with neighboring miRNAs and host genes”. In: *Rna* 11.3 (2005), pp. 241–247.
- [4] Gérard Biau. “Analysis of a random forests model”. In: *The Journal of Machine Learning Research* 13.1 (2012), pp. 1063–1095.
- [5] S. Boyd and A. Mutapcic. “Subgradient methods Notes for EE364b Stanford University”. In: (2006).
- [6] S. Boyd and L. Vandenberghe. *Subgradients*. https://see.stanford.edu/materials/lsoctee364b/01-subgradients_notes.pdf. 2008.
- [7] Thomas Brambor, William Roberts Clark, and Matt Golder. “Understanding interaction models: Improving empirical analyses”. In: *Political analysis* 14.1 (2006), pp. 63–82.
- [8] Jiahua Chen and Zehua Chen. “Extended Bayesian information criteria for model selection with large model spaces”. In: *Biometrika* 95.3 (2008), pp. 759–771.
- [9] Dmitry Chistikov et al. “The big-O problem”. In: *Logical Methods in Computer Science* 18 (2022).
- [10] Alhussein Fawzi, Mike Davies, and Pascal Frossard. “Dictionary learning for fast classification based on soft-thresholding”. In: *International Journal of Computer Vision* 114 (2015), pp. 306–321.
- [11] Gordon A Fletcher. *The Keynesian revolution and its critics: Issues of theory and policy for the monetary production economy*. Springer, 1987.
- [12] Jerome Friedman, Trevor Hastie, and Rob Tibshirani. “Regularization paths for generalized linear models via coordinate descent”. In: *Journal of statistical software* 33.1 (2010), p. 1.
- [13] Benyamin Ghojogh et al. “KKT conditions, first-order and second-order optimization, and distributed optimization: tutorial and survey”. In: *arXiv preprint arXiv:2110.01858* (2021).
- [14] Ning Hao and Hao Helen Zhang. “Interaction screening for ultrahigh-dimensional data”. In: *Journal of the American Statistical Association* 109.507 (2014), pp. 1285–1301.

- [15] Kevin L Keys, Hua Zhou, and Kenneth Lange. “Proximal distance algorithms: Theory and practice”. In: *Journal of Machine Learning Research* 20.66 (2019), pp. 1–38.
- [16] Jeonghwan Kim et al. “Development of regression models resolving high-dimensional data and multicollinearity problem for heavy rain damage data”. In: *KSCE Journal of Civil and Environmental Engineering Research* 38.6 (2018), pp. 801–808.
- [17] Mariana Lagos-Quintana et al. “Identification of novel genes coding for small expressed RNAs”. In: *Science* 294.5543 (2001), pp. 853–858.
- [18] Alfonso Landeros, Jason Xu, and Kenneth Lange. “MM optimization: Proximal distance algorithms, path following, and trust regions”. In: *Proceedings of the National Academy of Sciences* 120.27 (2023), e2303168120.
- [19] Rosalind C Lee, Rhonda L Feinbaum, and Victor Ambros. “The *C. elegans* heterochronic gene *lin-4* encodes small RNAs with antisense complementarity to *lin-14*”. In: *cell* 75.5 (1993), pp. 843–854.
- [20] Michael Lim and Trevor Hastie. “Learning interactions via hierarchical group-lasso regularization”. In: *Journal of Computational and Graphical Statistics* 24.3 (2015), pp. 627–654.
- [21] Donald W Marquardt. “Comment: You should standardize the predictor variables in your regression models”. In: *Journal of the American Statistical Association* 75.369 (1980), pp. 87–91.
- [22] Peter R Mercer and Peter R Mercer. “Convex Functions and Taylor’s Theorem”. In: *More Calculus of a Single Variable* (2014), pp. 171–208.
- [23] Khurram Nadeem and Mehdi-Abderrahman Jabri. “Stable variable ranking and selection in regularized logistic regression for severely imbalanced big binary data”. In: *Plos one* 18.1 (2023), e0280258.
- [24] Viswam S Nair et al. “Design and analysis for studying microRNAs in human disease: a primer on-omic technologies”. In: *American journal of epidemiology* 180.2 (2014), pp. 140–152.
- [25] Tomoyuki Obuchi and Yoshiyuki Kabashima. “Cross validation in LASSO and its acceleration”. In: *Journal of Statistical Mechanics: Theory and Experiment* 2016.5 (2016), p. 053304.
- [26] Michael A Poole and Patrick N O’Farrell. “The assumptions of the linear regression model”. In: *Transactions of the Institute of British Geographers* (1971), pp. 145–158.
- [27] Brenda J Reinhart et al. “The 21-nucleotide *let-7* RNA regulates developmental timing in *Caenorhabditis elegans*”. In: *nature* 403.6772 (2000), pp. 901–906.
- [28] Marco R Steenbergen and Bradford S Jones. “Modeling multilevel data structures”. In: *american Journal of political Science* (2002), pp. 218–237.
- [29] Robert Tibshirani and Jerome Friedman. “A pliable lasso”. In: *Journal of Computational and Graphical Statistics* 29.1 (2020), pp. 215–225.
- [30] Wessel N van Wieringen. “Lecture notes on ridge regression”. In: *arXiv preprint arXiv:1509.09169* (2015).

- [31] Barry M Wise and Neal B Gallagher. “An introduction to linear algebra”. In: *Critical reviews in analytical chemistry* 28.1 (1998), pp. 1–20.
- [32] Yi Yang and Hui Zou. “A coordinate majorization descent algorithm for ℓ_1 penalized learning”. In: *Journal of Statistical Computation and Simulation* 84.1 (2014), pp. 84–95.

A. Detailed Calculations

A.1. Back-transformation from Scaling coefficients

For this section, we will use the tilda symbol for scaled covariates and coefficients to clearly differentiate them from unscaled ones.

First, we show how to back-transform for the linear regression case:

$$\tilde{\mathbf{X}}_j = \frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j}, \quad j = 1, \dots, p.$$

Assuming we fit the model using the scaled covariates:

$$\mathbf{Y} = \tilde{\beta}_0 \mathbf{1} + \sum_{j=1}^p \tilde{\beta}_j \tilde{\mathbf{X}}_j + \boldsymbol{\varepsilon}.$$

We substitute the definition of $\tilde{\mathbf{X}}_j$:

$$\mathbf{Y} = \tilde{\beta}_0 \mathbf{1} + \sum_{j=1}^p \tilde{\beta}_j \left(\frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j} \right) + \boldsymbol{\varepsilon}.$$

We distribute and separate $\mathbf{X}_j$ -terms from intercept terms:

$$\mathbf{Y} = \tilde{\beta}_0 \mathbf{1} + \sum_{j=1}^p \frac{\tilde{\beta}_j}{\sigma_j} \mathbf{X}_j - \sum_{j=1}^p \tilde{\beta}_j \frac{\mu_j}{\sigma_j} \mathbf{1} + \boldsymbol{\varepsilon}.$$

We group the $\mathbf{1}$ terms to get the unscaled intercept:

$$\mathbf{Y} = \underbrace{\left(\tilde{\beta}_0 - \sum_{j=1}^p \tilde{\beta}_j \frac{\mu_j}{\sigma_j} \right)}_{\beta_0} \mathbf{1} + \sum_{j=1}^p \underbrace{\left(\frac{\tilde{\beta}_j}{\sigma_j} \right)}_{\beta_j} \mathbf{X}_j + \boldsymbol{\varepsilon}.$$

Hence, for the unscaled model

$$\mathbf{Y} = \beta_0 \mathbf{1} + \sum_{j=1}^p \beta_j \mathbf{X}_j + \boldsymbol{\varepsilon},$$

the back-transformation is

$$\beta_j = \frac{\tilde{\beta}_j}{\sigma_j}, \quad \beta_0 = \tilde{\beta}_0 - \sum_{j=1}^p \tilde{\beta}_j \frac{\mu_j}{\sigma_j}.$$

Now we back-transform for the interaction linear regression case:

We assume the quadratic and interaction terms were formed from the scaled variables Scaled-fit model:

$$\mathbf{Y} = \tilde{\beta}_0 \mathbf{1} + \sum_{j=1}^p \tilde{\beta}_j \tilde{\mathbf{X}}_j + \sum_{j=1}^p \tilde{\beta}_{jj} \tilde{\mathbf{X}}_j^2 + \sum_{1 \leq i < j \leq p} \tilde{\beta}_{ij} (\tilde{\mathbf{X}}_i \tilde{\mathbf{X}}_j) + \varepsilon.$$

We substitute

$$\tilde{\mathbf{X}}_j = \frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j}.$$

Step 1: We expand the linear terms.

$$\sum_{j=1}^p \tilde{\beta}_j \tilde{\mathbf{X}}_j = \sum_{j=1}^p \tilde{\beta}_j \left(\frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j} \right) = \sum_{j=1}^p \frac{\tilde{\beta}_j}{\sigma_j} \mathbf{X}_j - \sum_{j=1}^p \tilde{\beta}_j \frac{\mu_j}{\sigma_j} \mathbf{1}.$$

Step 2: We expand the quadratic terms. For each j ,

$$\tilde{\mathbf{X}}_j^2 = \left(\frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j} \right)^2 = \frac{\mathbf{X}_j^2 - 2\mu_j \mathbf{X}_j + \mu_j^2 \mathbf{1}}{\sigma_j^2}.$$

Therefore,

$$\sum_{j=1}^p \tilde{\beta}_{jj} \tilde{\mathbf{X}}_j^2 = \sum_{j=1}^p \frac{\tilde{\beta}_{jj}}{\sigma_j^2} \mathbf{X}_j^2 - \sum_{j=1}^p \frac{2\mu_j \tilde{\beta}_{jj}}{\sigma_j^2} \mathbf{X}_j + \sum_{j=1}^p \frac{\mu_j^2 \tilde{\beta}_{jj}}{\sigma_j^2} \mathbf{1}.$$

Step 3: We expand the interaction terms. For $i < j$,

$$\tilde{\mathbf{X}}_i \tilde{\mathbf{X}}_j = \left(\frac{\mathbf{X}_i - \mu_i \mathbf{1}}{\sigma_i} \right) \left(\frac{\mathbf{X}_j - \mu_j \mathbf{1}}{\sigma_j} \right) = \frac{(\mathbf{X}_i \mathbf{X}_j) - \mu_j \mathbf{X}_i - \mu_i \mathbf{X}_j + \mu_i \mu_j \mathbf{1}}{\sigma_i \sigma_j}.$$

Hence,

$$\sum_{1 \leq i < j \leq p} \tilde{\beta}_{ij} (\tilde{\mathbf{X}}_i \tilde{\mathbf{X}}_j) = \sum_{i < j} \frac{\tilde{\beta}_{ij}}{\sigma_i \sigma_j} (\mathbf{X}_i \mathbf{X}_j) - \sum_{i < j} \frac{\mu_j \tilde{\beta}_{ij}}{\sigma_i \sigma_j} \mathbf{X}_i - \sum_{i < j} \frac{\mu_i \tilde{\beta}_{ij}}{\sigma_i \sigma_j} \mathbf{X}_j + \sum_{i < j} \frac{\mu_i \mu_j \tilde{\beta}_{ij}}{\sigma_i \sigma_j} \mathbf{1}.$$

Step 4: We collect like terms. Putting Steps 1–3 back into the scaled-fit model yields an expression of the form

$$\mathbf{Y} = \beta_0 \mathbf{1} + \sum_{j=1}^p \beta_j \mathbf{X}_j + \sum_{j=1}^p \beta_{jj} \mathbf{X}_j^2 + \sum_{i < j} \beta_{ij} (\mathbf{X}_i \mathbf{X}_j) + \varepsilon,$$

from which we can get the unscaled coefficients.

Quadratic and interaction coefficients. The coefficients multiplying $\mathbf{X}_j^2$ and $\mathbf{X}_i \mathbf{X}_j$ are:

$$\beta_{jj} = \frac{\tilde{\beta}_{jj}}{\sigma_j^2}, \quad \beta_{ij} = \frac{\tilde{\beta}_{ij}}{\sigma_i \sigma_j} \quad (i < j).$$

Main-effect coefficients. For a fixed j , the coefficient of $\mathbf{X}_j$ receives contributions from: (i) the linear term, (ii) the quadratic term expansion, and (iii) all interactions involving j . Thus,

$$\beta_j = \frac{\tilde{\beta}_j}{\sigma_j} - \frac{2\mu_j\tilde{\beta}_{jj}}{\sigma_j^2} - \sum_{i < j} \frac{\mu_i\tilde{\beta}_{ij}}{\sigma_i\sigma_j} - \sum_{j < k} \frac{\mu_k\tilde{\beta}_{jk}}{\sigma_j\sigma_k}.$$

Intercept. All constant contributions combine into:

$$\beta_0 = \tilde{\beta}_0 - \sum_{j=1}^p \tilde{\beta}_j \frac{\mu_j}{\sigma_j} + \sum_{j=1}^p \tilde{\beta}_{jj} \frac{\mu_j^2}{\sigma_j^2} + \sum_{i < j} \tilde{\beta}_{ij} \frac{\mu_i\mu_j}{\sigma_i\sigma_j}.$$

A.2. Orthonormal design matrix allows coefficients decoupling

The Lasso cost function is:

$$J(\boldsymbol{\beta}) = \frac{1}{N} \underbrace{\|Y - X\boldsymbol{\beta}\|_2^2}_{RSS} + \lambda \underbrace{\|\boldsymbol{\beta}\|_1}_{R(\boldsymbol{\beta})}$$

The coupling can occur from the square on the RSS term. Below, we will explain how this is solved when the design matrix is orthonormal. First we execute the identity in the RSS term:

$$\begin{aligned} \|Y - X\boldsymbol{\beta}\|_2^2 &= (Y - X\boldsymbol{\beta})^\top (Y - X\boldsymbol{\beta}) \\ &= Y^\top Y - Y^\top X\boldsymbol{\beta} - \boldsymbol{\beta}^\top X^\top Y + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta} \\ &= Y^2 - Y^\top X\boldsymbol{\beta} - \boldsymbol{\beta}^\top X^\top Y + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta} \\ &= Y^2 - (X^\top Y)^\top \boldsymbol{\beta} - \boldsymbol{\beta}^\top (X^\top Y) + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta} \\ &= Y^2 - 2\boldsymbol{\beta}^\top (X^\top Y) + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta}. \end{aligned}$$

Now, because of the property of orthonormality ($X^\top X = I$) the last equation becomes:

$$\begin{aligned} &Y^2 - 2\boldsymbol{\beta}^\top (X^\top Y) + \boldsymbol{\beta}^\top I\boldsymbol{\beta} \\ &= Y^2 - 2\boldsymbol{\beta}^\top (X^\top Y) + \boldsymbol{\beta}^2 \end{aligned}$$

The above modification decouples β_j 's because it removes double indexes from the calculations as can be seen in the following equations:

$$\boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta} = \sum_{i=1}^p \sum_{j=1}^p \beta_i \beta_j (X^\top X)_{ij},$$

while

$$\boldsymbol{\beta}^2 = \sum_{i=1}^p \beta_i^2.$$

A.3. Soft-thresholding Lasso solution with decoupled coefficients

We will now follow through the steps that give us a closed form solution for the Lasso estimator provided the matrix is orthonormal.

$$\begin{aligned}\beta_j &= \arg \min_{\beta_j} (J(\beta_j)) \\ \iff \{\beta_j : \frac{\partial J}{\partial \beta_j} &= 0, \beta_j \in \mathbb{R}\} \\ \iff \{\beta_j : \frac{\partial(\frac{1}{N}\|Y - X_j\beta_j\|_2^2 + \lambda|\beta_j|)}{\partial \beta_j} &= 0, \beta_j \in \mathbb{R}\}\end{aligned}$$

Using the calculations from A.1.:

$$\begin{aligned}\frac{\partial J}{\partial \beta_j} &= \frac{\partial(\frac{1}{N}(Y^2 - 2\beta_j(X_j^\top Y) + \beta_j^2) + \lambda|\beta_j|)}{\partial \beta_j} \\ &\stackrel{N=2n}{=} \frac{1}{2n}(-2(X_j^\top Y) + 2\beta_j) + \lambda \frac{\partial(|\beta_j|)}{\partial \beta_j} \\ &= \frac{1}{n}(-(X_j^\top Y) + \beta_j) + \lambda \frac{\partial(|\beta_j|)}{\partial \beta_j}.\end{aligned}$$

Now the term $\frac{\partial(|\beta_j|)}{\partial \beta_j}$ is not differentiable at 0. When a convex function is not differentiable at a point, we can still define a generalized version of a "slope". Instead of a single tangent line, we can consider all lines that touch the curve at that point and lie below the function's graph. The corresponding slopes of all those lines constitute the set of subgradients. The subgradient of a function f is denoted as ∂f . For the formal definition of subgradient and further mathematical support of the notion of subgradient see [5].

So the subgradient of $|\beta_j|$ is:

$$\partial(|\beta_j|) = \begin{cases} \{1\} & \text{if } \beta_j > 0 \\ \{-1\} & \text{if } \beta_j < 0 \\ [-1, 1] & \text{if } \beta_j = 0 \end{cases}$$

As a result,

$$\frac{\partial J}{\partial \beta_j} = 0 \iff \beta_j = X_j^\top Y - \lambda n \partial(|\beta_j|)$$

1

$$\iff \beta_j = \begin{cases} X_j^\top Y - \lambda n & \text{if } \frac{X_j^\top Y}{n} > \lambda \\ X_j^\top Y + \lambda n & \text{if } \frac{X_j^\top Y}{n} < -\lambda \\ 0 & \text{if } \left| \frac{X_j^\top Y}{n} \right| \leq \lambda. \end{cases}$$

By the definition of the soft thresholding function $\beta_j = S(\frac{X_j^\top Y}{n}, \lambda)$.

¹2 was eliminated by equality with 0 and absorbed by the constant λ .

A.4. Derivation of likelihood function

We will now perform calculations for deriving the likelihood function:

$$\begin{aligned}
\mathcal{L}(\beta) &= \log\left(\prod_{i=1}^n f(\mathbf{X}_{i,*}, \beta)\right) \\
\iff \mathcal{L}(\beta) &= \log\left(\prod_{i=1}^n \underbrace{p_i^{Y_i} (1 - p_i)^{(1-Y_i)}}_{\text{pdf of } \mathcal{B}(p_i, 1)}\right) \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n \log(p_i^{Y_i} (1 - p_i)^{(1-Y_i)}) \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n [Y_i \log(p_i) + (1 - Y_i) \log(1 - p_i)] \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n \left[Y_i \log\left(\frac{e^{t_i}}{1 + e^{t_i}}\right) + (1 - Y_i) \log\left(1 - \frac{e^{t_i}}{1 + e^{t_i}}\right) \right] \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n \left[Y_i \log(e^{t_i}) - Y_i \log(1 + e^{t_i}) + \log\left(\frac{1}{1 + e^{t_i}}\right) - Y_i \log\left(\frac{1}{1 + e^{t_i}}\right) \right] \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n \left[Y_i \log(e^{t_i}) - \cancel{Y_i \log(1 + e^{t_i})} + \log 1^0 - \log(1 + e^{t_i}) - \cancel{Y_i \log 1^0} + \cancel{Y_i \log(1 + e^{t_i})} \right] \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n [Y_i \log(e^{t_i}) - \log(1 + e^{t_i})] \\
\iff \mathcal{L}(\beta) &= \sum_{i=1}^n [Y_i t_i \log(e) - \log(1 + e^{t_i})] .
\end{aligned}$$

Substituting t_i we get the final form:

$$\mathcal{L}(\beta) = \sum_{i=1}^n [Y_i \mathbf{X}_{i,*} \beta - \log(1 + e^{\mathbf{X}_{i,*} \beta})].$$

A.5. Completion of squares

We will now work on the equation of the likelihood function so that we bring it in a convenient form for the coordinate descent update for logistic regression:

$$\begin{aligned}
\mathcal{L}(w_j) &\approx \mathcal{L}(w_j^{old}) + d_j(w_j - w_j^{old}) + \frac{1}{2} H_{jj}(w_j - w_j^{old})^2 \\
\iff \mathcal{L}(w_j) &\approx \frac{1}{2} H_{jj}(w_j - w_j^{old})^2 + d_j(w_j - w_j^{old}) + \underbrace{c}_{\text{constant}} ,
\end{aligned}$$

where constant does not affect the minimization process

$$\begin{aligned}
&\Longleftrightarrow \mathcal{L}(w_j) \approx \frac{1}{2}H_{jj} \left[(w_j - w_j^{old})^2 + \frac{2d_j}{H_{jj}}(w_j - w_j^{old}) \right] \\
&\Longleftrightarrow \mathcal{L}(w_j) \approx \frac{1}{2}H_{jj} \left[\left(w_j - w_j^{old} + \frac{d_j}{H_{jj}} \right)^2 - \left(\frac{d_j}{H_{jj}} \right)^2 \right] \\
&\Longleftrightarrow \mathcal{L}(w_j) \approx \frac{1}{2}H_{jj} \left(w_j - w_j^{old} + \frac{d_j}{H_{jj}} \right)^2 - \underbrace{\frac{1}{2}H_{jj} \left(\frac{d_j}{H_{jj}} \right)^2}_{\text{constant with respect to } w_j}
\end{aligned}$$

We know that the NR update rule is

$$w_j = w_{old} - \frac{d_j}{H_{jj}}$$

and this w_j corresponds to the update for unregularized logistic regression so we can call it w_j^{unreg} . Then, the final form of $\mathcal{L}(\cdot)$ we will use for the minimization process becomes:

$$\mathcal{L}(w_j) \approx \frac{1}{2}H_{jj}(w_j - w_j^{unreg})^2.$$

A.6. FS & iFOR complexity

We will show the computational complexity of iFOR is

$$\mathcal{O}(n^2p + n^4).$$

This will be done in two steps. The first step is to show the complexity of FS per step is $\mathcal{O}(np)$ where n is the number of samples and p is the number of available predictors. The second step is to show the desired given our result from the first step.

At the first step of the Forward Selection (FS) algorithm, the predictor most correlated with the response is selected. For that purpose, we calculate Pearson's correlation coefficient

$$\rho_j = \frac{\sum_{i=1}^n (Y_i - \bar{Y})(X_{ij} - \bar{X}_j)}{\sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2} \sqrt{\sum_{i=1}^n (X_{ij} - \bar{X}_j)^2}}$$

for each predictor $\mathbf{X}_j$ for $j \in \{1, \dots, p\}$. This calculation is $\mathcal{O}(n)$, and since we repeat this for each of the p predictors, the total complexity of the first step of FS is $\mathcal{O}(np)$. For the next step, we want to choose the most correlated predictor with the remaining unexplained variance (instead of the response). So, a single-predictor regression model is fitted with the chosen predictor $\mathbf{X}_c$, and we keep the residual $\mathbf{r}_c$. We use that residual to calculate the correlation coefficients of the remaining predictors with that residual. This process repeats until the stopping criteria are met. Since each iteration requires computing at most p correlations and is repeated at most n times, the total complexity per step remains $\mathcal{O}(np)$.

Now, we are ready to show the final result. The number of possible predictors in iFOR is $p + \frac{n(n+1)}{2}$ because we have at most p main effects, and since this algorithm is directed

for HDD cases where $n \ll p$, the interactions follow the hierarchy condition and include the square interactions, in total we have $\frac{n(n+1)}{2}$ interactions. Therefore, the complexity of iFOR per step is $\mathcal{O}\left(n \cdot \left(p + \frac{n(n+1)}{2}\right)\right)$. The number of steps is at most n . As a result, in total, we have a complexity of

$$n \cdot \mathcal{O}\left(n \cdot \left(p + \frac{n(n+1)}{2}\right)\right) = \mathcal{O}(n^2p + n^4).$$

A.7. Shrinkage area of θ_j for Pliable Lasso (Nested Soft-Thresholding)

The derivative of the cost function $J(\beta, \Theta)$ with respect to θ_j is :

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} W_j^\top \mathbf{r}_{(-j)} + (1 - \alpha)\lambda \left(\frac{\theta_j}{\|(\beta_j, \theta_j)\|_2} + \frac{\theta_j}{\|\theta_j\|_2} \right) + \alpha\lambda \text{sign}(\theta_{jk}) \mathbf{e}_k,$$

where $(e_k)_i = 1$ if $i = k$ and 0 otherwise.

If $\beta_j = 0$ the cost function becomes:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} W_j^\top \mathbf{r}_{(-j)} + 2(1 - \alpha)\lambda \text{sign}(\theta_j) + \alpha\lambda \text{sign}(\theta_{jk}).$$

To have $\theta_j = \mathbf{0}$ we want $\mathbf{0} \in \frac{\partial J}{\partial \theta_j}$.

If we check

$$\mathbf{0} \in \frac{1}{n} W_j^\top \mathbf{r}_{(-j)} + \alpha\lambda \text{sign}(\theta_{jk}) \mathbf{e}_k$$

we find the solution is the shrinkage area of the following soft-thresholding operator

$$S\left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, \alpha\lambda\right).$$

If

$$\mathbf{0} \in \frac{1}{n} W_j^\top \mathbf{r}_{(-j)} + \alpha\lambda \text{sign}(\theta_{jk}) \mathbf{e}_k \quad \forall k$$

then $\theta_j = \mathbf{0}$ is already satisfied.

Otherwise, we need to check when

$$\mathbf{0} \in S\left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, \alpha\lambda\right) + 2(1 - \alpha)\lambda \text{sign}(\theta_j),$$

which leads us to the nested soft-thresholding shrinkage area:

$$\left\| S\left(\frac{W_j^\top \mathbf{r}_{(-j)}}{n}, \alpha\lambda\right) \right\|_2 \leq 2(1 - \alpha)\lambda.$$

Notice that it is a vector norm now, as it occurs from the subgradient of the vector θ_j . The Euclidean norm is suitable in that case (e.g., instead of $\|\cdot\|_1$) as we want to shrink components uniformly.

A.8. Deriving the solution system 3.20 from Equation 3.19.

The derivative of Equation 3.19 with respect to β_j is:

$$\frac{1}{t}(\hat{\beta}_j - \tilde{\beta}_j) + (1 - \alpha)\lambda \frac{\hat{\beta}_j}{\|\hat{\gamma}_j\|_2}.$$

If we set the above equation to 0 we get:

$$0 = \frac{1}{t}(\hat{\beta}_j - \tilde{\beta}_j) + (1 - \alpha)\lambda \frac{\hat{\beta}_j}{\|\hat{\gamma}_j\|_2}.$$

Rearranging and solving for $\tilde{\beta}_j$ we get:

$$\left(1 + t(1 - \alpha)\lambda \frac{1}{\|\hat{\gamma}_j\|_2}\right) \hat{\beta}_j = \tilde{\beta}_j.$$

which is the first equation we wanted to obtain in the system of Equations 3.20. The subderivative of 3.19 with respect to θ_j is:

$$\frac{1}{t}(\hat{\theta}_j - \tilde{\theta}_j) + (1 - \alpha)\lambda \frac{\hat{\theta}_j}{\|\hat{\gamma}_j\|_2} + (1 - \alpha)\lambda \frac{\hat{\theta}_j}{\|\hat{\theta}_j\|_2} + \alpha\lambda s,$$

where $s \in [-1, 1]$. Equating the derivative with 0 and rearranging the terms we get:

$$\left(1 + t(1 - \alpha)\lambda \left(\frac{1}{\|\hat{\theta}_j\|_2} + \frac{1}{\|\hat{\gamma}_j\|_2}\right)\right) \hat{\theta}_j = \tilde{\theta}_j + t\alpha\lambda s.$$

Or equivalently,

$$\left(1 + t(1 - \alpha)\lambda \left(\frac{1}{\|\hat{\theta}_j\|_2} + \frac{1}{\|\hat{\gamma}_j\|_2}\right)\right) \hat{\theta}_j = S(\tilde{\theta}_j, t\alpha\lambda).$$

B. Simulation Details

B.1. Auto-Regressive Correlations AR(1)

Consider the features in order $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_p$. An AR(1) (*Auto-Regressive of order 1*) correlation structure means that the dependence between two columns decays geometrically with their distance:

$$\text{Corr}(X_j, X_k) = \rho^{|j-k|}, \quad j, k \in \{1, \dots, p\},$$

so neighboring columns are correlated by ρ , two-apart by ρ^2 , three-apart by ρ^3 , etc.

- $\text{Corr}(X_j, X_j) = 1$ (with itself),
- $\text{Corr}(X_j, X_{j+1}) = \rho$ (consecutive features),
- $\text{Corr}(X_j, X_{j+2}) = \rho^2$,
- $\text{Corr}(X_j, X_{j+3}) = \rho^3$, and so on.

Equivalently, the correlation matrix for $p = 5$ would be:

$$\Sigma = \begin{bmatrix} 1 & \rho & \rho^2 & \rho^3 & \rho^4 \\ \rho & 1 & \rho & \rho^2 & \rho^3 \\ \rho^2 & \rho & 1 & \rho & \rho^2 \\ \rho^3 & \rho^2 & \rho & 1 & \rho \\ \rho^4 & \rho^3 & \rho^2 & \rho & 1 \end{bmatrix}.$$

For example:

- If $\rho = 0$, the columns are independent.
- If $\rho = 0.8$, adjacent columns have correlation 0.8, two apart $0.8^2 = 0.64$, etc.

B.2. Simulated Data

We summarize how the synthetic datasets with a continuous outcome were created. We generated a grid of scenarios by varying the case-study design matrix known characteristics and the noise level. This design helps us test the algorithm's performance and true covariate recovery ability in various setups allowing us to choose the algorithm that is more likely to generalize better.

Dataset grid: A base size of $n_0 = 500$ and $p_0 = 20$ is used. We generate datasets for all combinations of $n \in \{250, 500, 1000\}$ (multipliers 0.5, 1.0, 2.0), $p \in \{20, 500\}$ (multipliers 1, 25), Gaussian noise standard deviation $\sigma_\varepsilon \in \{0.5, 1.0, 2.0\}$, and AR(1) correlation $\rho \in \{0.0, 0.5, 0.8\}$. We used a fixed seed (42) for replicability. The above yields $3 \times 3 \times 2 \times 3 =$

54 datasets. Each dataset is stored as a CSV file with columns $X_1, \dots, X_p$, the continuous response y , and the noiseless signal μ .

Feature generation: For each (n, p, ρ) , we construct $X \in \mathbb{R}^{n \times p}$ whose rows are i.i.d. $\mathcal{N}(0, \Sigma)$ with AR(1) covariance:

$$\Sigma_{jk} = \rho^{|j-k|}, \quad j, k \in \{1, \dots, p\}.$$

When $\rho = 0$ we draw independent standard normal features.

Active sets $\mathcal{A}$: Ten main-effect indices are drawn uniformly without replacement from $\{1, \dots, p\}$. From these, five indices are chosen to form the *interaction core*, and all $\binom{5}{2} = 10$ pairwise interactions among them are included. Therefore, every interaction involves two variables that are also included as main effects (the strong hierarchy condition applies).

Coefficients: Main-effect coefficients β_j and interaction coefficients γ_{uv} for features in the active set are sampled as

$$\beta_j \sim 0.6 \mathcal{N}(0, 1), \quad \gamma_{uv} \sim 0.5 \mathcal{N}(0, 1).$$

All other coefficients are set to zero. The intercept is fixed at $\alpha = 0$.

Linear predictor (signal): For each sample $i \in \{1, \dots, n\}$ we form the continuous signal

$$\mu_i = \alpha + \sum_{j \in \{1, \dots, p\}} \beta_j X_{ij} + \sum_{(u,v) \in \mathcal{I}} \gamma_{uv} (X_{iu} X_{iv}),$$

where $\mathcal{I}$ denotes the set of selected interaction pairs. Thus μ_i is the noiseless mean response.

Outcome generation: The continuous outcome y_i is obtained by adding Gaussian noise:

$$y_i = \mu_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2), \quad i = 1, \dots, n,$$

with σ_ε given by the selected noise level in the grid.

C. Additional Tables & Figures

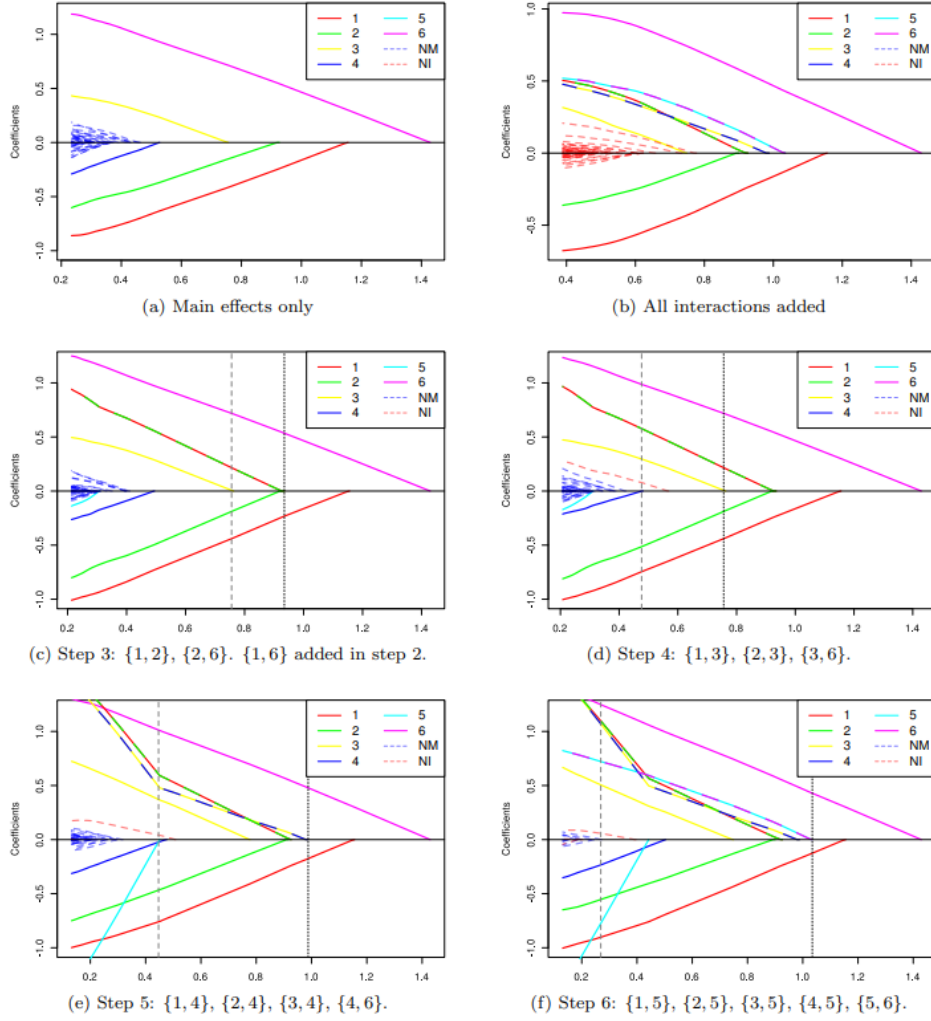


Figure C.1.: Different solution paths created with BT. (a) Solution path with a candidate set including main effects only and solution path of Step 1,2 of BT. (b) Hypothetical solution path with a candidate set including all main effects and all interactions. (a),(b) illustrate the importance of choosing variables for the candidate set for the final model selection. (c)-(f) steps 3-6 of BT. The number-labels is a notation for the corresponding number-predictor (e.g. j -label corresponds to $\mathbf{X}_j$). The NM and NI labels correspond to noise related with main and interaction effects respectively. The vertical dotted line corresponds to λ_{start} . The vertical dashed line corresponds to λ_{add} .

Table C.1.: iFORM: Evaluation Metrics for Main and Interaction Effects. Main refers to main effects and Inter refers to interaction effects.

n	noise sd	p	ρ	Main P	Main R	Main F1	Inter P	Inter R	Inter F1
250	0.5	20	0.0	1.000	0.900	0.947	0.900	0.900	0.900
250	0.5	20	0.5	1.000	1.000	1.000	0.900	0.900	0.900
250	0.5	20	0.8	1.000	1.000	1.000	0.714	0.500	0.588
250	0.5	500	0.0	0.571	0.800	0.667	0.714	1.000	0.833
250	0.5	500	0.5	1.000	0.400	0.571	0.000	0.000	0.000
250	0.5	500	0.8	1.000	0.500	0.667	1.000	0.100	0.182
250	1.0	20	0.0	1.000	0.700	0.824	1.000	0.500	0.667
250	1.0	20	0.5	0.875	0.700	0.778	0.500	0.400	0.444
250	1.0	20	0.8	0.750	0.600	0.667	0.143	0.200	0.167
250	1.0	500	0.0	0.800	0.800	0.800	0.750	0.300	0.429
250	1.0	500	0.5	1.000	0.400	0.571	0.000	0.000	0.000
250	1.0	500	0.8	0.714	0.500	0.588	0.667	0.200	0.308
250	2.0	20	0.0	1.000	0.400	0.571	0.000	0.000	0.000
250	2.0	20	0.5	0.818	0.900	0.857	0.833	0.500	0.625
250	2.0	20	0.8	0.500	0.100	0.167	0.000	0.000	0.000
250	2.0	500	0.0	1.000	0.100	0.182	0.000	0.000	0.000
250	2.0	500	0.5	0.000	0.000	0.000	0.000	0.000	0.000
250	2.0	500	0.8	1.000	0.100	0.182	0.000	0.000	0.000
500	0.5	20	0.0	1.000	0.800	0.889	1.000	0.500	0.667
500	0.5	20	0.5	1.000	1.000	1.000	0.778	0.700	0.737
500	0.5	20	0.8	1.000	0.700	0.824	0.286	0.200	0.235
500	0.5	500	0.0	1.000	0.900	0.947	1.000	0.800	0.889
500	0.5	500	0.5	0.889	0.800	0.842	0.833	0.500	0.625
500	0.5	500	0.8	1.000	1.000	1.000	1.000	0.900	0.947
500	1.0	20	0.0	1.000	1.000	1.000	0.875	0.700	0.778
500	1.0	20	0.5	1.000	0.600	0.750	0.750	0.600	0.667
500	1.0	20	0.8	1.000	0.500	0.667	0.333	0.100	0.154
500	1.0	500	0.0	1.000	0.900	0.947	1.000	0.500	0.667
500	1.0	500	0.5	0.857	0.600	0.706	1.000	0.300	0.462
500	1.0	500	0.8	0.778	0.700	0.737	1.000	0.400	0.571
500	2.0	20	0.0	1.000	0.600	0.750	1.000	0.100	0.182
500	2.0	20	0.5	1.000	0.200	0.333	0.000	0.000	0.000
500	2.0	20	0.8	0.667	0.400	0.500	0.000	0.000	0.000
500	2.0	500	0.0	0.636	0.700	0.667	1.000	0.700	0.824
500	2.0	500	0.5	1.000	0.100	0.182	0.000	0.000	0.000
500	2.0	500	0.8	0.000	0.000	0.000	0.000	0.000	0.000
1000	0.5	20	0.0	1.000	0.900	0.947	0.800	0.800	0.800
1000	0.5	20	0.5	1.000	1.000	1.000	1.000	0.900	0.947
1000	0.5	20	0.8	1.000	1.000	1.000	0.667	1.000	0.800
1000	0.5	500	0.0	1.000	0.700	0.824	1.000	0.200	0.333
1000	0.5	500	0.5	0.385	0.500	0.435	0.500	0.100	0.167
1000	0.5	500	0.8	1.000	0.900	0.947	1.000	0.400	0.571
1000	1.0	20	0.0	0.909	1.000	0.952	0.474	0.900	0.621
1000	1.0	20	0.5	1.000	0.900	0.947	0.700	0.700	0.700
1000	1.0	20	0.8	1.000	1.000	1.000	0.500	0.600	0.545
1000	1.0	500	0.0	0.727	0.800	0.762	0.800	0.400	0.533
1000	1.0	500	0.5	1.000	1.000	1.000	1.000	0.700	0.824
1000	1.0	500	0.8	1.000	0.800	0.889	1.000	0.400	0.571
1000	2.0	20	0.0	1.000	0.900	0.947	0.444	0.400	0.421
1000	2.0	20	0.5	1.000	0.700	0.824	1.000	0.700	0.824
1000	2.0	20	0.8	1.000	0.600	0.750	0.600	0.300	0.400
1000	2.0	500	0.0	1.000	0.800	0.889	1.000	0.500	0.667
1000	2.0	500	0.5	1.000	0.300	0.462	0.000	0.000	0.000
1000	2.0	500	0.8	1.000	0.600	0.750	1.000	0.100	0.182

Table C.2.: BT: Evaluation Metrics for Main and Interaction Effects. Main refers to main effects and Inter refers to interaction effects.

n	noise sd	p	ρ	Main P	Main R	Main F1	Inter P	Inter R	Inter F1
250	0.5	20	0.0	0.643	0.900	0.750	0.050	0.200	0.080
250	0.5	20	0.5	0.769	1.000	0.870	0.107	0.300	0.158
250	0.5	20	0.8	0.667	1.000	0.800	0.083	0.300	0.130
250	0.5	500	0.0	0.109	0.600	0.185	0.000	0.000	0.000
250	0.5	500	0.5	0.136	0.600	0.222	0.000	0.000	0.000
250	0.5	500	0.8	0.186	0.800	0.302	0.000	0.000	0.000
250	1.0	20	0.0	0.556	1.000	0.714	0.000	0.000	0.000
250	1.0	20	0.5	0.500	1.000	0.667	0.000	0.000	0.000
250	1.0	20	0.8	0.500	1.000	0.667	0.157	0.800	0.262
250	1.0	500	0.0	0.163	0.800	0.271	0.000	0.000	0.000
250	1.0	500	0.5	0.158	0.600	0.250	0.000	0.000	0.000
250	1.0	500	0.8	0.145	0.800	0.246	0.000	0.000	0.000
250	2.0	20	0.0	0.526	1.000	0.690	0.031	0.200	0.053
250	2.0	20	0.5	0.625	1.000	0.769	0.000	0.000	0.000
250	2.0	20	0.8	0.556	1.000	0.714	0.050	0.100	0.067
250	2.0	500	0.0	0.182	0.600	0.279	0.000	0.000	0.000
250	2.0	500	0.5	0.065	0.300	0.107	0.000	0.000	0.000
250	2.0	500	0.8	0.308	0.400	0.348	0.000	0.000	0.000
500	0.5	20	0.0	0.500	1.000	0.667	0.050	0.300	0.086
500	0.5	20	0.5	0.588	1.000	0.741	0.000	0.000	0.000
500	0.5	20	0.8	0.529	0.900	0.667	0.042	0.100	0.059
500	0.5	500	0.0	0.075	0.900	0.139	0.000	0.000	0.000
500	0.5	500	0.5	0.085	0.900	0.155	0.000	0.000	0.000
500	0.5	500	0.8	0.095	1.000	0.174	0.000	0.000	0.000
500	1.0	20	0.0	0.526	1.000	0.690	0.000	0.000	0.000
500	1.0	20	0.5	0.500	1.000	0.667	0.030	0.200	0.052
500	1.0	20	0.8	0.500	1.000	0.667	0.033	0.200	0.057
500	1.0	500	0.0	0.073	0.900	0.134	0.000	0.000	0.000
500	1.0	500	0.5	0.084	0.800	0.152	0.000	0.000	0.000
500	1.0	500	0.8	0.231	0.900	0.367	0.000	0.000	0.000
500	2.0	20	0.0	0.818	0.900	0.857	0.000	0.000	0.000
500	2.0	20	0.5	0.500	1.000	0.667	0.083	0.600	0.146
500	2.0	20	0.8	0.500	0.900	0.643	0.040	0.100	0.057
500	2.0	500	0.0	0.118	0.900	0.209	0.000	0.000	0.000
500	2.0	500	0.5	0.058	0.300	0.097	0.000	0.000	0.000
500	2.0	500	0.8	0.111	0.200	0.143	0.000	0.000	0.000
1000	0.5	20	0.0	0.500	0.900	0.643	0.070	0.500	0.123
1000	0.5	20	0.5	0.556	1.000	0.714	0.059	0.200	0.091
1000	0.5	20	0.8	0.500	1.000	0.667	0.000	0.000	0.000
1000	0.5	500	0.0	0.200	0.700	0.311	0.000	0.000	0.000
1000	0.5	500	0.5	0.286	0.600	0.387	0.000	0.000	0.000
1000	0.5	500	0.8	0.114	0.900	0.202	0.000	0.000	0.000
1000	1.0	20	0.0	0.588	1.000	0.741	0.028	0.200	0.049
1000	1.0	20	0.5	0.588	1.000	0.741	0.103	0.400	0.163
1000	1.0	20	0.8	0.714	1.000	0.833	0.000	0.000	0.000
1000	1.0	500	0.0	0.154	0.800	0.258	0.000	0.000	0.000
1000	1.0	500	0.5	0.093	1.000	0.170	0.000	0.000	0.000
1000	1.0	500	0.8	0.123	0.800	0.213	0.000	0.000	0.000
1000	2.0	20	0.0	0.500	1.000	0.667	0.059	0.300	0.098
1000	2.0	20	0.5	0.625	1.000	0.769	0.000	0.000	0.000
1000	2.0	20	0.8	0.526	1.000	0.690	0.000	0.000	0.000
1000	2.0	500	0.0	0.123	1.000	0.220	0.000	0.000	0.000
1000	2.0	500	0.5	0.101	0.700	0.177	0.000	0.000	0.000
1000	2.0	500	0.8	0.196	0.900	0.321	0.000	0.000	0.000

Table C.3.: PL: Evaluation Metrics for Main and Interaction Effects. Main refers to main effects and Inter refers to interaction effects.

n	noise sd	p	ρ	Main P	Main R	Main F1	Inter P	Inter R	Inter F1
250	0.5	20	0.0	0.526	1.000	0.690	0.151	0.800	0.254
250	0.5	20	0.5	0.556	1.000	0.714	0.169	1.000	0.290
250	0.5	20	0.8	0.500	1.000	0.667	0.148	0.900	0.254
250	0.5	500	0.0	0.153	0.900	0.261	0.529	0.900	0.667
250	0.5	500	0.5	0.104	0.800	0.184	0.500	0.700	0.583
250	0.5	500	0.8	0.257	0.900	0.400	0.318	0.700	0.438
250	1.0	20	0.0	0.750	0.900	0.818	0.391	0.900	0.545
250	1.0	20	0.5	0.526	1.000	0.690	0.158	0.900	0.269
250	1.0	20	0.8	0.588	1.000	0.741	0.167	1.000	0.286
250	1.0	500	0.0	0.075	0.700	0.136	0.500	0.600	0.545
250	1.0	500	0.5	0.110	0.800	0.193	0.250	0.600	0.353
250	1.0	500	0.8	0.108	0.800	0.190	0.300	0.600	0.400
250	2.0	20	0.0	0.800	0.800	0.800	0.438	0.700	0.538
250	2.0	20	0.5	0.571	0.800	0.667	0.250	0.700	0.368
250	2.0	20	0.8	0.556	1.000	0.714	0.169	1.000	0.290
250	2.0	500	0.0	0.500	0.100	0.167	0.000	0.000	0.000
250	2.0	500	0.5	0.055	0.300	0.092	0.571	0.400	0.471
250	2.0	500	0.8	0.222	0.200	0.211	0.000	0.000	0.000
500	0.5	20	0.0	0.625	1.000	0.769	0.391	0.900	0.545
500	0.5	20	0.5	0.667	1.000	0.800	0.300	0.900	0.450
500	0.5	20	0.8	0.500	0.900	0.643	0.164	1.000	0.282
500	0.5	500	0.0	0.123	1.000	0.220	0.360	0.900	0.514
500	0.5	500	0.5	0.257	0.900	0.400	0.625	1.000	0.769
500	0.5	500	0.8	0.143	1.000	0.250	0.145	0.800	0.246
500	1.0	20	0.0	0.909	1.000	0.952	0.889	0.800	0.842
500	1.0	20	0.5	0.600	0.900	0.720	0.213	1.000	0.351
500	1.0	20	0.8	0.500	0.800	0.615	0.164	0.900	0.277
500	1.0	500	0.0	0.123	0.900	0.217	0.562	0.900	0.692
500	1.0	500	0.5	0.083	0.700	0.149	0.281	0.900	0.429
500	1.0	500	0.8	0.136	0.900	0.237	0.195	0.800	0.314
500	2.0	20	0.0	0.667	1.000	0.800	0.267	0.800	0.400
500	2.0	20	0.5	0.556	1.000	0.714	0.167	1.000	0.286
500	2.0	20	0.8	0.500	1.000	0.667	0.169	1.000	0.290
500	2.0	500	0.0	0.138	0.800	0.235	0.800	0.800	0.800
500	2.0	500	0.5	0.070	0.700	0.127	0.265	0.900	0.409
500	2.0	500	0.8	0.067	0.600	0.120	0.200	0.900	0.327
1000	0.5	20	0.0	1.000	0.900	0.947	0.692	0.900	0.783
1000	0.5	20	0.5	0.556	1.000	0.714	0.172	1.000	0.294
1000	0.5	20	0.8	0.474	0.900	0.621	0.153	0.900	0.261
1000	0.5	500	0.0	0.086	0.900	0.157	0.213	1.000	0.351
1000	0.5	500	0.5	0.118	1.000	0.211	0.164	0.900	0.277
1000	0.5	500	0.8	0.391	0.900	0.545	0.280	0.700	0.400
1000	1.0	20	0.0	0.909	1.000	0.952	0.643	0.900	0.750
1000	1.0	20	0.5	0.588	1.000	0.741	0.225	0.900	0.360
1000	1.0	20	0.8	0.526	1.000	0.690	0.182	1.000	0.308
1000	1.0	500	0.0	0.100	0.900	0.180	0.270	1.000	0.426
1000	1.0	500	0.5	0.079	1.000	0.147	0.238	1.000	0.385
1000	1.0	500	0.8	0.625	0.500	0.556	1.000	0.300	0.462
1000	2.0	20	0.0	0.588	1.000	0.741	0.286	1.000	0.444
1000	2.0	20	0.5	0.588	1.000	0.741	0.231	0.900	0.367
1000	2.0	20	0.8	0.588	1.000	0.741	0.170	0.800	0.281
1000	2.0	500	0.0	0.101	0.900	0.182	0.412	0.700	0.519
1000	2.0	500	0.5	0.066	0.800	0.122	0.164	0.900	0.277
1000	2.0	500	0.8	0.141	1.000	0.247	0.188	0.900	0.310

microRNA name	Index
hsa_mir_X1a_3p	0
hsa_mir_X2b_5p	1
hsa_mir_X3c_3p	2
hsa_mir_X4d_3p	3
hsa_mir_X5e_5p	4
hsa_mir_X6f_3p	5
hsa_mir_X7g_3p	6
hsa_mir_X8h_5p	7
hsa_mir_X9j_5p	8
hsa_mir_X10j_3p	9
hsa_mir_X11k_3p	10
hsa_mir_X12l_5p	11
hsa_mir_X13m_3p	12
hsa_mir_X14h_5p	13
hsa_mir_X16p_5p	15
hsa_mir_X17q_5p	16
hsa_mir_X18r_3p	17
hsa_mir_X19s_5p	18
hsa_mir_X20t_5p	19
hsa_mir_X21t_5p	20

Table C.4.: Table of microRNA name and index correspondence.